SAPIENZA
UNIVERSITÀ DI ROMA

Centralized Scheduling in Satellite Computing: An LP-based Approach

Faculty of Information Engineering, Computer Science and Statistics
Computer Science

Claudio Bagini
ID number 2045337

Advisor
Prof. Emiliano Casalicchio

Academic Year 2025/2026

Thesis defended on 26 October 2026
in front of a Board of Examiners composed by:

Prof. Emiliano Casalicchio (chairman)
Prof. Danilo Avola
Prof. Daniele De Sensi
Prof. Dorjan Hitaj
Prof. Massimo La Morgia
Prof. Eugenio Nerio Nemmi
Prof. Paolo Zuliani

Centralized Scheduling in Satellite Computing: An LP-based Approach
Bachelor's Thesis. Sapienza University of Rome

© 2026 Claudio Bagini. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Version: October 12, 2026

Author's email: claudio.bagini@gmail.com

To countless cups of coffee, endless iterations,
and the version of me that refused to drop the task.

Abstract

The evolution of Low Earth Orbit (LEO) satellite megaconstellations has opened new perspectives for Satellite Computing, enabling the extension of edge computing capabilities directly into space (Orbital Edge Computing). However, the highly dynamic network topology, stringent energy constraints of satellite nodes, and strict deadlines of real-time applications make task orchestration and offloading critical challenges.

This thesis presents the design, implementation, and experimental evaluation of a centralized orchestrator based on Integer Linear Programming (ILP). To mitigate solver computational costs and prevent starvation, the system integrates an event-driven dynamic batching mechanism controlled by Batch Size (BS) and Batch Timeout (BT) thresholds, along with a coordinated formulation of timing constraints that accounts for queuing delay prior to resolution.

Using discrete-event simulations developed within the SimPy framework, the proposed architecture is evaluated under stress conditions across varying request arrival rates (λ), comparing *Mapping Time* and *Mapping Energy* optimization strategies. The analysis investigates the systemic bottlenecks that emerge under elevated workloads, examining the interplay between physical compute capacity, network saturation, and temporal deadline constraints.

Finally, the centralized orchestrator is benchmarked against state-of-the-art distributed and heuristic solutions (*OrbitAware*, *DTS-base*, *DTS-Optimal*, and distributed ILP) across heterogeneous workload profiles and operational energy budgets. Through this comprehensive evaluation, the thesis aims to characterize the operational boundaries and fundamental trade-offs between the theoretical global optimality of centralized mathematical scheduling and the responsiveness of decentralized models.

Contents

1 Introduction **1**
1.1 Motivation and Context 1
1.2 Problem Statement 2
1.3 Objectives and Main Contributions 3
1.4 Thesis Outline 4

2 Background and State of the Art **5**
2.1 Satellite Computing Paradigms 5
2.2 Task Offloading and Routing in LEO Networks 6
2.2.1 System Entities and Network Topology 6
2.2.2 Workload Profiles and Constraints 7
2.3 Centralized vs. Distributed Orchestration 8
2.3.1 Centralized Orchestration 8
2.3.2 Distributed Orchestration 8
2.4 State-of-the-Art Approaches 8
2.4.1 Distributed Heuristic Frameworks 9
2.4.2 Optimization-Based and Distributed ILP Formulations 9
2.5 Research Gap and Thesis Positioning 9

3 System Architecture, Mathematical Formulation, and Implementation **11**
3.1 Regional Constellation Architecture and Offloading Model 11
3.1.1 The Decoupled Three-Phase Lifecycle 12
3.2 Event-Driven Dynamic Batching Engine 13
3.2.1 State Consistency and Discrete-Event Synchronization 13
3.3 Constellation Topology Mapping and Metric Normalization 16
3.3.1 Dimensionless Link Cost Normalization 16
3.4 Centralized Integer Linear Programming Formulation 19
3.4.1 Physical Delay, Energy, and Orbital Eclipse Modeling 19
3.4.2 Multi-Objective Function and System Constraints 20
3.4.3 Solver Resolution and Cascade Rejection Diagnostics 21
3.5 Task Execution and Resilient Hybrid Routing 23
3.5.1 Dispatching and Buffer Re-Enqueuing Pipeline 23
3.5.2 Resilient Hybrid Packet Routing 24
3.6 Computational Complexity and Scalability Analysis 25
3.6.1 Topology Mapping and Pathfinding Complexity 25

3.6.2 ILP Formulation and Branch-and-Cut Solver Complexity . . 26
3.6.3 Hybrid Routing and Forwarding Complexity 27
3.6.4 Constellation Scalability and Telemetry Overhead 27

4 Performance Analysis, Optimization, and Experimental Evaluation 28
4.1 Simulation Environment and Methodological Setup 29
4.1.1 Constellation Topology and Scenario Parameters 29
4.1.2 Two-Tier Optimization Methodology 29
4.2 Macro-Tuning: Exploration and Selection of Batching Policies 30
4.2.1 Success Rate Dynamics 30
4.2.2 Diagnostics of Failure Causes 33
4.2.3 Response Time Decomposition 38
4.2.4 Selection of Viable Batching Archetypes 41
4.3 Micro-Tuning: Multi-Objective Weight Calibration under Stress ($\gamma \times \alpha$) 43
4.3.1 Impact on Success Rate 43
4.3.2 Failure Diagnostics and Resource Bottlenecks 46
4.3.3 Latency Inflation and Path-Bloat Dynamics 49
4.3.4 Selection of Calibrated Objective Weights 51
4.4 Consolidated Validation: Pre- vs. Post-Optimization Benchmark . . 51
4.4.1 Success Rate Recovery and Mapping Divergence 52
4.4.2 Rejection Causes Evolution 53
4.4.3 Response Time Containment and Execution Flattening . . . 54
4.5 Summary of Findings and Architectural Readiness 55

5 Comparative Benchmark 57
5.1 Experimental Setup and Baseline Algorithms 57
5.1.1 Benchmark Dimensioning and Scenario Parameters 57
5.1.2 Evaluated Algorithms and State-of-the-Art Baselines 58
5.2 Comparative Performance Evaluation across Workload Profiles . . . 59
5.2.1 Mixed Multi-Service Workload Profile ($\boldsymbol{\beta} = (0.40, 0.35, 0.25)$) 59
5.2.1.1 Success Rate Dynamics 61
5.2.1.2 Response Time Analysis 62
5.2.1.3 Failure Diagnostics 64
5.2.2 Computation-Heavy Stress: CPU-Intensive Profile ($\boldsymbol{\beta} = (0, 1, 0)$) 65
5.2.2.1 Success Rate Dynamics and Invariance across Payload Distributions 67
5.2.2.2 Response Time Analysis 67
5.2.2.3 Failure Diagnostics and Bottleneck Analysis 69
5.2.3 Communication-Heavy Stress: Data-Intensive Profile ($\boldsymbol{\beta} = (0, 0, 1)$) 71
5.2.3.1 Success Rate Dynamics and Optimality Supremacy 71
5.2.3.2 Response Time Analysis 73
5.2.3.3 Failure Diagnostics and Link Saturation 74
5.3 Trade-Off Analysis and Operational Discussion 76
5.3.1 Theoretical Global Optimality vs. Ingress Queuing Latency . 76
5.3.2 Energy Budget Sensitivity Across Dimensional Boundaries . . 77
5.3.3 Fast-Dispatch vs. Controlled Small-Batching Orchestration . 77

5.3.4 Architectural Synthesis and Deployment Guidelines 78
5.4 Summary of Chapter Findings 78

6 Conclusions and Future Work **80**
6.1 Synthesis of Contributions and Research Question Resolution 80
6.1.1 Answering the Core Research Question 80
6.1.2 Algorithmic Archetypes and Workload Sensitivity 82
6.2 Critical Limitations of the Centralized Approach 82
6.2.1 Ingress Buffering and the Batching Dilemma 83
6.2.2 Single Point of Failure and Space Hazards 83
6.2.3 State Telemetry Overhead and Information Staleness 83
6.3 Future Research Directions 83
6.3.1 Two-Tier Hybrid Orchestration Architectures 83
6.3.2 Traffic-Aware Adaptive Batching via Reinforcement Learning 84
6.3.3 Multi-Agent Distributed Reinforcement Learning (MADRL) . 84
6.3.4 Hardware-in-the-Loop (HIL) Prototyping on Space-Qualified Hardware 84
6.4 Concluding Remarks 85

Bibliography **86**

Acknowledgments **89**

Chapter 1

Introduction

In recent years, space communications and satellite networks have undergone a monumental paradigm shift. The rapid miniaturization of space components, combined with drastically reduced launch costs, has catalyzed the deployment of Mega-Constellations operating in Low Earth Orbit (LEO) [1]. Traditionally, satellite architectures functioned strictly under a "bent-pipe" model, where satellites served as passive transparent relays forwarding raw telemetry or communication signals back to Ground Stations (GS) for processing. However, the unprecedented volume of data produced by modern spaceborne applications—such as high-resolution Earth Observation (EO) [2], environmental monitoring, and ubiquitous Internet of Things (IoT) sensing [3]—has rendered this traditional pipeline highly inefficient.

To overcome the severe bandwidth limits of space-to-ground downlinks and the high propagation delays associated with remote terrestrial processing, the paradigm of **Orbital Edge Computing (OEC)**—also broadly termed Satellite Computing (SC)—has emerged [4, 5]. By equipping LEO satellites with onboard processing hardware, data can be computed directly in space close to the source, transmitting only actionable insights or highly compressed results to ground users. Despite its immense potential, executing compute-intensive tasks on resource-constrained satellite nodes introduces complex orchestration challenges that require advanced decision-making frameworks [1, 6].

1.1 Motivation and Context

The transition towards Orbital Edge Computing is driven by two main technological trends: the exponential increase in space-generated data and the strict temporal requirements of next-generation applications. Applications such as real-time disaster response, autonomous maritime navigation, and defense surveillance demand sub-second latency for task execution and reporting [2]. Processing these tasks on Earth introduces unavoidable latency bottlenecks due to intermittent Ground Station contact windows, dynamic weather interference, and severe downlink congestion.

LEO megaconstellations (such as Starlink, OneWeb, and Kuiper) consist of hundreds to thousands of satellites orbiting at altitudes between 300 km and 2000 km. These satellites move at high orbital velocities (approximately 7.5 km/s relative to the Earth's surface), resulting in a continuously changing network topology. Inter-

Satellite Links (ISLs)—utilizing high-throughput Free-Space Optics (FSO) or Radio Frequency (RF) channels—enable direct communication between adjacent satellites without routing signals down to the ground [7, 8].

Consequently, a LEO constellation effectively forms a distributed, high-velocity edge computing network in orbit. However, executing applications across this orbital fabric requires intelligent **Task Offloading and Routing Orchestration** [9, 10]. Ground users or source satellites generate computational tasks that must be dynamically assigned to suitable processing nodes across the constellation. Deciding *where* and *when* to execute a given task, while routing its associated payload across multiple dynamic ISL hops, constitutes a high-dimensional optimization problem that directly governs overall network performance.

1.2 Problem Statement

Orchestrating task offloading in Orbital Edge Computing environments presents a unique set of constraints and operational challenges that set it apart from conventional terrestrial cloud/edge systems [6]:

- **Severe Energy Budget Limits:** Satellite Edge Nodes (SENs) rely on solar panels and onboard battery storage. Computationally intensive task execution and RF/optical transmissions consume substantial power. Over-utilizing specific nodes leads to rapid battery depletion, thermal overhead, or total operational failure [6].
- **Heterogeneous and Limited Hardware Resources:** Onboard processing units (e.g., embedded CPUs, specialized AI accelerators) are constrained by strict Space, Weight, and Power (SWaP) requirements, as well as radiation-hardening constraints [5].
- **Dynamic Topology and Intermittent Connectivity:** High orbital mobility leads to frequent topology reconfigurations, changing link distances, and periodic loss of line-of-sight. Task routing paths must be continuously recalculated to avoid dropped links [7].
- **Strict Temporal Constraints and Queuing Delays:** Tasks possess strict deadlines (D_r). If the sum of transmission latency, queuing delay, and computation time exceeds D_r, the task fails, resulting in wasted system energy and degraded quality of service.

To achieve theoretical optimality in task placement and energy management, mathematical optimization tools such as **Integer Linear Programming (ILP)** are commonly employed [11, 12]. An ILP formulation can search the joint solution space of task routing, CPU allocation, and energy minimization to find globally optimal schedules.

However, centralized ILP solvers suffer from inherent computational complexity (NP-hard) [11]. In dynamic satellite networks with continuously arriving workloads, solving an ILP instance for every individual task request introduces unacceptable

computational overhead and processing latency at the central orchestrator. Conversely, accumulating task requests to solve them in batches introduces a trade-off: larger batches allow the ILP solver to achieve superior global resource allocation, but prolonged waiting times in orchestrator queues increase the risk of task deadline expiration (*Deadline Exceeded* failures).

1.3 Objectives and Main Contributions

The main objective of this thesis is to design, implement, and rigorously evaluate a **Centralized ILP Orchestration Framework** optimized for task routing and resource allocation in LEO satellite networks, incorporating a **Dynamic Batching Mechanism** to balance solver computational complexity with network queuing delay.

Specifically, this work addresses the key question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

The main contributions of this thesis are summarized as follows:

1. **Formalization of the Centralized ILP Optimization Model:** We formulate a comprehensive ILP mathematical model that jointly optimizes task routing across Inter-Satellite Links and computational resource allocation on Satellite Edge Nodes. The model supports dynamic weighting between latency optimization (*Mapping Time*) and total energy preservation (*Mapping Energy*).

2. **Design of an Integrated Dynamic Batching & Timeout Mechanism:** To mitigate the computational overhead of solving the ILP model continuously, we integrate an adaptive batching policy governed by Batch Size (BS) and Batch Timeout (BT). Furthermore, we propose a mathematical deadline compensation formula to adjust task deadlines based on orchestrator queuing duration (T_{batch}).

3. **Design and Integration of the Centralized Orchestration Layer:** We design and implement the centralized ILP orchestration module and dynamic batching logic, integrating them into an existing Python/`SimPy` discrete-event simulation framework [13, 14]. The resulting environment accurately captures request arrival dynamics (λ), inter-satellite communication links, queue management, hardware processing, and fleet energy consumption under heterogeneous workload conditions.

4. **In-Depth Performance and Bottleneck Analysis:** We perform extensive stress testing under high workload arrival rates to identify system collapse thresholds, distinguishing between physical resource saturation (CPU/Network capacity) and temporal violations (*Deadline Exceeded*). We also conduct sensitivity analyses regarding deadline relaxation (ΔD).

5. **Comparative Benchmark Against State-of-the-Art Solutions:** We evaluate the centralized ILP orchestrator against established baseline heuristics (*OrbitAware*, *DTS-base*, *DTS-Optimal*) and distributed ILP formulations

from the literature [6], establishing clear operational trade-offs regarding task completion rate, overall latency, and energy efficiency.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

- **Chapter 2 (Background and State of the Art):** Reviews the fundamental concepts of Satellite Computing, LEO network dynamics, and existing literature on task offloading. It provides a taxonomy of centralized versus distributed orchestration schemes and details key reference algorithms.
- **Chapter 3 (System Architecture and ILP Formulation):** Presents the formal mathematical system model, decision variables, objective functions (*Mapping Time* vs. *Mapping Energy*), constraints, and the dynamic batching/timeout mechanisms.
- **Chapter 4 (Performance Analysis and Evaluation):** Describes the `SimPy` discrete-event experimental setup and evaluates the proposed centralized orchestrator under varying batching configurations, heavy-load conditions, and deadline relaxation scenarios.
- **Chapter 5 (Comparative Benchmark):** Details the comparative analysis benchmarking our centralized orchestrator against state-of-the-art distributed heuristics and formulations, highlighting key trade-offs in latencies, energy consumption, and global optimality.
- **Chapter 6 (Conclusions and Future Work):** Summarizes the main findings of this research, outlines limitations of centralized orbital orchestration, and proposes future research directions toward hybrid adaptive architectures.

Chapter 2

Background and State of the Art

The rapid proliferation of Low Earth Orbit (LEO) satellite constellations has fundamentally redefined the landscape of satellite communications and space-borne computing [1]. Transitioning from traditional passive data relays to active processing nodes requires a comprehensive understanding of orbital dynamics, task offloading models, and orchestration strategies [4, 5]. This chapter provides the theoretical foundation and contextual literature for this thesis. It details the paradigms of Orbital Edge Computing (OEC), analyzes task offloading and routing dynamics in LEO networks, compares centralized and distributed orchestration models, reviews state-of-the-art baseline solutions, and articulates the research gap addressed by our work.

2.1 Satellite Computing Paradigms

Space-based networking architectures have historically been categorized by their orbital altitudes: Geostationary Earth Orbit (GEO), Medium Earth Orbit (MEO), and Low Earth Orbit (LEO) [1]. GEO satellites, positioned at approximately $35,786$ km, offer stationary coverage over vast geographic areas but suffer from high round-trip propagation delays (exceeding 250 ms) and high free-space path loss. MEO satellites ($2,000$ km $- 35,786$ km) reduce latency slightly but remain unsuitable for ultra-low-latency processing.

In contrast, LEO satellites operate at altitudes ranging from 300 km to $2,000$ km. Their proximity to Earth drastically lowers propagation latency ($10 - 40$ ms round-trip) and reduces power loss, making them ideal for high-throughput, real-time communications [7, 2]. However, LEO satellites travel at extreme velocities relative to the surface (~ 7.5 km/s), completing an orbit in approximately $90 - 120$ minutes. This rapid mobility results in dynamic topologies where individual satellites remain in line-of-sight with a fixed ground position for only a few minutes [3].

To process data efficiently within this dynamic environment, satellite network architectures have evolved through three primary paradigms [4, 1]:

1. **Transparent Bent-Pipe Relay:** Satellites act merely as transparent repeaters. Raw data captured by Earth Observation (EO) sensors or transmitted by Ground Users (GUs) is amplified and forwarded directly to Ground Stations (GSs). Computation occurs entirely in terrestrial cloud centers. This

model suffers from severe downlink bottlenecks and complete connectivity loss when flying over remote regions (e.g., oceans or polar zones) lacking GS infrastructure [2].

2. **Terrestrial Cloud Offloading with Inter-Satellite Routing:** Satellites utilize Inter-Satellite Links (ISLs) to route raw data across orbital planes until a satellite with an active GS connection is reached. While this mitigates local GS coverage gaps, the volume of raw data still strains optical and RF link bandwidth [7, 8].

3. **Orbital Edge Computing (OEC) / Satellite Edge Computing (SEC):** Satellites are equipped with onboard processing units—termed Satellite Edge Nodes (SENs)—capable of executing compute-intensive algorithm payloads (e.g., AI inference, image compression, signal filtering) directly in orbit [5, 10]. Raw data is processed locally at the source node or offloaded across ISLs to adjacent SENs. Only refined, actionable insights or small computation results are downlinked to Earth, drastically reducing bandwidth consumption and end-to-end response times [3, 15].

2.2 Task Offloading and Routing in LEO Networks

Task offloading in Orbital Edge Computing involves receiving task execution requests generated by Ground Users (GUs) or space-borne sensors and strategically allocating their computational execution across available SENs while routing payloads through Inter-Satellite Links (ISLs) [9, 6].

2.2.1 System Entities and Network Topology

An OEC network architecture consists of three core structural entities [6, 16]:

- **Ground Users (GUs) / Remote Sensors:** Distributed terrestrial entities that issue computational task requests r. Each request is characterized by an input data payload size s_r (in megabytes), a computational execution demand $d_{cpu,r}$ (in seconds), and a strict operational deadline D_r (in seconds) [6].

- **Satellite Edge Nodes (SENs):** LEO satellites equipped with CPU cores, memory modules, and onboard energy reserves (solar panels and battery banks). Each SEN s_j possesses a processing capacity f_j (cycles/second) and an operational energy budget E_j [6].

- **Inter-Satellite Links (ISLs):** High-speed communication channels connecting adjacent satellites. Intra-plane ISLs connect satellites within the same orbital plane (stable distance), whereas inter-plane ISLs connect satellites in adjacent planes (dynamically varying distance and pointing angles) [7, 8]. Each ISL link (u, v) is constrained by bandwidth B_{uv} and propagation delay τ_{uv}.

2.2.2 Workload Profiles and Constraints

Unlike conventional terrestrial cloud datacenters operating on uniform, high-throughput fabrics, Orbital Edge Computing must accommodate highly heterogeneous task demands under severe physical boundaries [6]. In spaceborne environments, computational workloads are fundamentally characterized by their temporal tolerance and their resource footprint [4, 2]:

- **Real-Time, Mission-Critical Workloads:** Time-sensitive applications—such as real-time disaster detection, defense alerts, and active maritime tracking—demanding sub-second or deterministic response latencies ($D_r \leq 1.0$–2.0 s) [2]. For these services, queuing delays or multi-hop routing stalls directly jeopardize operational utility.
- **Batch and Delay-Tolerant Workloads:** Routine background processes, including scientific telemetry compression, orbital ephemeris synchronization, and non-urgent Earth Observation indexing [4]. These tasks tolerate deferred execution or multi-hop store-and-forward routing, running during operational lulls when satellite resources are underutilized.

Orthogonally to their temporal deadline, workloads impose asymmetric resource stress across the orbital infrastructure [6]: *computation-heavy* tasks demand extensive CPU execution cycles on the processing unit, whereas *data-intensive* transfers require transmitting multi-megabyte payloads across Inter-Satellite Links, driving network interface and transceiver saturation [6].

Consequently, task offloading and scheduling decisions must jointly satisfy four coupled systemic constraints [9, 6]:

1. **Computational Capacity:** The aggregate processing demand scheduled on satellite node j cannot exceed its physical onboard compute frequency f_j and memory limits, avoiding processor starvation and excessive queue backlog.
2. **Network Link Bandwidth:** Payload traffic routed across directed Inter-Satellite Link (u, v) is strictly bounded by the available transceiver bandwidth B_{uv}, preventing packet buffer overflows and queue-induced latency inflation.
3. **Fleet Energy Budgets:** Because satellites rely on cyclic photovoltaic recharging and finite onboard battery storage (E_j), the cumulative energy expended for RF/optical transmission (P_{tx}) and dynamic CPU switching (P_{comp}) must remain within available battery thresholds per orbital pass.
4. **Strict End-to-End Service Deadlines:** The end-to-end task completion latency—spanning ingress queuing, multi-hop uplink ISL forwarding (T_{up}), onboard queuing and execution (T_{cpu}), and downstream result return (T_{down})—must strictly comply with the application deadline:

$$T_{total}(r) = T_{up}(r) + T_{cpu}(r) + T_{down}(r) \leq D_r \tag{2.1}$$

   Exceeding D_r triggers immediate task drops (*Deadline Exceeded*), wasting both scarce constellation energy and orbital link capacity.

2.3 Centralized vs. Distributed Orchestration

Managing resource allocation across hundreds of moving SENs requires an orchestration paradigm [1]. Literature categorizes these paradigms into two primary architectures: Centralized and Distributed.

2.3.1 Centralized Orchestration

In a centralized orchestration architecture, a dedicated master entity—such as a designated lead satellite (Orbital Orchestrator) or a ground control station—collects global network state information (CPU load, battery state, link traffic) from all SENs [1, 6].

- **Advantages:** The central orchestrator maintains complete global visibility over all network resources, enabling mathematically optimal decision-making. Global optimization models (e.g., Integer Linear Programming) can eliminate load imbalances, prevent local hotspots, and minimize network-wide energy consumption [11, 12].
- **Disadvantages:** Centralized schemes incur significant control signaling overhead to aggregate state information across the constellation. Furthermore, routing all requests to a central orchestrator creates queuing delays and bottlenecks under high request arrival rates (λ) [6]. Most critically, relying on a centralized controller introduces a Single Point of Failure (SPOF): if the orchestrator experiences hardware faults, orbital visibility loss, or link outages, the entire scheduling and task-handling pipeline is compromised, severely undermining system fault tolerance [5].

2.3.2 Distributed Orchestration

In distributed orchestration, individual SENs or localized clusters make autonomous offloading and routing decisions based solely on locally observed state information or partial neighborhood telemetry [17, 6].

- **Advantages:** Distributed schemes eliminate centralized control bottlenecks, offer sub-millisecond decision latency, and demonstrate high fault tolerance against dynamic topology changes and satellite failures [6].
- **Disadvantages:** Lacking global visibility, distributed algorithms often converge to sub-optimal solutions. They are prone to localized network congestion, unbalanced CPU utilization, and premature battery depletion on popular forwarding paths [18, 9].

2.4 State-of-the-Art Approaches

To place our research in context, we review key baseline algorithms and mathematical models established in recent IEEE literature on Orbital Edge Computing and Task Offloading.

2.4.1 Distributed Heuristic Frameworks

Heuristic approaches sacrifice theoretical optimality in favor of computational speed and low decision latency [6]:

- **OrbitAware Architecture:** A dynamic heuristic framework designed specifically for highly dynamic LEO topologies. *OrbitAware* leverages periodic orbital trajectory updates to predict link availability and select shortest processing/routing paths locally, reducing control overhead at the cost of potential link queue saturation [6].
- **DTS-base (Dynamic Task Scheduling - Base):** A greedy distributed scheduling baseline that assigns incoming tasks to the nearest available SEN with sufficient idle CPU capacity. While fast, it ignores global energy budgets and multi-hop link traffic [6].
- **DTS-Optimal (Dynamic Task Scheduling - Access Point Optimized):** An enhanced version of *DTS-base* that incorporates multi-criteria decision metrics, optimizing both the local access point selection and initial ISL hop bandwidth before dispatching tasks [6].

2.4.2 Optimization-Based and Distributed ILP Formulations

To improve allocation quality, recent studies have formulated mathematical optimization models using Integer Linear Programming (ILP) and mixed-integer techniques [11, 12]:

- **Distributed / Hierarchical ILP:** Frameworks that decompose a global ILP problem into smaller sub-problems solved locally across satellite clusters [6]. While more scalable than pure centralized ILP, distributed ILP approaches suffer from convergence delays and coordination overhead during inter-cluster task transfers [19].
- **Weighted Objective Functions:** Optimization models in the literature frequently balance competing goals—such as latency minimization versus energy preservation—by assigning scalar weights (W_{lat}, W_{en}) to objective terms [9, 6].

2.5 Research Gap and Thesis Positioning

While state-of-the-art distributed heuristics offer rapid processing times and distributed ILP frameworks attempt local decomposition, a significant research gap remains regarding the practical limits of **Centralized ILP Orchestration** in LEO satellite networks [6, 1].

Existing literature frequently assumes either ideal zero-latency centralized orchestrators or dismisses centralized optimization entirely due to its computational complexity. Few works rigorously examine the trade-offs when a centralized ILP orchestrator is equipped with an integrated **Dynamic Batching** and **Timeout Mechanism** (BS/BT) to manage request queues under heavy workloads ($\lambda > 6$ req/s) [6].

This thesis addresses this exact gap by:

1. Formalizing a centralized ILP model that dynamically toggles between latency minimization (*Mapping Time*) and energy conservation (*Mapping Energy*) [6].
2. Quantifying the systemic performance bounds of centralized optimization when coupled with dynamic request batching and queuing delay compensation.
3. Conducting a rigorous comparative benchmark against established state-of-the-art heuristics (*OrbitAware*, *DTS-base*, *DTS-Optimal*) and distributed ILP variants under realistic discrete-event simulation conditions (`SimPy`) [13, 6].

Chapter 3

System Architecture, Mathematical Formulation, and Implementation

This chapter provides a comprehensive dissertation on the theoretical framework, mathematical optimization formulation, and concrete software architecture of the proposed Centralized Integer Linear Programming (ILP) Orchestrator. In satellite edge computing, theoretical formulations often suffer from an abstraction penalty: static graph models fail to capture the physical reality of rapid orbital kinematics, fluctuating inter-satellite links, and discrete queue dynamics. To bridge this gap, each section systematically couples the formal analytical modeling with its concrete algorithmic realization. The full simulation environment is developed in Python using the `SimPy` discrete-event engine [13] and the `PuLP` mathematical programming library [14] backed by the open-source `HiGHS` branch-and-cut solver [20]. The complete, documented source code is publicly accessible in the reference repository [21].

The remainder of this chapter is structured as follows. Section 3.1 formalizes the regional Walker-Delta constellation, the offloading task abstraction, and the decoupled three-phase operational circuit. Section 3.2 details the event-driven batching engine designed to throttle solver invocation while enforcing bounded delay. Section 3.3 introduces the multi-source topology exploration and dimension-free metric normalization scheme. Section 3.4 derives the multi-objective ILP formulation, its physical constraints, and the cascade diagnostic engine for dropped tasks. Section 3.5 elaborates on task dispatching, buffer re-enqueuing, and the hybrid routing protocol combining planned ILP paths with reactive greedy geometric recovery. Finally, Section 3.6 provides an asymptotic complexity and scalability analysis covering runtime footprint and control telemetry overhead.

3.1 Regional Constellation Architecture and Offloading Model

We model a regional Low Earth Orbit (LEO) satellite network operating within a Walker-Delta orbital constellation [16, 1]. LEO satellites traverse circular orbits

at altitudes ranging between 500 km and 1200 km, resulting in orbital velocities exceeding 7.5 km/s and orbital periods of roughly 90–100 minutes. The operational network comprises a set of candidate Satellite Edge Nodes (SENs), denoted by $S = \{1, 2, \ldots, n\}$, and a set of regional Access Points (APs) or satellite ground stations, denoted by AP_{global} [6]. Each SEN is equipped with compute capabilities, a finite battery reserve, and inter-satellite communication transceivers. Within the observed geographical region, a designated coordinator acts as the centralized *Master Orchestrator*. The Master maintains real-time visibility over constellation kinematics, dynamic processor queue occupancies, residual energy budgets, and active Inter-Satellite Link (ISL) topologies. The orchestration logic and event management are implemented in the `Orchestrator` class, while the physical state of orbital nodes is encapsulated in `EdgeServer.py` [21].

Ground Users (GUs) deploy compute-intensive applications (e.g., Earth observation inference, disaster response imagery processing) whose requests arrive dynamically at regional gateways. Each offloading request $r \in R$ is represented by a five-element tuple:

$$r = (d_{cpu,r}, D_r, s_r, t_{arr}, AP_r) \tag{3.1}$$

where $d_{cpu,r}$ is the computational workload execution demand (in seconds), D_r represents the maximum allowable service processing deadline, s_r denotes the input dataset payload size (in MB), t_{arr} is the arrival timestamp at the entry node, and $AP_r \in AP_{global}$ denotes the originating ground access point [6].

3.1.1 The Decoupled Three-Phase Lifecycle

Due to the extreme relative velocity between the LEO space segment and ground reference points, the topology of the network is inherently time-varying. A satellite visible to a ground station at arrival timestamp t_{arr} may traverse its elevation cone and disappear behind the local horizon within 5–10 minutes. To prevent solver stale-state failure and guarantee deterministic execution, the lifecycle of each offloaded task r is decoupled into three distinct operational phases:

1. **Uplink Multi-Hop Forwarding Phase (T_{up}):** The originating gateway AP_r injects the task input payload s_r into the orbital network. The payload is forwarded across a multi-hop ISL path $\vec{P}_{up}(AP_r, i)$, traversing $h_{up}(r, i)$ intermediate satellite hops until it reaches the selected compute node $i \in S$ [6].

2. **On-Board Computational Execution Phase (T_{cpu}):** Upon arrival at satellite i, the task is enqueued into the local processor schedule. The overall execution delay encompasses both the queuing wait time $W_{cpu,i}$ and the physical compute duration $d_{cpu,r}$ [4, 6].

3. **Post-Execution Geographic Result Delivery (T_{down}):** Once computation terminates, the resulting output payload must be transferred back to the originating ground gateway AP_r.

Design Rationale for Algorithmic Decoupling. In traditional terrestrial edge computing, dispatchers optimize both forward and return paths simultaneously. In LEO networks, however, the latency required to execute long-running tasks

($W_{cpu,i} + d_{cpu,r} \gg 10$ s) frequently exceeds the stability window of individual laser ISL alignments. Jointly planning the downlink path at time t_{arr} would force the optimizer to rely on long-range topological predictions that are vulnerable to transient link disruptions, orbital plane crossings, and ground station visibility loss. Consequently, the centralized ILP orchestrator restricts its search space to the joint optimization of satellite candidate selection, uplink ISL forwarding, energy conservation, and deadline compliance. The return phase (T_{down}) is delegated to a distributed, autonomous geographic greedy routing protocol executed on-demand at execution completion time.

Table 3.1 compiles the primary mathematical notations, parameters, and variable definitions utilized throughout the mathematical model and software implementation.

3.2 Event-Driven Dynamic Batching Engine

Formulating and solving an NP-hard Mixed-Integer Linear Program for every incoming task arrival introduces prohibitive computational latency. Under high offloading request rates λ, instantaneous per-task optimization causes compute-orchestrator starvation: the time spent by the Master node searching the branch-and-bound tree exceeds the task inter-arrival time, resulting in an unstable queuing regime [11, 6]. Conversely, static time-window batching introduces artificial buffering delay, potentially causing deadline violations for latency-sensitive tasks during light traffic periods.

To reconcile these competing operational demands, the Master Orchestrator implements an event-driven dynamic batching mechanism within a `SimPy` discrete-event environment [13]. The engine is parameterized by two complementary thresholds:

- **Maximum Batch Size (BS):** A capacity ceiling that limits the cardinality of the task set $|R|$ submitted to the solver. Bounding $|R|$ bounds the binary variable space $|R| \times |S|$, establishing a deterministic upper limit on branch-and-cut execution time.
- **Batch Timeout (BT):** A temporal window that bounds the maximum buffering delay experienced by any single request, preventing starvation during low-throughput operational phases.

3.2.1 State Consistency and Discrete-Event Synchronization

In a discrete-event simulation model, physical processes (e.g., satellite movement, inter-satellite distance variations, queue servicing) evolve concurrently across distinct simulation threads. Satellite coordinates and elevation angles are updated dynamically according to orbital ephemeris [22]. If the orchestrator were to capture constellation telemetry immediately upon the satisfaction of a batch trigger, pending thread events scheduled at the exact simulation time t might remain uncommitted, leading to stale queue readings or inconsistent link states.

To enforce strict state consistency, the orchestrator introduces a zero-delay event barrier:

$$\textbf{yield} \quad \textsc{Timeout}(env, 0) \tag{3.2}$$

Table 3.1. Summary of mathematical notations and system parameters used in the Centralized ILP Orchestrator model.

Category	Symbol	Description
Sets & Topology	R	Set of task requests accumulated in the active batch ($\|R\| \leq BS$).
	S	Set of candidate Satellite Edge Nodes (SENs) in the regional constellation.
	AP_{global}	Set of active regional Access Points / satellite gateways.
	AP_r	Originating Access Point for computational task request r.
	$\vec{P}_{up}(AP_r, i)$	Pre-computed shortest multi-hop ISL path from AP_r to satellite i.
Task Requests	r	Task tuple: $r = (d_{cpu,r}, D_r, s_r, t_{arr}, AP_r)$.
	$d_{cpu,r}$	Computational workload processing demand (execution duration in seconds).
	D_r	Maximum allowable end-to-end service completion deadline.
	s_r	Input dataset payload size (in MB; $s_r^{bytes} = s_r \cdot 1024^2$).
	t_{arr}	Task generation timestamp at the Access Point.
	S_{ref}	Reference link payload for topological Dijkstra normalization (1 MB).
Network Links	$\tau_{u,v}$	Propagation delay of the directed link (u, v).
	$B_{u,v}$	Available transmission bandwidth of link (u, v) (in Bytes/s).
	P_{net}	Radio frequency / optical laser transmission power coefficient (P_{trasm}).
	$c_{link}(u, v)$	Normalized dimensionless cost of link (u, v) in Dijkstra mapping.
	w_R, w_e	Routing cost strategy weights for delay and energy ($w_R + w_e = 1$).
	$W_{net,u}$	Network queue waiting time at intermediate forwarding node u.
Satellite Nodes	$C_{sen,i}$	Processor clock frequency / compute capacity of satellite i.
	κ_i	Dynamic CPU energy switching capacitance coefficient (e_coeff).
	B_i	Usable residual battery energy budget of satellite i.
	$W_{cpu,i}$	Estimated queuing delay in the onboard processor execution queue.
	$t_{sunset,i}$	Solar visibility horizon / orbital eclipse timestamp for satellite i.
	Ω_i	Normalized orbital eclipse penalty factor.
Optimization & ILP	$x_{r,i}$	Binary decision variable (1 if task r is assigned to SEN i; 0 otherwise).
	y_r	Binary rejection variable (1 if task r is dropped by the solver; 0 otherwise).
	$R_{r,i}$	Total predicted response time for routing and executing task r on satellite i.
	$\epsilon_{r,i}$	Fleet cumulative energy expenditure for routing and executing task r on i.
	P_{drop}	Heavy penalty scalar applied to rejected tasks ($P_{drop} = 10^6$).
	α, β, γ	Multi-objective hierarchical weighting factors (Time, Energy, Sunset).
	BS, BT	Batch Size and Batch Timeout control thresholds for dynamic batching.

This operational step temporarily relinquishes thread execution back to the `SimPy` event scheduler. The engine flushes and commits all physical updates occurring at time t across the entire constellation before the orchestrator initiates graph extraction and solver instantiation.

Algorithm 1 formalizes the dual lifecycle of task ingestion and batch dispatching, corresponding to the `add_task`, `_run_loop`, and `_process_batch` methods of `Orchestrator.py` [21].

Algorithm 1 Event-Driven Batch Ingestion and Orchestrator Lifecycle

Require: Simulation environment env, buffer capacity BS, timeout duration BT
Ensure: Aggregation of incoming tasks into batches and invocation of optimization pipeline

Global State: $\mathcal{B} \leftarrow [\,]$, $E_{arrival} \leftarrow$ CreateEvent(env), $E_{ready} \leftarrow$ CreateEvent(env)

procedure AddTask($task_data, task_id, visible_aps$) ▷ Invoked upon task arrival
  if $\mathcal{B} = [\,]$ **and** $\neg$IsTriggered($E_{arrival}$) **then**
    TriggerEvent($E_{arrival}$) ▷ Signal arrival of first batch task
  end if
  Append $(task_id, task_data, visible_aps, env.now)$ to $\mathcal{B}$
  if $|\mathcal{B}| \geq BS$ **and** $\neg$IsTriggered(E_{ready}) **then**
    TriggerEvent(E_{ready}) ▷ Signal maximum capacity reached
  end if
end procedure

procedure RunLifecycleLoop ▷ Continuous orchestrator background thread
  loop
    if $\mathcal{B} = [\,]$ **then**
      await $E_{arrival}$ ▷ Sleep until first request arrives
    end if
    $E_{timeout} \leftarrow$ StartTimer(env, BT) ▷ Start temporal aggregation window
    await $E_{ready} \vee E_{timeout}$ ▷ Wake on capacity reached or timeout expiration
    if $\mathcal{B} \neq [\,]$ **then**
      yield Timeout($env, 0$) ▷ Commit concurrent physical and orbital updates
      ProcessBatch()
    end if
    $E_{arrival} \leftarrow$ CreateEvent(env), $E_{ready} \leftarrow$ CreateEvent(env) ▷ Reset triggers
  end loop
end procedure

procedure ProcessBatch ▷ Batch execution pipeline
  $\mathcal{B}_{active} \leftarrow \mathcal{B}$, $\mathcal{B} \leftarrow [\,]$
  $(\mathcal{T}_{route}, \mathcal{M}_{nodes}) \leftarrow$ MapConstellationTopology()
  $\mathcal{A} \leftarrow$ SolveILP($\mathcal{T}_{route}, \mathcal{M}_{nodes}, \mathcal{B}_{active}$)
  DispatchTasks($\mathcal{A}$)
end procedure

3.3 Constellation Topology Mapping and Metric Normalization

Before formulating the optimization model, the Master Orchestrator must compute the optimal multi-hop ISL routing paths from every active ground Access Point to all candidate Satellite Edge Nodes. This exploration is conducted by the `_map_constellation` method in `Orchestrator.py` using a Multi-Source Dijkstra shortest-path algorithm [23, 21].

3.3.1 Dimensionless Link Cost Normalization

A major challenge in evaluating dynamic satellite links lies in the dimensional disparity between physical performance metrics. Directly summing link propagation latency (measured in milliseconds) and radio frequency/optical transmission energy (measured in microjoules or joules) is physically inconsistent and introduces scale bias into shortest-path calculations. Furthermore, the actual payload size s_r varies across requests, making it impossible to compute a single static link weight that accurately reflects every pending task.

To decouple graph exploration from individual task sizes, each directed inter-satellite edge (u, v) is probed using a standardized reference payload:

$$S_{ref} = 1 \text{ MB} = 1024^2 \text{ bytes} \tag{3.3}$$

For an edge (u, v) characterized by propagation delay $\tau_{u,v}$ ($\tau_{u,v} = \frac{C}{D_{u,v}}$ where C is the speed of light and $D_{u,v}$ is the distance between nodes) and transmission bandwidth $B_{u,v}$ (in Bytes/s), the physical traversal time and energy consumption for the reference payload over optical ISL channels [7, 8] are formulated as:

$$t_{link}(u, v) = \tau_{u,v} + \frac{S_{ref}}{B_{u,v}} \tag{3.4}$$

$$e_{link}(u, v) = P_{net} \cdot \frac{S_{ref}}{B_{u,v}} \tag{3.5}$$

where P_{net} is the active laser transceiver power (`Ptrasm`) [6].

To compute a composite, scale-free cost, the metrics are normalized against theoretical upper bounds, T_{max} and E_{max}. These bounds represent a worst-case link operating under minimum network bandwidth ($B_{min} = 100$ KB/s) and maximum slant propagation delay ($\tau_{max} = 100$ ms):

$$T_{max} = \tau_{max} + \frac{S_{ref}}{B_{min}}, \quad E_{max} = P_{net} \cdot \frac{S_{ref}}{B_{min}} \tag{3.6}$$

The dimensionless, normalized delay and energy metrics are thus:

$$\hat{t}_{link}(u, v) = \frac{t_{link}(u, v)}{T_{max}}, \quad \hat{e}_{link}(u, v) = \frac{e_{link}(u, v)}{E_{max}} \tag{3.7}$$

The composite scalar cost $c_{link}(u, v) \in [0, 1]$ is established as a convex combination governed by the user-defined routing policy weights w_R (`centralized_w_r`) and w_e (`centralized_w_e`):

$$c_{link}(u, v) = w_R \cdot \hat{t}_{link}(u, v) + w_e \cdot \hat{e}_{link}(u, v), \quad \text{where } w_R + w_e = 1 \tag{3.8}$$

Structural Decoupling Principle. The Multi-Source Dijkstra procedure uses S_{ref} solely to discover the structural shortest-path backbone $\vec{P}_{up}(AP_r, i)$ between access points and candidate satellites. Once these paths are established, the subsequent ILP optimization model scales transmission delay and forwarding energy according to the actual input payload size $s_r^{bytes} = s_r \cdot 1024^2$ of each individual request.

Concurrently with path discovery, the orchestrator queries all candidate nodes in `globals.edge_servers` to construct an up-to-date telemetry state snapshot:

$$\mathbf{v}_i = [B_i, W_{cpu,i}, W_{net,i}, C_{sen,i}, \kappa_i, t_{sunset,i}] \tag{3.9}$$

capturing residual battery levels, CPU queue wait times, network queue buffer latencies, processor frequencies, switching capacitance coefficients, and orbital eclipse horizons. The complete procedure is formalized in Algorithm 2.

Algorithm 2 Multi-Source Constellation Topology and Telemetry Mapping

Require: Access points AP_{global}, satellites S, weights w_R, w_e, reference size S_{ref}, bounds T_{max}, E_{max}
Ensure: Routing table $\mathcal{T}_{route}$ and node operational telemetry dictionary $\mathcal{M}_{nodes}$

$\mathcal{T}_{route} \leftarrow \emptyset, \quad \mathcal{M}_{nodes} \leftarrow \emptyset$
for all $ap \in AP_{global}$ **do**
  $\mathcal{T}_{route}[ap] \leftarrow \emptyset$
  Initialize min-heap $\mathcal{Q} \leftarrow \emptyset$, distance map $dist[v] \leftarrow \infty, \forall v \in (AP_{global} \cup S)$
  $dist[ap] \leftarrow 0.0$
  PUSH$(\mathcal{Q}, (0.0, 0.0, 0.0, ap, [ap]))$ ▷ Store $(cost, time, energy, accesspoint, path)$
  while $\mathcal{Q} \neq \emptyset$ **do**
    $(cost, time, energy, u, path) \leftarrow$ POPMIN$(\mathcal{Q})$
    if $cost > dist[u]$ **then**
      continue
    end if
    if $u \in S$ **then**
      $\mathcal{T}_{route}[ap][u] \leftarrow \{\text{cost} : cost, \text{time} : time, \text{energy} : energy, \text{path} : path\}$
    end if
    for all $v \in$ NEIGHBORS(u) **do**
      $\tau_{u,v} \leftarrow$ GETLATENCY$(u, v), \quad B_{u,v} \leftarrow$ GETBANDWIDTH$(u, v) \cdot 1024^2$
      if $B_{u,v} \leq 0$ **then** ▷ Skip inactive or broken links
        continue
      end if
      $t_{hop} \leftarrow \tau_{u,v} + \frac{S_{ref}}{B_{u,v}}, \quad e_{hop} \leftarrow P_{net} \cdot \frac{S_{ref}}{B_{u,v}}$
      $\hat{t} \leftarrow t_{hop}/T_{max}, \quad \hat{e} \leftarrow e_{hop}/E_{max}$
      $c_{link} \leftarrow w_R \cdot \hat{t} + w_e \cdot \hat{e}$
      if $dist[u] + c_{link} < dist[v]$ **then**
        $dist[v] \leftarrow dist[u] + c_{link}$
        PUSH$(\mathcal{Q}, (dist[v], time + t_{hop}, energy + e_{hop}, v, path \cup [v]))$
      end if
    end for
  end while
end for
for all $i \in S$ **do** ▷ Capture telemetry snapshot across all candidate compute nodes
  $W_{cpu,i} \leftarrow$ W_CPU$(i), \quad W_{net,i} \leftarrow$ W_NET(i)
  $B_i \leftarrow$ GETENERGY$(i), \quad C_{sen,i} \leftarrow$ GETCLOCKSPEED$(i), \quad t_{sunset,i} \leftarrow$ GETSUNSET(i)
  $\mathcal{M}_{nodes}[i] \leftarrow \{B_i, W_{cpu,i}, W_{net,i}, C_{sen,i}, \kappa_i, t_{sunset,i}\}$
end for
return $(\mathcal{T}_{route}, \mathcal{M}_{nodes})$

3.4 Centralized Integer Linear Programming Formulation

Let R be the set of tasks accumulated within the current batch ($|R| \leq BS$) and S the set of candidate Satellite Edge Nodes. Building upon combinatorial optimization theory [11, 12], the decision space is modeled via two sets of binary decision variables:

$$x_{r,i} \in \{0,1\}, \quad \forall r \in R, \forall i \in S \quad \text{(1 if task } r \text{ is assigned to SEN } i\text{; 0 otherwise)} \tag{3.10}$$

$$y_r \in \{0,1\}, \quad \forall r \in R \quad \text{(1 if task } r \text{ is explicitly rejected; 0 otherwise)} \tag{3.11}$$

3.4.1 Physical Delay, Energy, and Orbital Eclipse Modeling

For each candidate assignment pair (r,i), the total predicted response time $R_{r,i}$ accounts for the multi-hop transmission delay of the actual payload $s_r^{bytes} = s_r \cdot 1024^2$ along path $\vec{P}_{up}(AP_r, i)$, network buffering delays at intermediate hops, and compute queue and execution times at satellite i [6]:

$$R_{r,i} = \underbrace{\sum_{(u,v)\in\vec{P}_{up}(AP_r,i)} \left(W_{net,u} + \frac{s_r^{bytes}}{B_{u,v}} + \tau_{u,v} \right)}_{T_{up}(r,i)} + (W_{cpu,i} + d_{cpu,r}) \tag{3.12}$$

Similarly, the cumulative fleet energy expenditure $\epsilon_{r,i}$ evaluates the communication energy expended by all intermediate forwarding satellites along $\vec{P}_{up}(AP_r, i)$, combined with the dynamic CPU energy consumed by destination node i during local processing:

$$\epsilon_{r,i} = \sum_{(u,v)\in\vec{P}_{up}(AP_r,i)} P_{net} \frac{s_r^{bytes}}{B_{u,v}} + \kappa_i \cdot d_{cpu,r} \cdot C_{sen,i}^3 \tag{3.13}$$

The CPU energy model is derived from dynamic CMOS switching power dissipation:

$$P_{cpu} = \kappa_i \cdot C_{sen,i}^3 \tag{3.14}$$

which, integrated over the computational duration $d_{cpu,r}$, yields the second term in Equation (3.13) [6].

To protect satellites approaching orbital eclipse from battery depletion, we define an orbital sunset penalty $\Omega_i \in [0,1]$. Assigning tasks to a node nearing eclipse risks unexpected outages. Let $t_{sunset,i}$ represent the remaining visibility time for satellite i. The penalty factor is defined as:

$$\Omega_i = \begin{cases} 1.0 - \frac{t_{sunset,i}}{\max_{k\in S}(t_{sunset,k})}, & \text{if } \max_{k\in S}(t_{sunset,k}) > 0 \\ 1.0, & \text{if } t_{sunset,i} = 0 \\ 0.0, & \text{otherwise} \end{cases} \tag{3.15}$$

This term penalizes allocations to satellites entering eclipse without introducing non-linear step-function discontinuities into the objective function.

3.4.2 Multi-Objective Function and System Constraints

Task rejection ($y_r = 1$) is penalized via a large scalar coefficient $P_{drop} = 10^6$. This ensures that the solver rejects tasks only when physical capacity, energy, or deadline constraints make acceptance strictly infeasible. The global fleet performance metrics for delay (Z_{time}), energy (Z_{energy}), and eclipse degradation (Z_{sunset}) are defined as:

$$Z_{time} = \sum_{r \in R} \sum_{i \in S} x_{r,i} R_{r,i} + \sum_{r \in R} y_r P_{drop} \tag{3.16}$$

$$Z_{energy} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \epsilon_{r,i} + \sum_{r \in R} y_r P_{drop} \tag{3.17}$$

$$Z_{sunset} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \Omega_i \tag{3.18}$$

Depending on the operational strategy configured via `centralized_primary_objective` (Time-first vs. Energy-first), the composite objective function Z applies hierarchical scalar weights (α, β, γ) [12, 9]:

$$\min Z = \begin{cases} \alpha Z_{time} + \beta Z_{energy} + \gamma Z_{sunset}, & \text{if primary strategy is Time} \\ \alpha Z_{energy} + \beta Z_{time} + \gamma Z_{sunset}, & \text{if primary strategy is Energy} \end{cases} \tag{3.19}$$

where α scales the primary objective, β weights the secondary objective, and γ governs the eclipse penalty.

The feasible assignment space is governed by four linear constraints:

1. **Decision Uniqueness:** Every offloaded request $r \in R$ must either be allocated to exactly one satellite node $i \in S$ or explicitly dropped:

$$\sum_{i \in S} x_{r,i} + y_r = 1, \quad \forall r \in R \tag{3.20}$$

2. **Strict Processing Deadline Compliance:** The cumulative predicted latency (uplink forwarding, queue buffering, and execution) must not exceed the application deadline D_r [6]:

$$x_{r,i} \cdot R_{r,i} \leq D_r, \quad \forall r \in R, \forall i \in S \tag{3.21}$$

   If candidate satellite i cannot satisfy the deadline ($R_{r,i} > D_r$), the linear constraint forces $x_{r,i} = 0$.

3. **Distributed Fleet Energy Budget:** The total energy required from satellite $k \in S$—both as an intermediate packet forwarder along path $\vec{P}_{up}(AP_r, i)$ and as a compute node—must not exceed its usable residual battery budget B_k:

$$\sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \epsilon_{r,i,k} \leq B_k, \quad \forall k \in S \tag{3.22}$$

   where $\epsilon_{r,i,k}$ captures the specific energy portion expended by satellite k when task r is processed on satellite i.

4. **Constellation Load Balancing (Anti-Congestion):** To prevent localized bottlenecks and head-of-line blocking in satellite compute queues, the solver caps the total number of tasks allocated to any single satellite within the batch:

$$\sum_{r \in R} x_{r,i} \leq \left\lfloor \frac{|R|}{|S|} \right\rfloor + 5, \quad \forall i \in S \tag{3.23}$$

The uniform distribution term $\lfloor |R|/|S| \rfloor$ establishes a fair-share baseline, while the constant slack term $+5$ allows the solver to opportunistically exploit high-performance or low-latency nodes when capacity permits [18].

3.4.3 Solver Resolution and Cascade Rejection Diagnostics

The mathematical program is built dynamically in Python via `PuLP` [14] within the `_solve_ilp` method and solved using `HiGHS` [20, 21]. While conventional optimization formulations treat task rejection as an undifferentiated binary outcome ($y_r = 1$), root-cause attribution is essential for system monitoring and telemetry.

To provide operational visibility, when $y_r^* = 1$, the orchestrator executes a post-solution cascade diagnostic evaluation across all reachable candidate satellites $S_{reach}(r) \subseteq S$. For each candidate $i \in S_{reach}(r)$, the system evaluates physical feasibility conditions in order of operational priority:

1. **No Route Found:** If the access point cannot establish a path to any candidate node ($S_{reach}(r) = \emptyset$), the drop cause is classified as `"No_Route"`.

2. **Deadline Violation:** If the path and execution delay exceed the deadline ($R_{r,i} > D_r$), candidate i is flagged as `"Deadline_Violation"`.

3. **Eclipse/Sunset Infeasibility:** If the projected execution window extends past the satellite's solar horizon ($R_{r,i} > t_{sunset,i}$), the candidate is classified as `"Sunset_Violation"`.

4. **Battery Energy Depletion:** If the node's residual battery is insufficient to cover compute or forwarding demands ($B_k - E_{allocated,k} < \epsilon_{r,i,k}$), candidate i is labeled as `"Energy_Exhaustion"`.

5. **Capacity Saturation:** If node i has already reached its allocation cap under the load balancing bound, it is marked as `"Load_Balancing_Rejected"`.

6. **Global ILP Objective Trade-Off:** If candidate i is physically capable of servicing the task, but the solver dropped it to avoid exceeding fleet-wide budget constraints or penalizing the global objective function, the assignment is recorded as `"Global_ILP_Conflict"`.

Algorithm 3 details the complete optimization assembly, branch-and-cut resolution, and the post-solution cascade diagnostics implemented in `_solve_ilp` [21].

Algorithm 3 ILP Problem Assembly, Resolution, and Cascade Diagnostics

Require: Routing table $\mathcal{T}_{route}$, node states $\mathcal{M}_{nodes}$, batch R, weights α, β, γ
Ensure: Task assignments list $\mathcal{A}$ with assigned nodes, paths, and rejection causes

$\mathcal{P} \leftarrow$ CreateProblem("Centralized_ILP", min)
Instantiate variables: $x_{r,i} \in \{0,1\}, \forall r \in R, i \in S$ and $y_r \in \{0,1\}, \forall r \in R$
▷ **Step 1: Constraint Matrix Construction**
for all $r \in R$ **do**
    Add uniqueness constraint: $\sum_{i \in S} x_{r,i} + y_r = 1$ ▷ Eq. (3.20)
    for all $i \in S$ **do**
        Compute predicted delay $R_{r,i}$ (Eq. (3.12)) and energy $\epsilon_{r,i}$ (Eq. (3.13))
        Add deadline constraint: $x_{r,i} \cdot R_{r,i} \leq D_r$ ▷ Eq. (3.21)
    end for
end for
for all $k \in S$ **do**
    Add battery constraint: $\sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \epsilon_{r,i,k} \leq B_k$ ▷ Eq. (3.22)
    Add load balance constraint: $\sum_{r \in R} x_{r,i} \leq \lfloor |R|/|S| \rfloor + 5$ ▷ Eq. (3.23)
end for
▷ **Step 2: Model Resolution**
Set objective Z (Eq. (3.19)) and solve: $status \leftarrow$ SolveHiGHS($\mathcal{P}$)
$\mathcal{A} \leftarrow [\,]$
if $status \notin \{\text{Optimal}, \text{Suboptimal}\}$ **then**
    return $\mathcal{A}$ with all tasks $r \in R$ flagged as "Infeasible"
end if
▷ **Step 3: Decision Extraction and Cascade Diagnostics**
for all $r \in R$ **do**
    if $y_r^* < 0.5$ **then** ▷ Task accepted
        $i^* \leftarrow \arg\max_{i \in S} x_{r,i}^*, \quad path \leftarrow \mathcal{T}_{route}[AP_r][i^*]$
        Append $(r, i^*, path,$ "Assigned") to $\mathcal{A}$
    else ▷ Task rejected: execute cascade diagnostic
        $S_{reach} \leftarrow \{i \in S \mid \mathcal{T}_{route}[AP_r][i]$ exists$\}$
        $cause \leftarrow$ "No_Route"
        if $S_{reach} \neq \emptyset$ **then**
            $\mathcal{C} \leftarrow [\,]$
            for all $i \in S_{reach}$ **do**
                if $R_{r,i} > D_r$ **then**
                    Append "Deadline_Violation" to $\mathcal{C}$
                else if $R_{r,i} > t_{sunset,i}$ **then**
                    Append "Sunset_Violation" to $\mathcal{C}$
                else if $B_i - E_{allocated,i} < \epsilon_{r,i}$ **then**
                    Append "Energy_Exhaustion" to $\mathcal{C}$
                else if $AllocatedCount(i) \geq MaxCap(i)$ **then**
                    Append "Load_Balancing" to $\mathcal{C}$
                else
                    Append "Global_ILP_Conflict" to $\mathcal{C}$
                end if
            end for
            $cause \leftarrow$ DominantMode($\mathcal{C}$)
        end if
        Append $(r,$ **null**$, [\,],$ "Scartato_" $+ cause)$ to $\mathcal{A}$
    end if
end for
return $\mathcal{A}$

3.5 Task Execution and Resilient Hybrid Routing

Once the solver generates the optimal assignment matrix $\mathcal{X}^*$, the orchestrator invokes `_dispatch_tasks` to process each decision. Depending on solver status and network availability, tasks are either routed, re-enqueued, or permanently dropped.

3.5.1 Dispatching and Buffer Re-Enqueuing Pipeline

The dispatching layer acts as an admission and recovery filter before physical packet injection:

- **Accepted Tasks ($status$ = "Assigned"):** The orchestrator spawns an asynchronous `_route_and_execute_task` process within the simulation environment, passing the planned forwarding path $\vec{P}_{up}(AP_r, i^*)$.
- **Transient Failures ("No_Route" or "Infeasible"):** A task marked with an infeasible status or lacking a physical ISL route at instant t is not necessarily unservable; orbital motion may restore connectivity within seconds. The orchestrator re-enqueues the request into $\mathcal{B}$ while preserving its original arrival timestamp t_{arr}. If the re-enqueued requests saturate the buffer ($|\mathcal{B}| \geq BS$), E_{ready} is triggered immediately.
- **Permanent Rejections ("Energy_Exhaustion", "Deadline_Violation", etc.):** Tasks dropped due to deterministic physical violations are logged directly at the entry gateway AP_r via `record_rejected_task` and permanently discarded.

Algorithm 4 formalizes this dispatching and re-enqueuing engine, implemented in the `_dispatch_tasks` method of `Orchestrator.py` [21].

Algorithm 4 Task Dispatching, Re-Enqueuing, and Drop Accounting

Require: Assignment list $\mathcal{A}$, constellation nodes $\mathcal{N}$, task buffer $\mathcal{B}$, capacity BS
Ensure: Task process spawning, transient task re-enqueuing, and terminal drop logging

for all $task \in \mathcal{A}$ **do**
  $t_id \leftarrow task.task_id$, $data \leftarrow task.data$, $status \leftarrow task.status$
  if $status$ = "Assigned" **then**
    Spawn discrete-event process: RouteAndExecuteTask($task, \mathcal{N}, env.now$)
  else if "No_Route" $\in status \vee$ "Infeasible" $\in status$ **then** ▷ Transient orbital failure
    Re-enqueue ($t_id, data, AP_{global}, data.arrival_time$) into $\mathcal{B}$
    if $|\mathcal{B}| \geq BS$ **and** $\neg$IsTriggered(E_{ready}) **then**
      TriggerEvent(E_{ready}) ▷ Trigger early batch for retry
    end if
  else ▷ Deterministic drop (e.g., Energy Exhaustion, Deadline Violation)
    $ap \leftarrow AP_{global}[0]$
    ap.RecordRejectedTask($t_id, data.type, env.now, data.image_size, status, data.d_cpu$)
  end if
end for

3.5.2 Resilient Hybrid Packet Routing

In dynamic LEO constellations, planned ISL trajectories are vulnerable to mid-flight disruption. Thermal expansion, high-speed orbital plane crossings, antenna micro-vibrations, and transient Doppler shifts can degrade or break an inter-satellite link while a packet is in transit. Relying entirely on a pre-computed path risks packet loss. To achieve operational resilience, the data forwarding layer implements a dynamic hybrid routing protocol:

- **Nominal Planned Routing (Phase A):** Under nominal conditions, the data packet follows the shortest-path trajectory $\vec{P}_{up}(AP_r, i)$ computed during topology mapping. At each node, forwarding occurs if the next scheduled link remains operational ($B_{curr,next} > 0$).
- **Emergency Geographic Greedy Fallback (Phase B):** If an intermediate forwarding link is severed ($B_{curr,next} \leq 0$), the current node autonomously transitions to emergency greedy recovery. The forwarding satellite queries its active physical neighbors via `get_neighbors()`, calculates their Euclidean distance to the destination satellite using orbital position vectors (`get_pos_proximity`), and forwards the payload to the closest eligible neighbor.

Loop Prevention and Dead-End Recovery. To prevent packet cycling during greedy fallback, each task maintains a traversal set $\mathcal{V}_{visited}$ inside its header metadata (`Task.visited`). Nodes already traversed are excluded from fallback selection. If a node encounters a topological dead end where all active neighbors are either broken or already in $\mathcal{V}_{visited}$, the packet is dropped with status `"DROPPED_GREEDY_DEADEND"`, preventing buffer congestion.

Upon arrival at the destination node i^*, transmission latency and hop counts are committed to `globals.gbl_generated_tasks_data`, and the thread yields to the compute queue simulator (`ILP_simulation.TaskAssignment_ILP` [21]). Algorithm 5 details the complete hybrid routing and execution pipeline.

Algorithm 5 Dynamic Hybrid Routing and Task Execution

Require: Accepted task r, destination SEN i^*, pre-computed path $\vec{P}_{up} = (v_0, v_1, \ldots, v_m)$ where $v_0 = AP_r, v_m = i^*$
Ensure: Reliable multi-hop forwarding and discrete-event CPU execution
Initialize $curr \leftarrow v_0$, $idx \leftarrow 0$, $is_fallback \leftarrow$ **false**, $\mathcal{V}_{visited} \leftarrow \{v_0\}$
while $curr \neq i^*$ **do**
  if $\neg is_fallback$ **then** ▷ Phase A: Nominal Planned Path
    $next \leftarrow v_{idx+1}$
    if $B_{curr,next} > 0$ **then**
      Forward payload across link $(curr, next)$
      $idx \leftarrow idx + 1$, $curr \leftarrow next$, $\mathcal{V}_{visited} \leftarrow \mathcal{V}_{visited} \cup \{curr\}$
    else
      $is_fallback \leftarrow$ **true** ▷ Link broken: engage emergency recovery
    end if
  end if
  if $is_fallback$ **then** ▷ Phase B: Autonomous Geographic Greedy Forwarding
    Retrieve neighbor set $\mathcal{N}_{active} \leftarrow \{u \in \textsc{Neighbors}(curr) \mid B_{curr,u} > 0 \wedge u \notin \mathcal{V}_{visited}\}$
    if $\mathcal{N}_{active} = \emptyset$ **then**
      return DROP(`"DROPPED_GREEDY_DEADEND"`)
    end if
    Select best neighbor: $next^* \leftarrow \arg\min_{u \in \mathcal{N}_{active}} \textsc{EuclideanDistance}(\mathbf{p}_u, \mathbf{p}_{i^*})$
    Forward payload across link $(curr, next^*)$
    $curr \leftarrow next^*$, $\mathcal{V}_{visited} \leftarrow \mathcal{V}_{visited} \cup \{curr\}$
  end if
end while
Record transfer latency $T_{transfer} \leftarrow t_{now} - t_{arr}$ and final hop count $|\mathcal{V}_{visited}| - 1$
Yield to discrete-event simulator: $\textsc{TaskAssignmentILP}(env, SelectedServer, TaskData)$

3.6 Computational Complexity and Scalability Analysis

Deploying mathematical optimization within dynamic LEO constellations introduces fundamental trade-offs between theoretical optimality and runtime overhead [6]. To establish the operational bounds of the centralized architecture, this section formalizes the asymptotic time and space complexity across the three pipeline stages: topology mapping, ILP solver resolution, and packet forwarding.

3.6.1 Topology Mapping and Pathfinding Complexity

Prior to solver instantiation, the orchestrator maps the constellation routing fabric using a Multi-Source Dijkstra shortest-path algorithm over the directed graph $G = (V, E)$ [23]. The node set $V = AP_{global} \cup S$ comprises all active regional access points and candidate satellites, while E represents active ground-to-satellite links and inter-satellite links [6].

Using an adjacency list representation paired with a binary min-heap priority queue (via Python's `heapq` module), the asymptotic time complexity across all active access points $|AP|$ is:

$$\mathcal{T}_{map} = \mathcal{O}\left(|AP| \cdot (|E| + |V| \log |V|)\right) \tag{3.24}$$

In standard Walker-Delta constellations, satellite transceivers are hardware-limited to at most four optical heads: two intra-plane links to ahead/behind peers and two inter-plane links to adjacent orbital planes [16, 7]. Consequently, the maximum node degree is strictly bounded ($\deg(v) \leq 4$), yielding a sparse graph where the total edge cardinality scales linearly with constellation size:

$$|E| \leq 4|S| + 2|AP| \implies |E| = \mathcal{O}(|S|) \tag{3.25}$$

Substituting this structural property into Equation (3.24) yields an effective time complexity of:

$$\mathcal{T}_{map} = \mathcal{O}\left(|AP| \cdot |S| \log |S|\right) \tag{3.26}$$

Regarding auxiliary memory footprint, the orchestrator stores the adjacency graph, node telemetry vectors, and pre-computed paths. The space complexity is $\mathcal{O}(|AP| \cdot |S| + |E|)$. For regional constellations ($|S| \leq 50$), this requires under a few megabytes, fitting comfortably within the memory constraints of embedded aerospace processing boards.

3.6.2 ILP Formulation and Branch-and-Cut Solver Complexity

For an active batch containing $|R|$ buffered tasks and $|S|$ candidate Satellite Edge Nodes:

- **Decision Variable Space:** The model allocates $|R| \cdot |S|$ binary assignment variables $x_{r,i}$ alongside $|R|$ binary drop variables y_r. The total decision dimension is:
$$\mathcal{D}_{vars} = |R| \cdot |S| + |R| = \mathcal{O}(|R| \cdot |S|) \tag{3.27}$$
- **Constraint Matrix Dimensions:** The constraint system enforces $|R|$ uniqueness equations, $|R| \cdot |S|$ deadline compliance bounds, $|S|$ fleet battery budget limits, and $|S|$ load balancing bounds. The total constraint row complexity is:
$$\mathcal{C}_{rows} = |R| \cdot (|S| + 1) + 2|S| = \mathcal{O}(|R| \cdot |S| + |R| + 2|S|) \tag{3.28}$$

While 0-1 Integer Linear Programs are strongly NP-hard in the general case, with a worst-case complexity of $\mathcal{O}(2^{|R| \cdot |S|})$ [11], the modern branch-and-cut implementation in `HiGHS` mitigates combinatorial explosion through presolve reductions, cutting planes, and polynomial-time linear programming relaxations ($\mathcal{O}(n^{2.5})$) solved via the dual revised simplex method [20, 11].

The *Dynamic Batching Engine* acts as an architectural boundary on problem scale. By capping batch size ($|R| \leq BS$ with $BS \leq 20$), the optimization problem space does not scale with external arrival rate spikes λ. For a representative regional constellation slice ($|S| \approx 15$), the model instantiates at most 300 binary variables and 330 constraint rows. Because the constraint matrix is sparse—each uniqueness constraint contains only $|S|$ non-zero entries, and each deadline constraint contains exactly one—the LP relaxations converge in fewer than 50 simplex pivots, keeping solver execution time bounded within $T_{exec} \leq 0.30$–0.60 ms.

3.6.3 Hybrid Routing and Forwarding Complexity

The computational overhead of the forwarding layer depends on network link stability:

- **Nominal Planned Forwarding (Phase A):** Forwarding nodes traverse the pre-computed Dijkstra path $\vec{P}_{up}(AP_r, i)$. Each hop requires an $\mathcal{O}(1)$ pointer lookup in the local routing table. The total time complexity for routing across h_{up} hops is bounded by the network diameter:

$$\mathcal{T}_{planned} = \mathcal{O}(h_{up}) \leq \mathcal{O}(\text{diam}(G)) \ll \mathcal{O}(|S|) \tag{3.29}$$

- **Greedy Recovery Fallback (Phase B):** If a planned link fails, the forwarding node evaluates adjacent neighbor distances. Sorting candidate neighbors requires $\mathcal{O}(k \log k)$ time, where k is the physical node degree. Because optical transceiver architectures enforce $k \leq 4$, this sorting step operates in negligible constant time:

$$\mathcal{T}_{greedy} = \mathcal{O}(k \log k) \approx \mathcal{O}(1) \quad (\text{since } k \leq 4) \tag{3.30}$$

  Visited node lookups are evaluated against a hash-set structure ($\mathcal{V}_{visited}$), providing $\mathcal{O}(1)$ average-time checks and preventing forwarding loops.

3.6.4 Constellation Scalability and Telemetry Overhead

A central factor governing the feasibility of centralized orbital orchestration is the control signaling bandwidth required to maintain constellation state at the Master node [1, 6]. To construct the optimization model, the orchestrator periodically collects a compact telemetry vector $\mathbf{v}_i = [W_{cpu,i}, W_{net,i}, B_i, t_{sunset,i}]$ from each satellite.

Transmitting these telemetry updates across the ISL mesh introduces a control communication overhead of $\mathcal{O}(|S|)$ messages per update period. In regional deployments ($|S| \leq 50$), telemetry updates consume less than 0.1% of the available optical link bandwidth, ensuring that control signaling does not congest data transmission channels. However, as constellation scales grow globally to thousands of spacecraft ($|S| \gg 1000$), the propagation latency and stale-state overhead of a single master node become limiting factors, motivating regional partitioning or hierarchical decomposition for global deployments [6].

Chapter 4

Performance Analysis, Optimization, and Experimental Evaluation

This chapter presents the experimental evaluation, systematic parametric tuning, and consolidated validation of the proposed Centralized Integer Linear Programming (ILP) Orchestrator [6]. While Chapter 3 established the formal mathematical optimization model and its discrete-event simulation architecture, deploying exact centralized scheduling in Low Earth Orbit (LEO) constellations introduces fundamental operational trade-offs [4, 1]. In particular, centralized optimization must balance the queuing latency introduced by task accumulation in ingress buffers against the physical dynamics of dynamic inter-satellite topologies and strict application deadlines [6, 9].

To address these challenges, this chapter follows a progressive two-stage tuning methodology concluded by a final validation of the resulting configurations against the pre-optimization baseline:

1. **Stage 1: Macro-Tuning of Ingress Batching ($BS \times BT$):** We explore an exhaustive design space of 28 batching configurations across increasing traffic arrival rates ($\lambda \in [2, 10]$ req/s) and deadline relaxation thresholds (ΔD) using baseline default weights. This isolates the queuing latency bottleneck and identifies the viable operational batching policies.

2. **Stage 2: Micro-Tuning of Objective Weights ($\gamma \times \alpha$):** Focusing on the viable batching policies under heavy load ($\lambda = 8$ req/s and $\lambda = 10$ req/s), we perform a cross-sensitivity analysis between the primary objective weight (α) and the orbital sunset penalty (γ) to maximize feasibility while preventing the assignment of tasks to satellites nearing orbital eclipse.

3. **Stage 3: Consolidated Validation:** With the optimal objective weights determined, we validate the selected configurations across the full arrival rate spectrum ($\lambda \in [2, 10]$ req/s), directly comparing their performance against the pre-optimization baseline.

4.1 Simulation Environment and Methodological Setup

The experimental campaign was carried out using an event-driven discrete-event simulation engine implemented in Python with `SimPy` [13], interfaced with the `PuLP` mathematical programming library [14] and solved via the open-source `HiGHS` branch-and-cut solver [20]. Numerical telemetry and array manipulations were processed using `NumPy` [24] and `SciPy` [25], while performance distributions and diagnostic curves were generated using `Matplotlib` [26]. The simulator models orbital kinematics [22], optical Inter-Satellite Links (ISLs) [7, 8], dynamic ground-to-satellite line-of-sight contact windows, and heterogeneous onboard Satellite Edge Nodes (SENs) [6].

4.1.1 Constellation Topology and Scenario Parameters

The orbital network is modeled as a regional Walker-Delta LEO constellation operating at an altitude of 600 km with an inclination of 53° [16]. Satellites are interconnected via full-duplex optical ISLs characterized by nominal transmission bandwidths $B_{u,v}$ and propagation delays $\tau_{u,v}$ computed dynamically from orbital positions [8]. A designated Master in the operational region collects offloaded task requests forwarded by Ground Users (GUs) through regional Ground Access Points (AP_{global}) [6].

Workload demands and request dimensions are parameterized through three profiling vectors, consistent with recent benchmarking literature [6]:

- **Workload Distribution Vector $\boldsymbol{\beta} = (\beta_G, \beta_C, \beta_{CD})$:** Dictates the proportion of Generic tasks, CPU-intensive tasks, and CPU + Data-intensive tasks. In the baseline batching and tuning experiments, the default multi-service profile is set to $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$ [6].
- **Real-Time Payload Vector $\boldsymbol{\alpha} = (\alpha_M, \alpha_L, \alpha_V)$:** Governs the distribution of input/output dataset dimensions across Medium (1–5 MB), Large (5–20 MB), and Very Large (20–50 MB) files, parameterized as $\boldsymbol{\alpha} = (0.30, 0.50, 0.20)$ [6, 2].
- **Batch Task Data Vector $\boldsymbol{\gamma}_{data} = (\gamma_L, \gamma_V)$:** Controls delay-tolerant background transfers, fixed to $\boldsymbol{\gamma}_{data} = (0.70, 0.30)$ [4].

Candidate edge nodes are assigned operational battery reserves of $E_i \in \{40, 80\}$ kJ per orbital pass [6, 19]. The request arrival rate λ varies systematically from 2 req/s (low-load, mean inter-arrival time of 0.50 s) up to 10 req/s (heavy-load stress regime, mean inter-arrival time of 0.10 s) [6].

4.1.2 Two-Tier Optimization Methodology

Directly testing all permutations of batching thresholds, objective weights, and network loads would require thousands of simulations and obscure the physical origins of performance bottlenecks. We therefore decouple the optimization process:

- **Architectural Decoupling:** Ingress batching (BS, BT) governs the static waiting time of tasks in the coordinator queue *before* the ILP model is instantiated. Conversely, objective function weights (α, γ) guide the mathematical branch-and-bound trade-offs *during* solver execution [11, 12].

- **Sequential Tuning:** By first sweeping $BS \times BT$ under default weights, we isolate the feasible batching configurations that do not cause queue-induced deadline collapse. Next, we tune the objective weights (α, γ) under heavy stress on these selected batching archetypes. Finally, we benchmark the optimized configurations against the pre-tuning baseline across the entire workload spectrum.

4.2 Macro-Tuning: Exploration and Selection of Batching Policies

In a centralized orchestrator, incoming requests are buffered in ingress queues until either the Batch Size (BS) or the Batch Timeout (BT) threshold is triggered. This introduces a fundamental operational trade-off: larger batches aggregate broader spatial-temporal context, enabling the ILP model to optimize joint multi-task routing and node allocation, but prolonged buffering directly consumes application deadline margins [6].

To isolate the queuing latency bottleneck and establish the viable operating envelope, a parametric sweep of 28 configurations was executed across arrival rates $\lambda \in [2, 10]$ req/s under both Energy Mapping and Time Mapping, evaluated across deadline relaxation levels $\Delta D \in \{10\%, 20\%, 30\%\}$.

4.2.1 Success Rate Dynamics

Figures 4.1 and 4.2 report the Task Success Rate across the parameter space for Energy and Time mapping, respectively.

The empirical curves highlight five key operational dynamics:

- **Peak Low-Load Throughput and Stress Convergence of Instantaneous Dispatch ($BT = 0.0$ s):** Enforcing an immediate dispatch policy ($BS = 20, BT = 0.0$ s, uppermost curve) maximizes throughput under light-to-moderate traffic, achieving $\approx 92\%$–93% success at $\lambda \leq 3$ req/s and maintaining $\approx 89\%$ at $\lambda = 4$ req/s. Beyond moderate load, however, it experiences a steep decline, falling to $\approx 68\%$–82% at $\lambda = 6$ req/s and collapsing to $\approx 41\%$–48% at $\lambda = 10$ req/s. Crucially, at peak load ($\lambda \geq 8$ req/s), the throughput of $BT = 0.0$ s intersects and converges with the reactive policy ($BT = 0.05$ s) at $\approx 53\%$ ($\lambda = 8$ req/s) and $\approx 41\%$–42% ($\lambda = 10$ req/s), demonstrating that unbuffered single-task dispatch provides no throughput advantage under sustained high-rate arrivals.

- **Controlled Small-Batching Dynamics and High-Load Convergence (BS-Sweep):** Small aggregation quotas ($BS \in \{1, 2\}$ with $BT = 2.0$ s) deliver robust performance across the arrival spectrum. While assembling larger batches under low traffic causes severe degradation due to prolonged timeout dwell times ($\approx 67\%$ for $BS \leq 2$ vs. $\approx 44\%$ for $BS = 10$ at $\lambda = 2$ req/s under Energy Mapping), all batch size configurations converge to a narrow band ($\approx 38\%$–48%) at peak stress ($\lambda = 10$ req/s). Under heavy load, the rapid

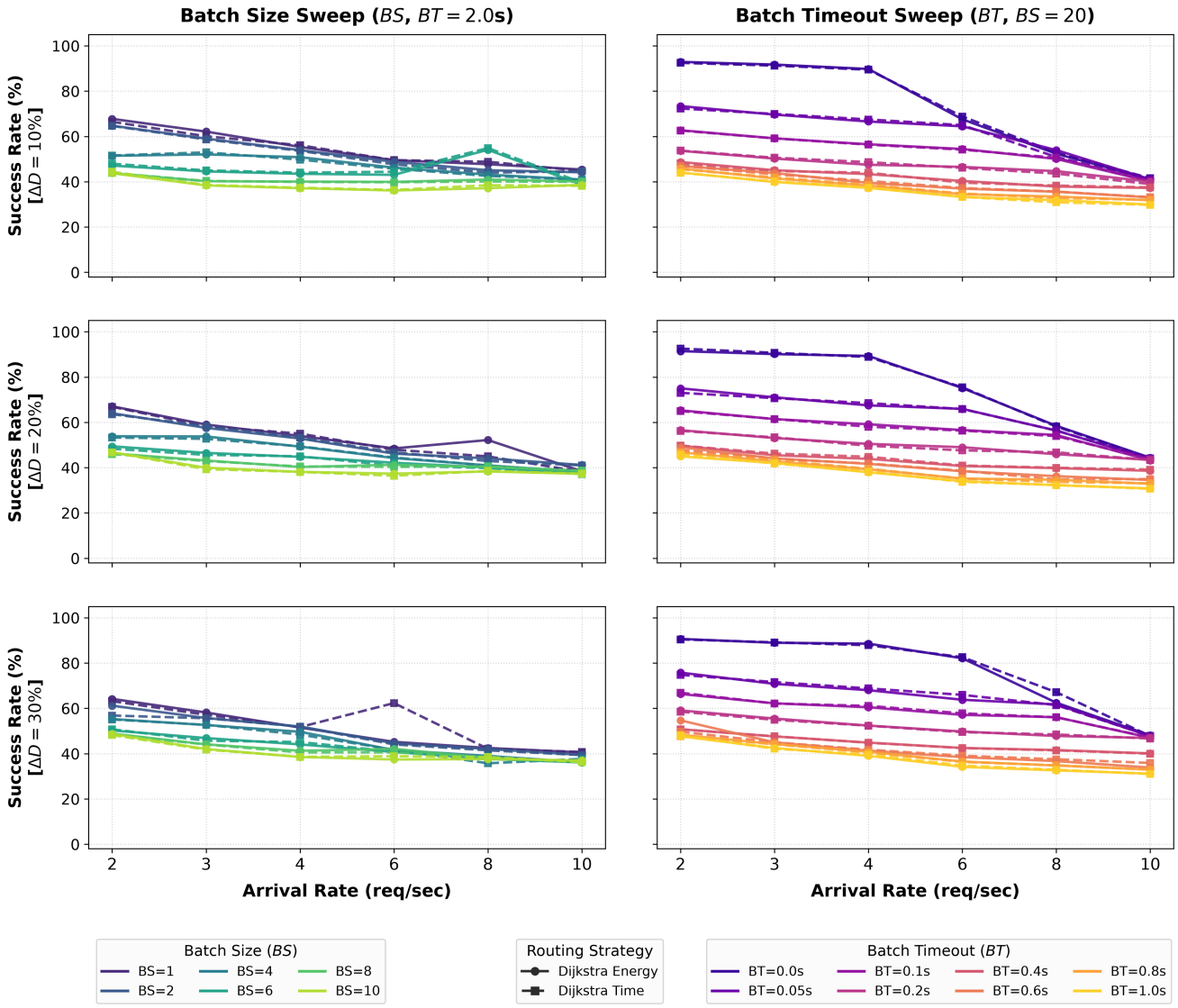

Figure 4.1. Success Rate across arrival rates under Energy Mapping for BS-sweep ($BT = 2.0$ s) and BT-sweep ($BS = 20$) across deadline relaxation levels $\Delta D \in \{10\%, 20\%, 30\%\}$.

inter-arrival window (≈ 100 ms) fills batch quotas within 0.2–1.0 s, well before the 2.0 s timeout expiration.

- **Systematic Degradation under High-Timeout Regimes ($BT \geq 0.4$ s):** Relying on generous buffer timeouts ($BT \geq 0.4$ s, lower orange and yellow curves in the right panels) consistently degrades performance across all traffic regimes. Buffer residency directly consumes the deadline budget, capping success rates below 50% even under light load ($\lambda = 2$ req/s) and deteriorating to 30%–35% under peak load ($\lambda = 10$ req/s).

- **Mapping Metric Sensitivity at Low Load:** While Energy and Time mappings produce almost identical trends under the BT-sweep, the BS-sweep

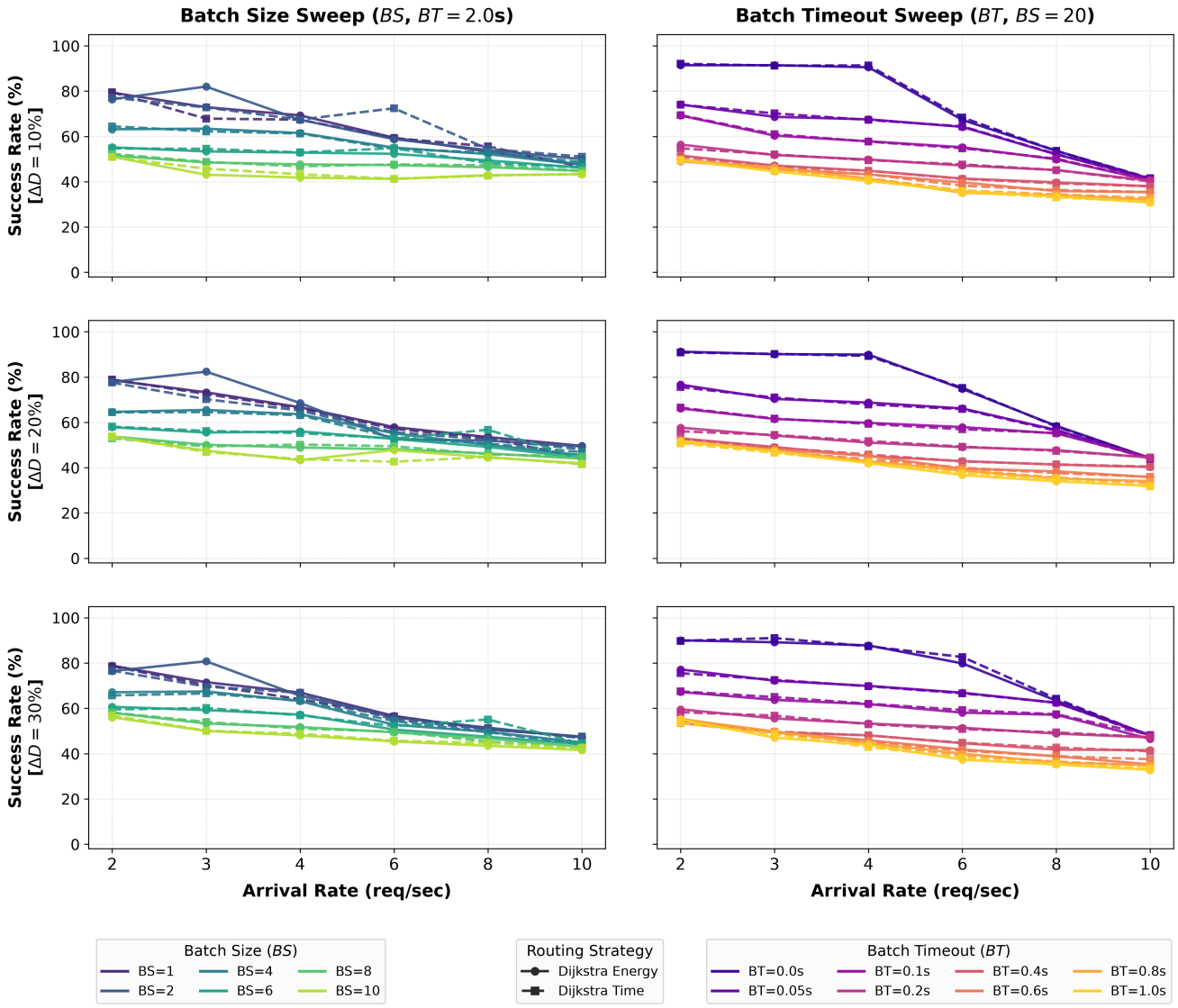

Figure 4.2. Success Rate across arrival rates under Time Mapping for BS-sweep ($BT = 2.0$ s) and BT-sweep ($BS = 20$) across deadline relaxation levels $\Delta D \in \{10\%, 20\%, 30\%\}$.

reveals an observable sensitivity to the mapping objective under light traffic ($\lambda \leq 3$ req/s): Time Mapping achieves $\approx 78\%$–82% success for $BS \in \{1, 2\}$, outperforming Energy Mapping ($\approx 65\%$–68%) by over 10% through latency-minimal server selection. At higher loads ($\lambda \geq 8$ req/s), this divergence vanishes as physical resource scarcity dominates solver decisions.

- **Routing Parity and Localized Topological Spikes:** While Dijkstra-Energy (solid lines) and Dijkstra-Time (dashed lines) track each other closely in most operational regimes, Dijkstra-Time exhibits pronounced localized throughput gains at intermediate loads. Notably, at $\lambda = 6$ req/s for $BS = 2$, Dijkstra-Time achieves an upward spike of $+14\%$ under Time Mapping ($\approx 72\%$ vs. $\approx 58\%$, $\Delta D = 10\%$) and $+17\%$ under Energy Mapping ($\approx 62\%$ vs.

$\approx 45\%$, $\Delta D = 30\%$). This reflects specific orbital alignments where ISL delay minimization rescues borderline tasks from deadline expiration.

4.2.2 Diagnostics of Failure Causes

Figures 4.3 and 4.4 (under Energy Mapping), alongside Figures 4.5 and 4.6 (under Time Mapping), illustrate the relative distribution of rejection causes across the parameter sweeps.

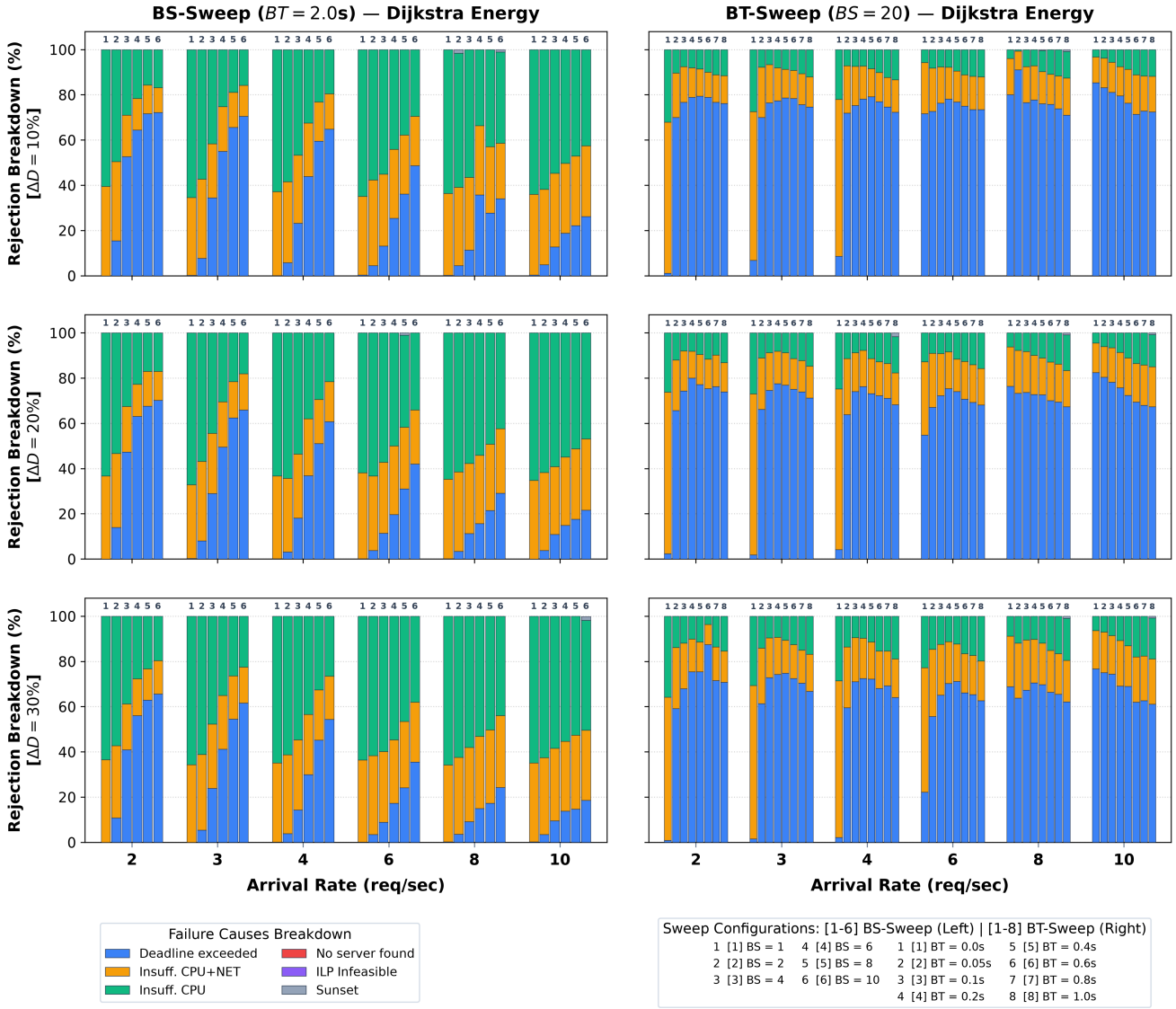

Figure 4.3. Rejection causes distribution across arrival rates under Energy Mapping with Dijkstra Energy routing, comparing BS-sweep ($BT = 2.0$ s) against BT-sweep ($BS = 20$) for $\Delta D \in \{10\%, 20\%, 30\%\}$.

The failure diagnostics reveal three structural properties of the constellation orchestrator:

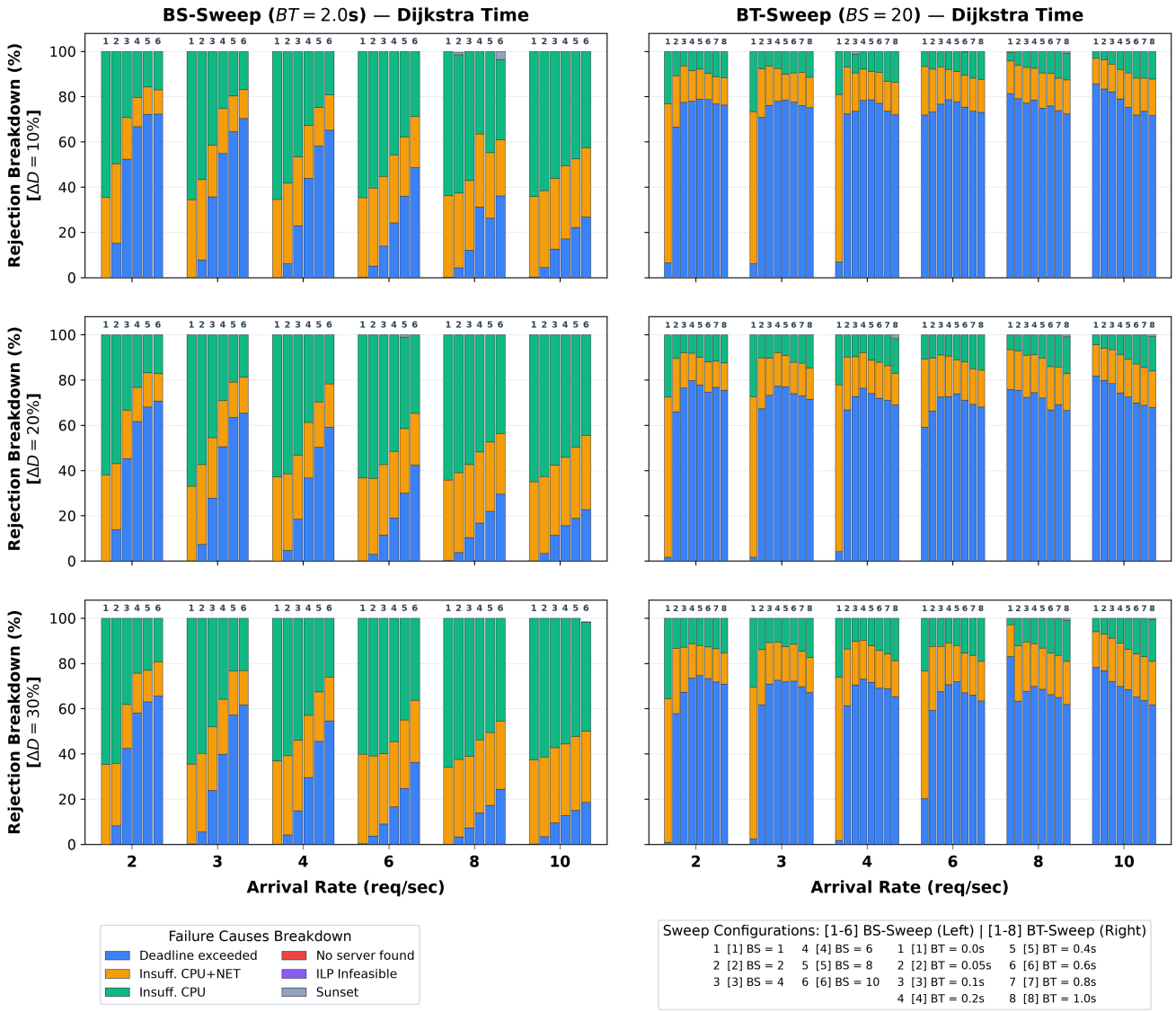

Figure 4.4. Rejection causes distribution across arrival rates under Energy Mapping with Dijkstra Time routing, comparing BS-sweep ($BT = 2.0$ s) against BT-sweep ($BS = 20$) for $\Delta D \in \{10\%, 20\%, 30\%\}$.

- **Elimination of Algorithmic and Contact Window Drops:** Across all tested configurations and arrival rates, rejections due to *ILP Infeasible*, *Sunset*, and *No Server Found* remain at 0.0%. This confirms that the ILP mathematical formulation preserves solvability, the contact window predictor successfully avoids satellites nearing orbital sunset, and ingress ground stations consistently maintain multi-satellite visibility. Task rejections are strictly categorized into temporal expirations (*Deadline Exceeded*) or physical resource exhaustion (*Insufficient CPU* and *Insufficient CPU+NET*).

- **Load-Dependent Failure Transition in Instantaneous Dispatch ($BS = 20, BT = 0.0$ s):** When buffer dwell time is structurally eliminated ($BT = 0.0$ s,

outermost left bar of each arrival rate cluster in the right panels), failure causes exhibit a sharp, load-dependent transition:

  - Under light-to-moderate traffic ($\lambda \in [2, 4]$ req/s), task rejections due to *Deadline Exceeded* (blue) are strictly suppressed ($< 5\%$). Rejections are overwhelmingly driven by physical resource scarcity: *Insufficient CPU+NET* (orange) accounts for $\approx 70\%$–75% of drops, while *Insufficient CPU* (green) accounts for the remaining $\approx 20\%$–25%. Without coordinator buffering, tasks fail directly because spatial link bandwidth or onboard compute slots are unavailable upon arrival.
  - Under heavy stress ($\lambda \geq 6$ req/s), this behavior inverts drastically. As arrival frequencies exceed the processing rate of individual satellites, uncoordinated single-task dispatch causes localized onboard execution queues ($W_{cpu,i}$) to expand rapidly. Consequently, even without ingress buffering delay, cumulative processing queues push execution past task deadlines, causing *Deadline Exceeded* to surge to $\approx 55\%$–58% at $\lambda = 6$ req/s and reaching $\approx 80\%$–85% at $\lambda \in [8, 10]$ req/s, while physical capacity drops diminish below 15%.

- **Rapid Deadline Collapse in Buffered Policies ($BT \geq 0.05$ s):** Introducing even a marginal buffer delay alters the failure distribution. For $BT = 0.05$ s, *Deadline Exceeded* accounts for $\approx 50\%$–55% of drops at $\lambda = 2$ req/s, escalating to $\approx 80\%$–85% at $\lambda = 10$ req/s. For $BT \geq 0.1$ s, buffer dwell time exhausts available application slack times, making *Deadline Exceeded* the primary cause of rejection (70%–85%) across all loads, while physical resource rejections contract to 15%–30%.

- **Bottleneck Inversion in Batch Size Aggregation (BS-Sweep, $BT = 2.0$ s):** The BS-sweep exhibits an inversion in rejection mechanics:

  - For minimal aggregation ($BS \in \{1, 2\}$), deadline failures remain suppressed ($< 15\%$ at $\lambda = 2$ req/s and $< 8\%$ at $\lambda \geq 6$ req/s), with physical constraints accounting for $> 85\%$ of drops (*Insufficient CPU* $\approx 50\%$–60%, *Insufficient CPU+NET* $\approx 35\%$–40%).
  - For large batches ($BS \geq 6$) under low traffic ($\lambda = 2$ req/s), waiting for subsequent arrivals triggers the 2.0 s timeout, making *Deadline Exceeded* the dominant cause (70%–80%). However, at peak traffic ($\lambda = 10$ req/s), frequent arrivals fill batch quotas before timeout expiration, causing deadline failures to contract to 20%–28% while physical capacity bottlenecks re-emerge as the primary constraint ($> 70\%$).

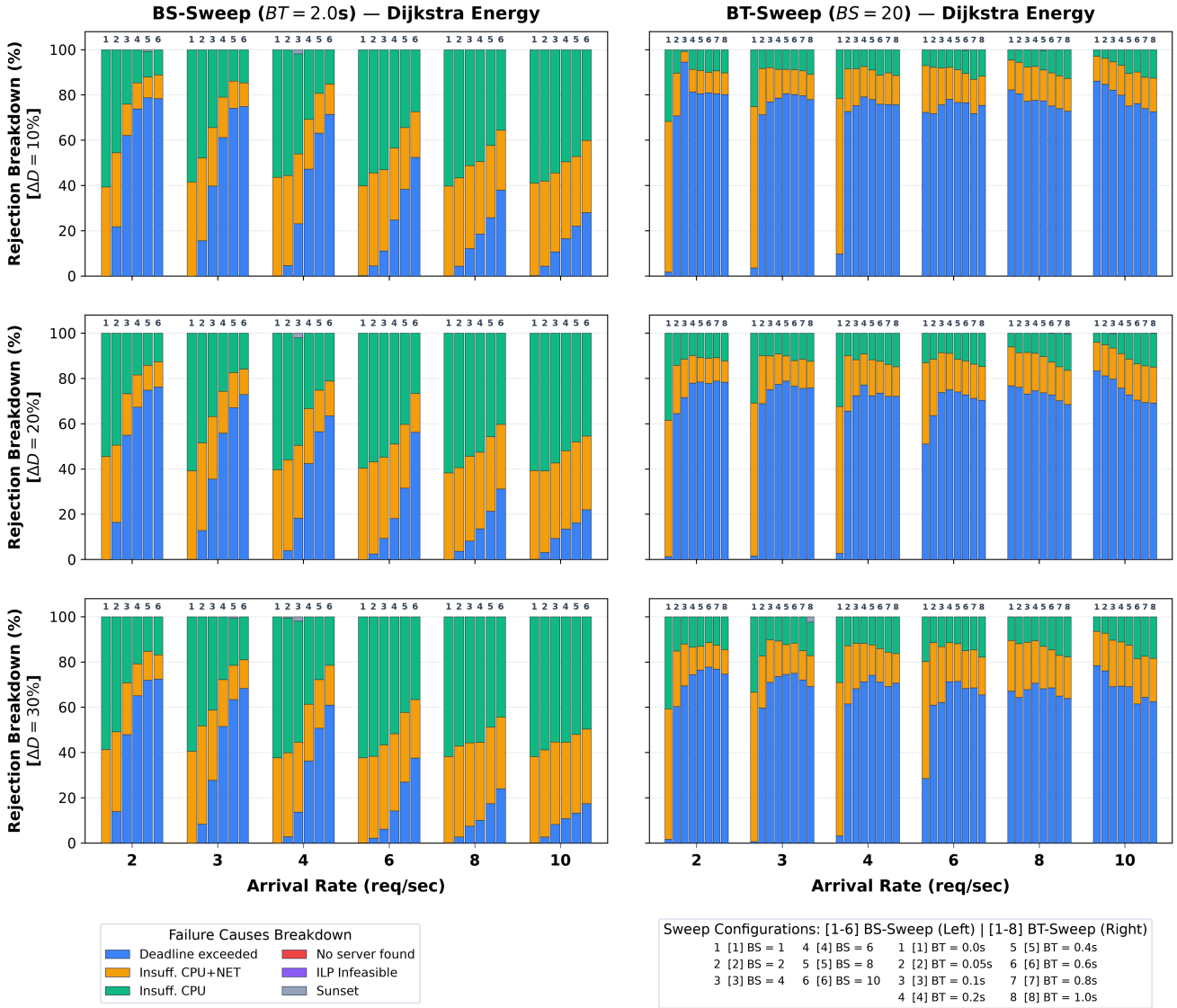

Figure 4.5. Rejection causes distribution across arrival rates under Time Mapping with Dijkstra Energy routing, comparing BS-sweep ($BT = 2.0$ s) against BT-sweep ($BS = 20$) for $\Delta D \in \{10\%, 20\%, 30\%\}$.

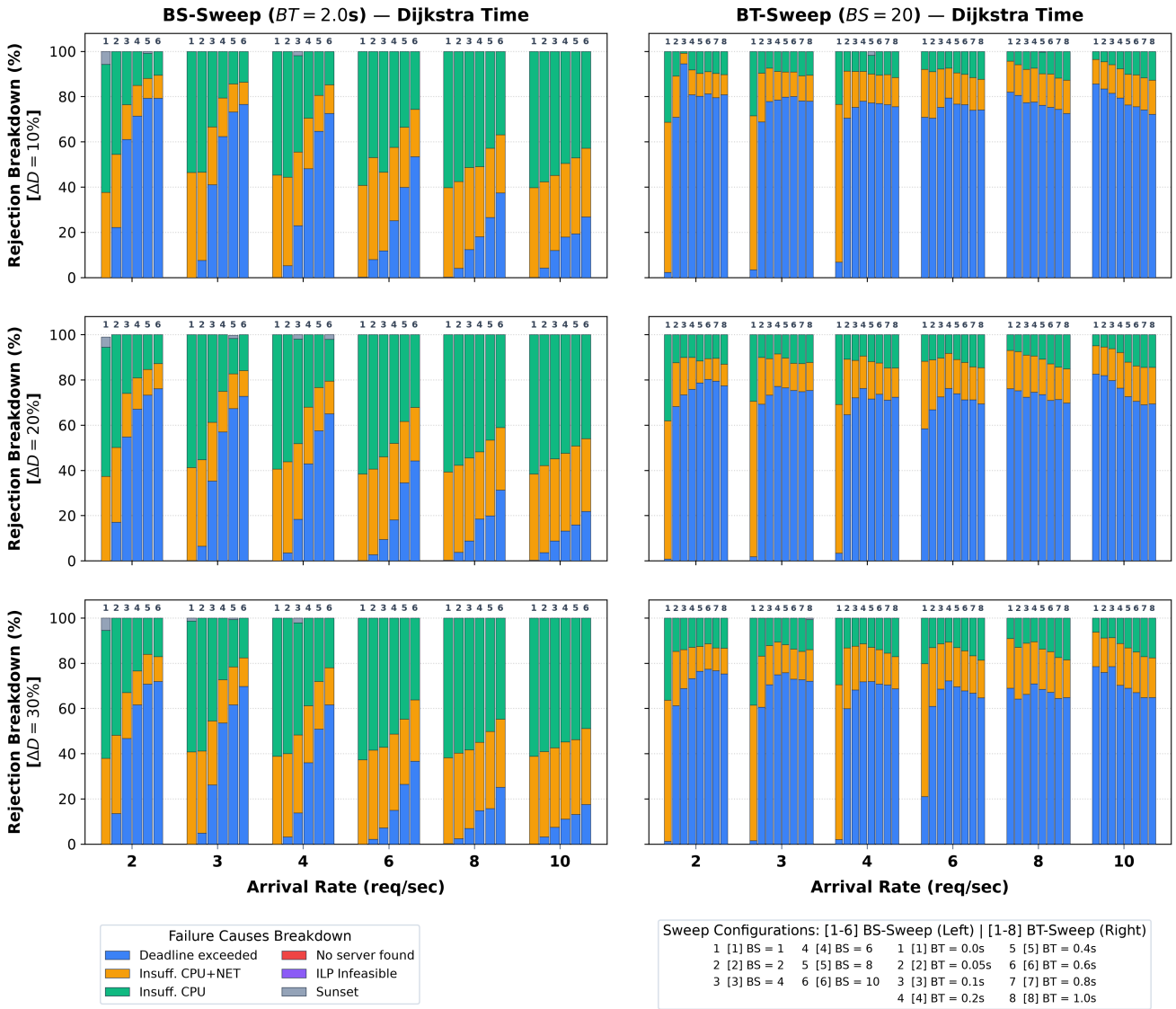

Figure 4.6. Rejection causes distribution across arrival rates under Time Mapping with Dijkstra Time routing, comparing BS-sweep ($BT = 2.0$ s) against BT-sweep ($BS = 20$) for $\Delta D \in \{10\%, 20\%, 30\%\}$.

4.2.3 Response Time Decomposition

Figures 4.7, 4.8, 4.9, and 4.10 show the end-to-end response time decomposed into Onboard Task Execution Time (T_{exec}) and Network/Wait Time ($T_{net+wait}$).

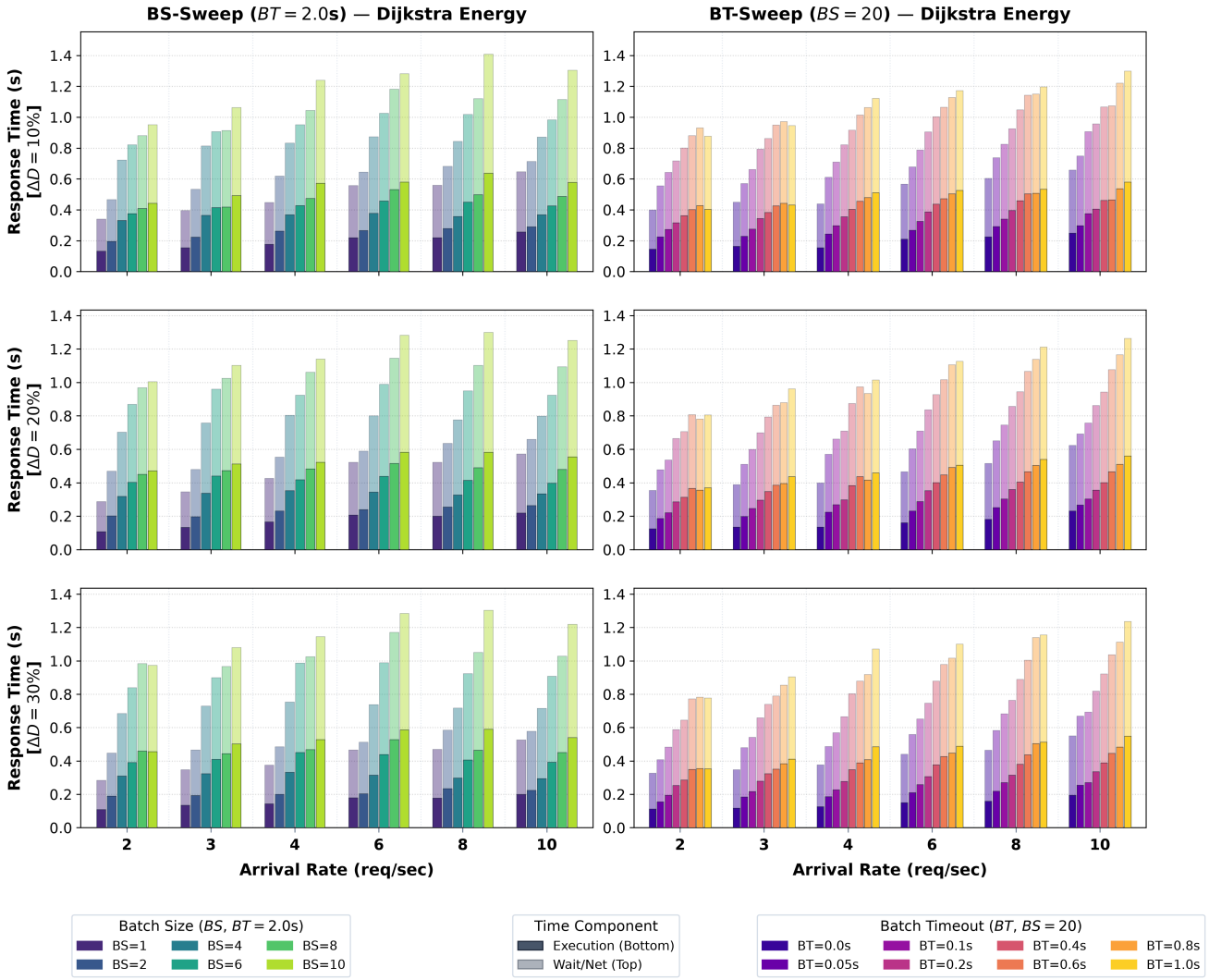

Figure 4.7. Response Time decomposition under Energy Mapping with Dijkstra Energy routing across arrival rates for BS-sweep and BT-sweep configurations under $\Delta D \in \{10\%, 20\%, 30\%\}$.

The empirical decompositions validate the analytical response time model:

$$T_{total} = T_{exec}(|R|, \lambda) + T_{net+wait}(\lambda, BS, BT) \tag{4.1}$$

where onboard execution time (T_{exec}, bottom darker segments) is strongly governed by the effective batch size $|R|$ dispatched to candidate nodes:

$$T_{exec} \approx \begin{cases} 0.15\text{–}0.25 \text{ s}, & \text{for small batches } (BS \leq 2 \text{ or } BT \leq 0.05 \text{ s}) \\ 0.45\text{–}0.62 \text{ s}, & \text{for large batches } (BS \geq 8 \text{ or } BT \geq 0.6 \text{ s}) \end{cases} \tag{4.2}$$

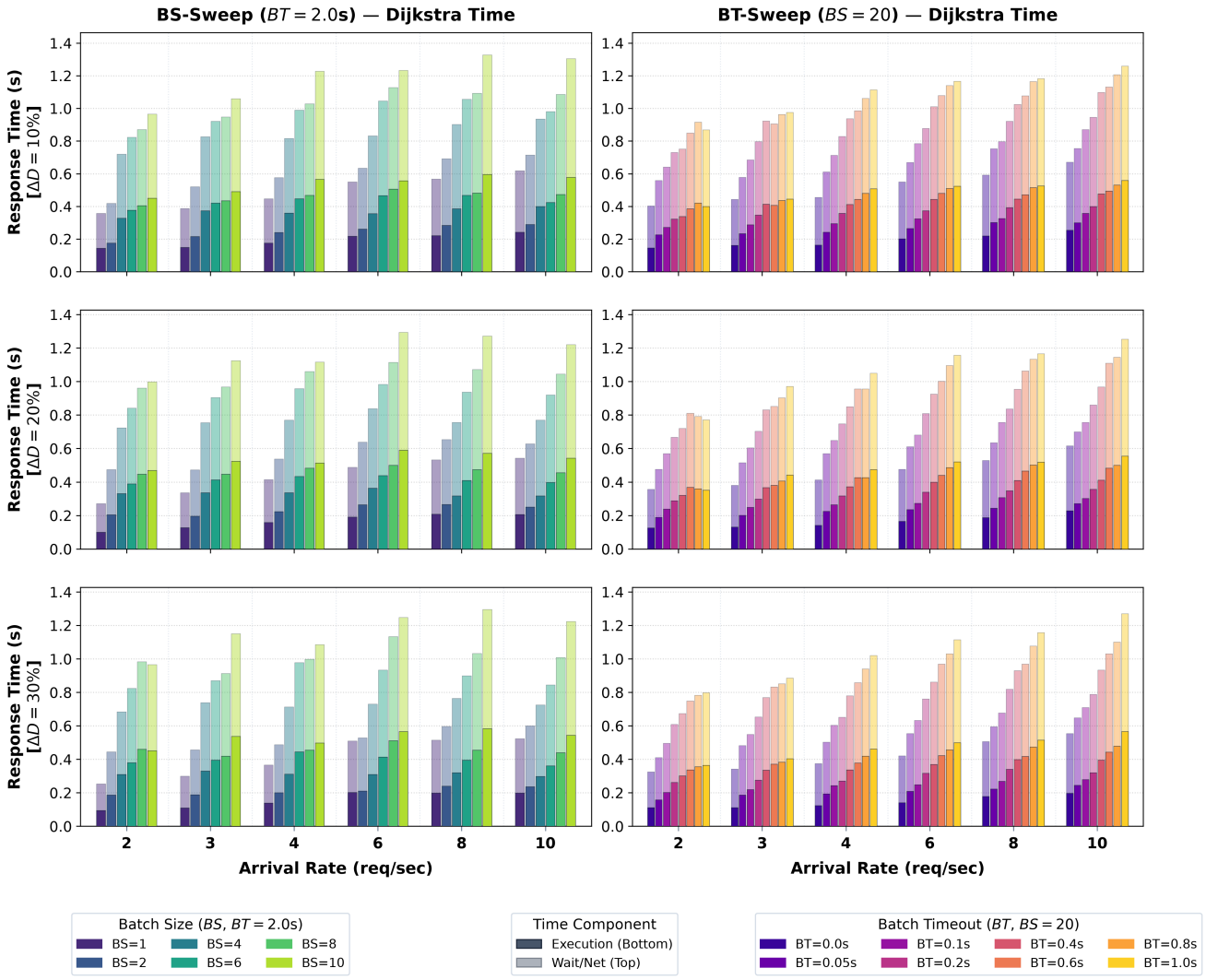

Figure 4.8. Response Time decomposition under Energy Mapping with Dijkstra Time routing across arrival rates for BS-sweep and BT-sweep configurations under $\Delta D \in \{10\%, 20\%, 30\%\}$.

When larger batches are dispatched simultaneously, multiple co-allocated tasks enter satellite CPU execution queues concurrently, driving up T_{exec}. This behavior is directly visible in the BT-sweep ($BS = 20$), where expanding BT from 0.0 s to 1.0 s allows more tasks to accumulate per cycle, increasing T_{exec} from ≈ 0.18 s to ≈ 0.58 s.

The transmission and coordinator wait time ($T_{net+wait}$, top translucent segments) governs the remaining latency:

- **Batch Size Sweep ($BT = 2.0$ s):** Small batch sizes limit total latency across all arrival rates ($T_{total} \approx 0.35$–0.55 s for $BS = 1$, and ≈ 0.40–0.65 s for $BS = 2$). For large aggregations ($BS \geq 8$), ingress queuing and multi-task ISL contention maintain elevated latency across the entire spectrum, spanning 0.95–1.05 s at $\lambda = 2$ req/s and expanding up to 1.25–1.42 s at $\lambda \geq 6$ req/s.

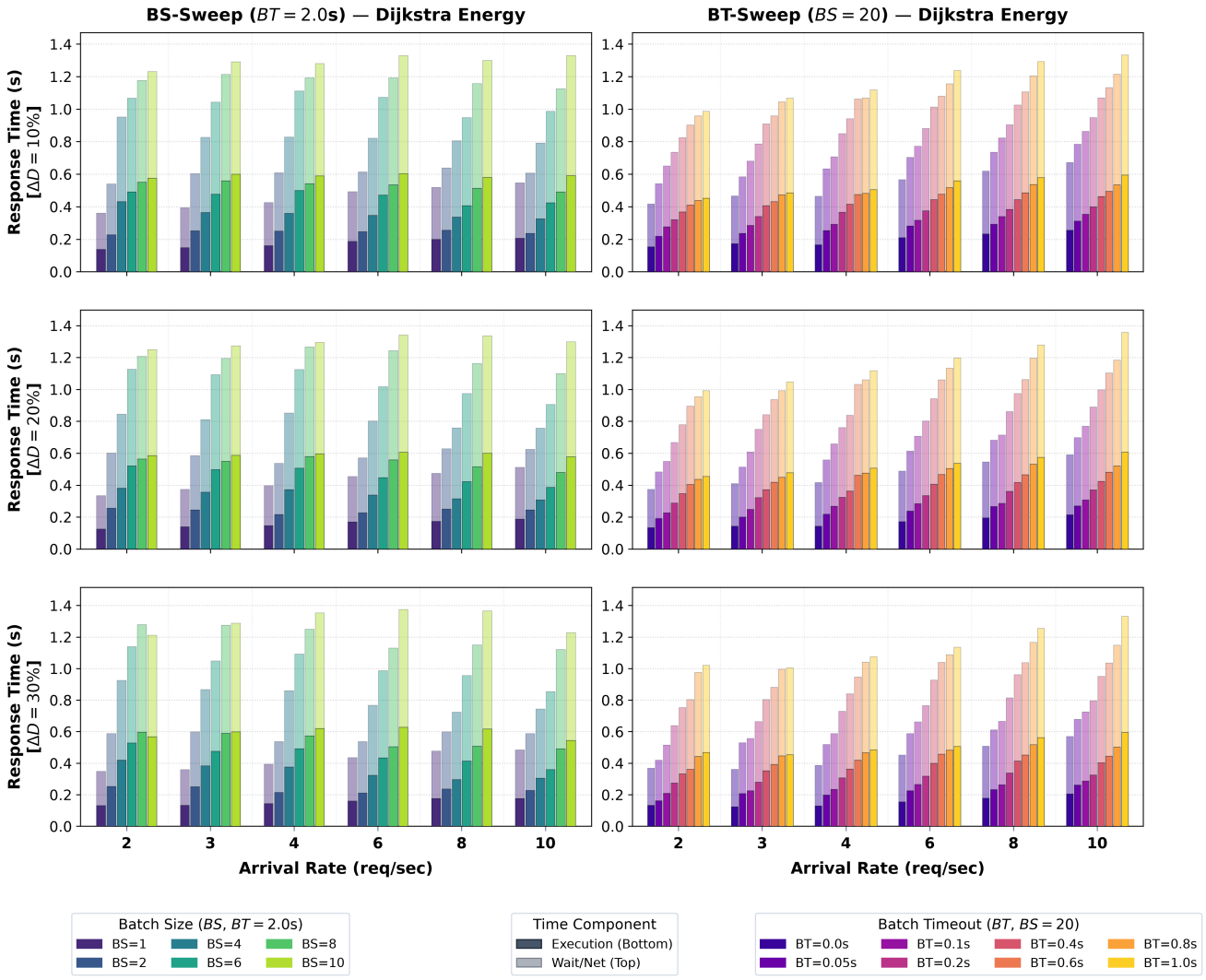

Figure 4.9. Response Time decomposition under Time Mapping with Dijkstra Energy routing across arrival rates for BS-sweep and BT-sweep configurations under $\Delta D \in \{10\%, 20\%, 30\%\}$.

- **Batch Timeout Sweep ($BS = 20$):** The timeout threshold sets an explicit upper bound on ingress queue dwell time. Instantaneous dispatch ($BT = 0.0$ s) yields the lowest latency ($T_{total} \approx 0.35$–0.65 s), closely followed by reactive dispatch ($BT = 0.05$ s, $T_{total} \approx 0.42$–0.73 s). As BT scales toward 1.0 s, $T_{net+wait}$ expands monotonically, driving total response time to 1.20–1.35 s under heavy load.

Finally, comparing across the relaxation levels ($\Delta D \in \{10\%, 20\%, 30\%\}$) confirms that while deadline relaxation affects task admission rates, the response time profiles of successfully scheduled tasks remain invariant.

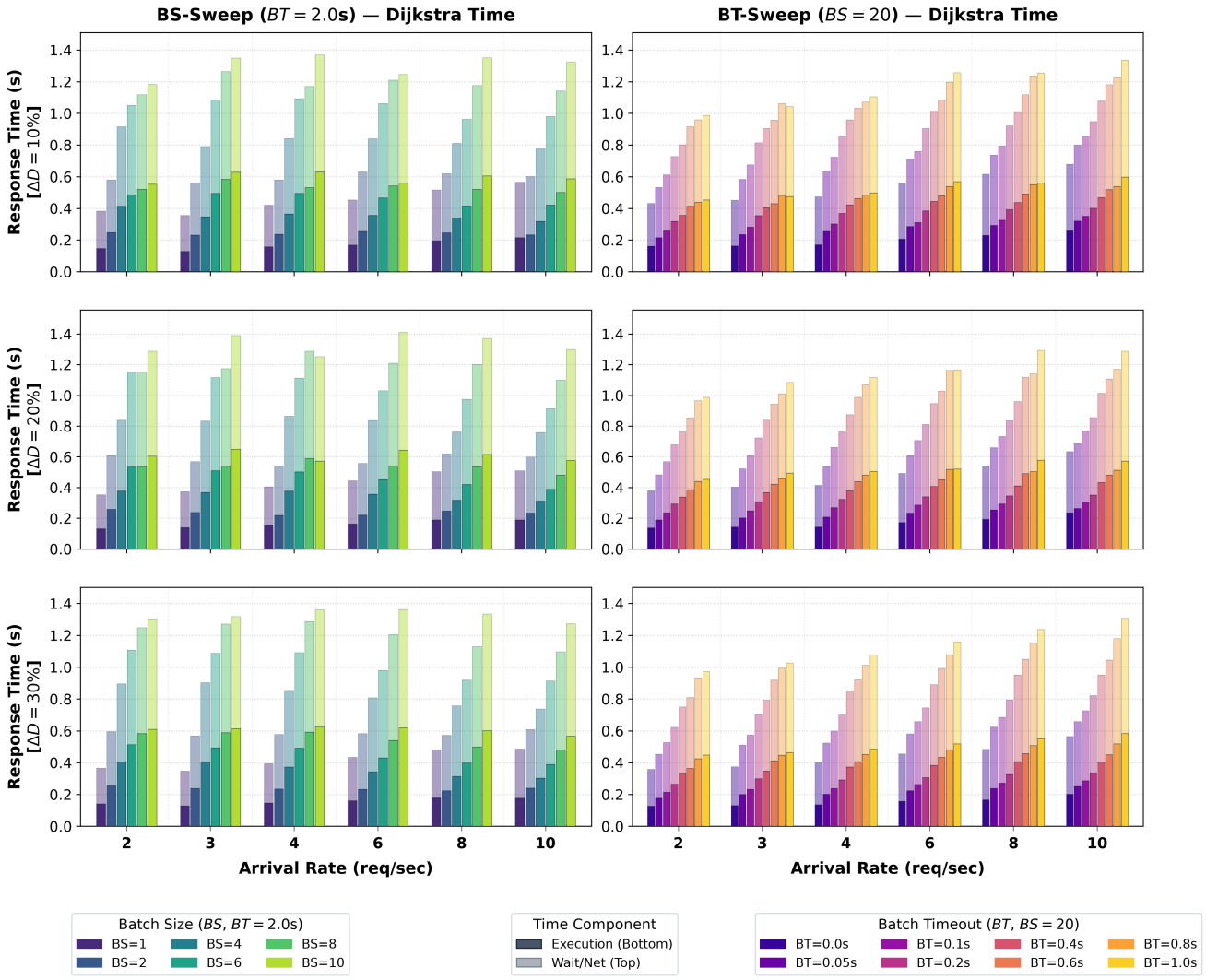

Figure 4.10. Response Time decomposition under Time Mapping with Dijkstra Time routing across arrival rates for BS-sweep and BT-sweep configurations under $\Delta D \in \{10\%, 20\%, 30\%\}$.

4.2.4 Selection of Viable Batching Archetypes

The parametric exploration establishes clear operational boundaries for centralized orchestration in dynamic LEO satellite networks [6, 1]. On the one hand, configurations relying on generous buffering thresholds ($BT \geq 0.2$ s or $BS \geq 6$) prove structurally unviable for delay-sensitive edge services: coordinator ingress dwell times systematically consume application slack times, resulting in severe deadline-induced drop rates (70%–85%, as shown in Figures **??** and **??**) and elevated end-to-end response times (> 1.0 s, Figures **??** and **??**) [6]. On the other hand, while pure instantaneous dispatch ($BT = 0.0$ s) eliminates coordinator buffer latency, it introduces severe architectural and scheduling limitations:

- **Degeneration into Myopic Greedy Allocation:** Structuring the optimiza-

tion problem around isolated single-task instances ($|R| = 1$) fundamentally collapses the combinatorial search space. The mathematical program degenerates into a single-request shortest-path and server selection routine, blinding the orchestrator to imminent concurrent demands. In dense arrival regimes where requests arrive within tens of milliseconds of each other, they are scheduled strictly sequentially without joint fleet awareness: the first request greedily claims the lowest-latency ISL hops and the closest edge compute cores, inadvertently forcing immediately subsequent requests onto congested multi-hop detours or causing their outright rejection due to newly saturated link capacities.

- **Localized Onboard Queue Inflation and Deadline Collapse:** Because isolated single-task scheduling lacks joint visibility across concurrent demands, the solver repeatedly steers closely arriving tasks toward the same nominally optimal Satellite Edge Nodes. Although coordinator ingress buffering is eliminated ($T_{wait} = 0$), this uncoordinated dispatch rapidly congests destination node execution queues ($W_{cpu,i}$). As arrival rates exceed the processing capacity of individual satellites, the cumulative queuing delay onboard the spacecraft pushes overall task completion beyond strict application deadlines. This structural lack of multi-task coordination explains the steep throughput collapse observed under unbuffered dispatch at peak load ($\approx$ 41%–42% success at $\lambda = 10$ req/s, Figures 4.1 and 4.2).

To harness the multi-task optimization potential of the centralized ILP formulation while maintaining strict temporal feasibility, an effective batching policy must strike an operational balance: it must accumulate enough requests to enable joint combinatorial scheduling, yet flush the queue rapidly enough to preserve application deadline margins.

We therefore select two complementary batching archetypes that capture these operational trade-offs for subsequent objective weight micro-tuning:

1. **Reactive Fast-Dispatch ($BS = 20, BT = 0.05$ s):** This policy operates as a rate-limiting micro-batcher with an aggressive 50 ms timeout. Because the inter-arrival window across the evaluated arrival spectrum ($\lambda \in [2, 10]$ req/s) ranges between 100 ms and 500 ms, the timeout triggers almost continuously, dispatching requests either individually or in tightly coupled burst pairs ($|R| \in \{1, 2\}$). By enforcing an explicit 50 ms ceiling on ingress delay, it caps end-to-end response times below 0.73 s (Figures **??** and **??**) and prevents uncoordinated single-task allocation by coalescing sub-millisecond arrival bursts into a unified mathematical program, significantly reducing intermediate ISL contention and localized node congestion compared to unbuffered dispatch.

2. **Controlled Small-Batching ($BS = 2, BT = 2.0$ s):** This policy establishes the minimal non-trivial combinatorial optimization horizon ($|R| = 2$) with a fallback timeout of 2.0 s. At operational loads ($\lambda \geq 4$ req/s), the batch size threshold is fulfilled within 0.10–0.25 s, triggering execution well before the timeout can expire. This provides the ILP solver with a joint optimization window across pairs of competing requests, enabling spatial load balancing [18]

(e.g., allocating complementary CPU-intensive and Data-intensive tasks to disjoint SENs and non-overlapping ISL paths). This pairwise coupling bounds deadline-induced failures to $\leq 10\%$ at medium-to-high loads (Figures **??** and **??**) while maintaining compact response times ($T_{total} \leq 0.65$ s, Figures **??** and **??**).

4.3 Micro-Tuning: Multi-Objective Weight Calibration under Stress ($\gamma \times \alpha$)

Having isolated the viable batching configurations ($BS = 2, BT = 2.0$ s and $BS = 20, BT = 0.05$ s), we calibrate the scalarization weights of the multi-objective function [12]:

$$\min Z = \begin{cases} \alpha Z_{time} + \beta Z_{energy} + \gamma Z_{sunset}, & \text{if primary objective is Time} \\ \alpha Z_{energy} + \beta Z_{time} + \gamma Z_{sunset}, & \text{if primary objective is Energy} \end{cases} \tag{4.3}$$

where $\beta = 1.0$ serves as the secondary objective normalization, the primary objective weight is evaluated across $\alpha \in \{10, 50, 1000\}$, and the orbital sunset deterrent penalty is evaluated across $\gamma \in \{2, 4, 6, 8, 10, 1000, 5000, 10000\}$. Note that these scalar weights are distinct from the workload profiling vectors defined in Section 4.1.1. The cross-sensitivity experiment was conducted under heavy stress regimes ($\lambda = 8$ req/s and $\lambda = 10$ req/s), comparing both Dijkstra-Time and Dijkstra-Energy link routing metrics [23, 6].

4.3.1 Impact on Success Rate

Figures 4.11, 4.12, 4.13, and 4.14 depict the Task Success Rate across the ($\gamma \times \alpha$) parameter space for $\lambda = 8$ req/s and $\lambda = 10$ req/s, respectively.

The empirical trends indicate:

- **Decisive Impact of Primary Weight $\alpha = 1000$ in Latency-Centric Batching:** The influence of weight scaling is acutely concentrated in batching configurations governed by latency minimization ($Time \mid BS : 2, BT : 2.0$). In these panels, setting $\alpha = 1000$ (green dash-dotted lines) yields a stable $\approx 71\%$–74% completion rate at $\lambda = 8$ req/s and $\approx 67\%$–68% at $\lambda = 10$ req/s, substantially outperforming the baseline weights $\alpha = 10$ ($\approx 50\%$–53%) and $\alpha = 50$ ($\approx 55\%$–58%). Because the solver jointly optimizes scheduling over pairs of requests, enforcing an aggressive primary weight compels the branch-and-cut routine to strictly prioritize deadline compliance over secondary scalarization terms.

- **Optimal Sunset Trade-off Window $\gamma \in [6, 10]$:** In configurations responsive to weight calibration, peak throughput consistently concentrates within $\gamma \in [6, 10]$. Under Dijkstra-Time at $\lambda = 8$ req/s, $\alpha = 1000$ achieves its maximum completion rate at $\gamma = 10$ ($\approx 73.5\%$), while maintaining an equally robust plateau under Dijkstra-Energy ($\approx 72.5\%$). Within this parameter window, the orbital penalty effectively discourages assignments to borderline satellites without distorting overall route efficiency.

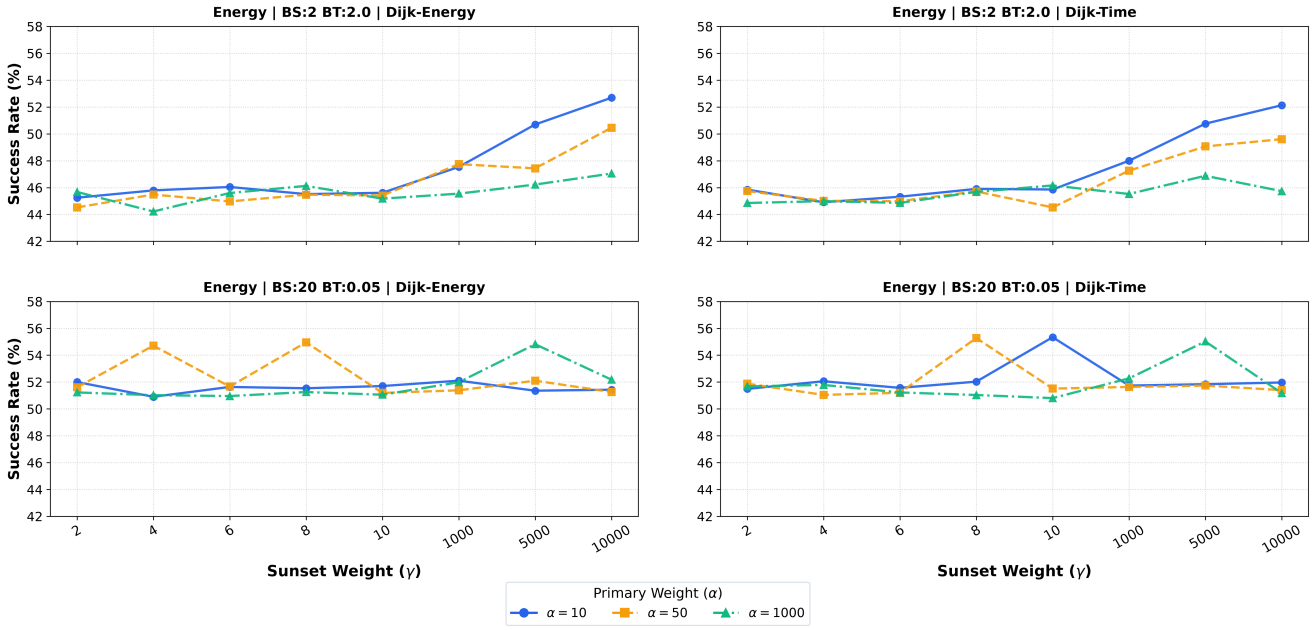

Figure 4.11. Cross-sensitivity Success Rate ($\gamma \times \alpha$) under Energy Mapping at $\lambda = 8$ req/s across batching configurations and Dijkstra metrics.

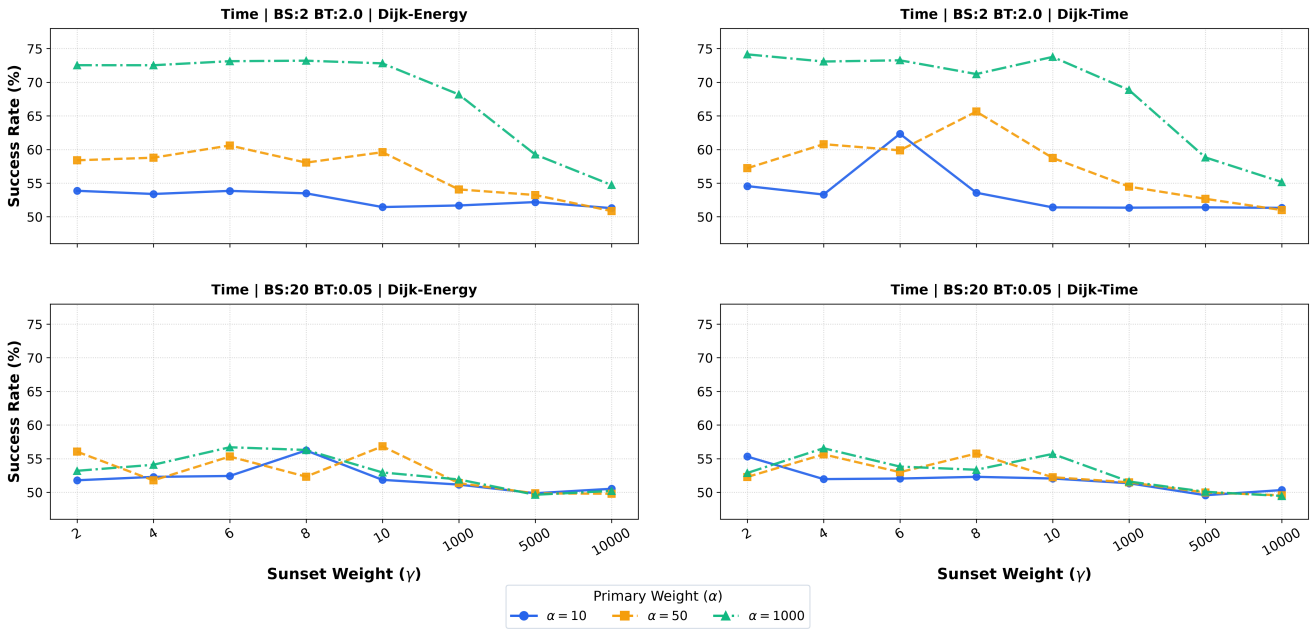

Figure 4.12. Cross-sensitivity Success Rate ($\gamma \times \alpha$) under Time Mapping at $\lambda = 8$ req/s across batching configurations and Dijkstra metrics.

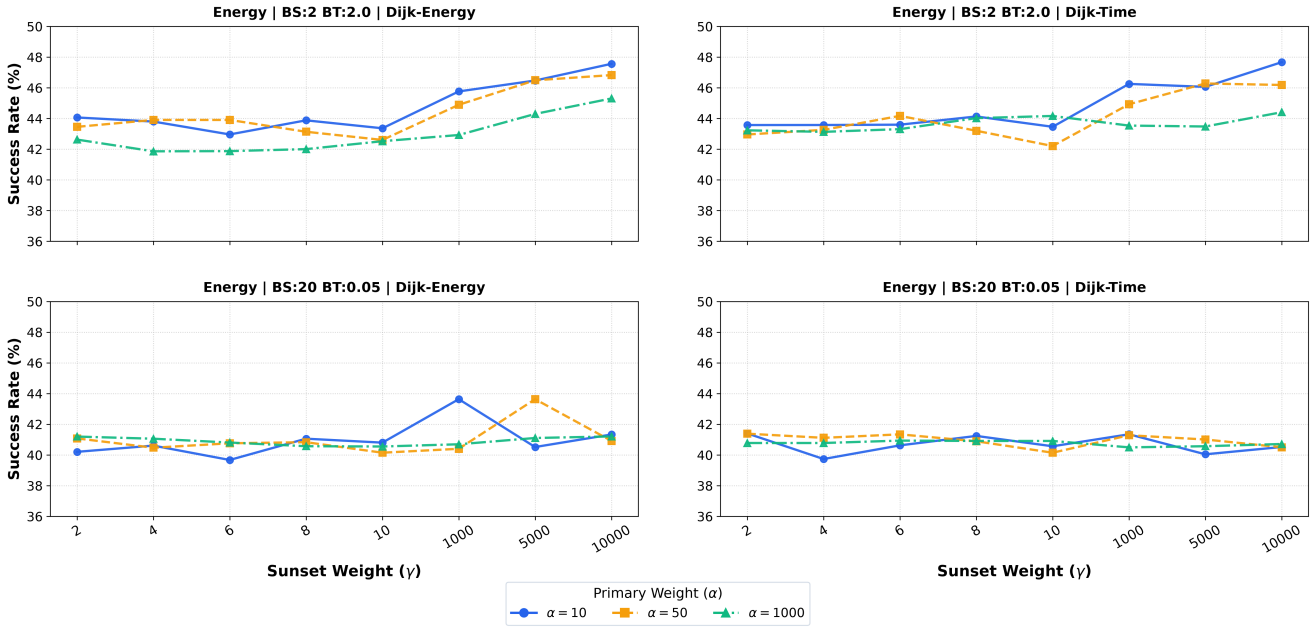

Figure 4.13. Cross-sensitivity Success Rate ($\gamma \times \alpha$) under Energy Mapping at $\lambda = 10$ req/s across batching configurations and Dijkstra metrics.

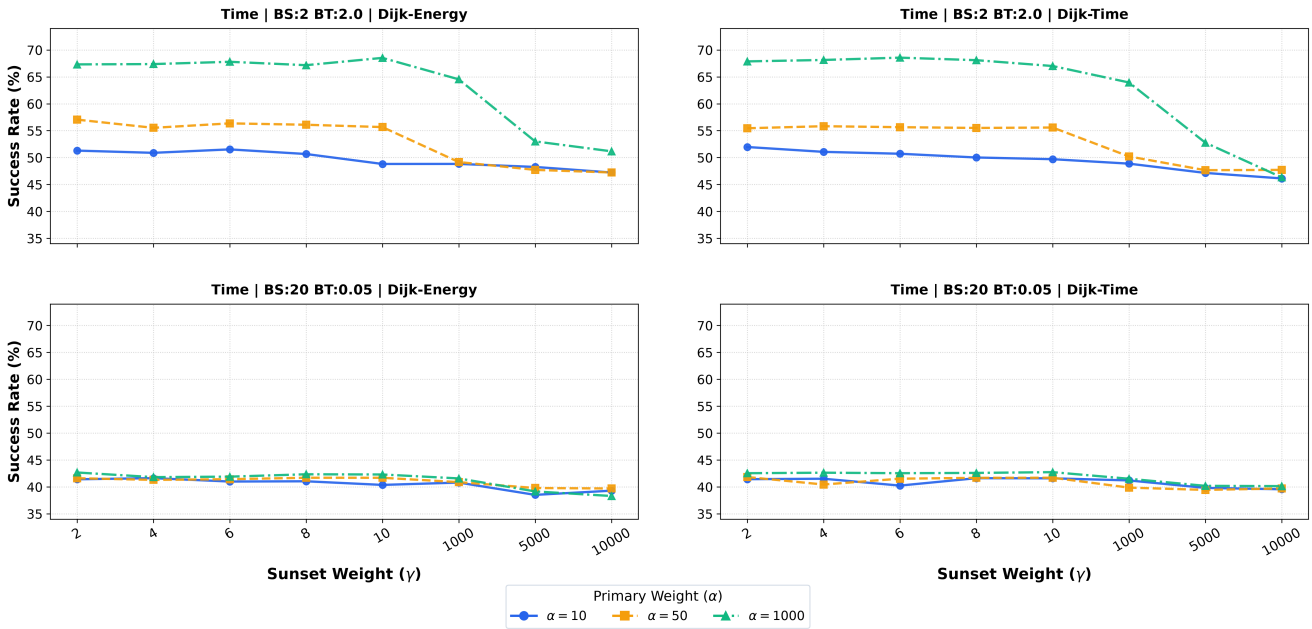

Figure 4.14. Cross-sensitivity Success Rate ($\gamma \times \alpha$) under Time Mapping at $\lambda = 10$ req/s across batching configurations and Dijkstra metrics.

- **Performance Degradation in Over-Penalized Regimes ($\gamma \geq 1000$):** When the eclipse deterrent is set to extreme values ($\gamma \geq 1000$), the success rate of $\alpha = 1000$ under $BS = 2$ suffers a steep decline, retreating to $\approx 54\%$ at $\lambda = 8$ req/s and $\approx 46\%$–50% at $\lambda = 10$ req/s, completely erasing the advantage gained over smaller weights. An inflated penalty term over-constrains the optimization search space: the solver becomes overly cautious, avoiding viable satellites nearing the eclipse boundary and rejecting requests that could otherwise complete safely.
- **Parametric Invariance in Fast-Dispatch and Energy Mapping:** Conversely, under fast-dispatch regimes ($BS : 20, BT : 0.05$) and *Energy Mapping*, the success rate exhibits structural invariance across all (α, γ) combinations, remaining flat at $\approx 51\%$–54% at $\lambda = 8$ req/s and $\approx 40\%$–42% at $\lambda = 10$ req/s.

4.3.2 Failure Diagnostics and Resource Bottlenecks

Figures 4.15, 4.16, 4.17, and 4.18 show the failure cause distributions across the $(\gamma \times \alpha)$ test matrix.

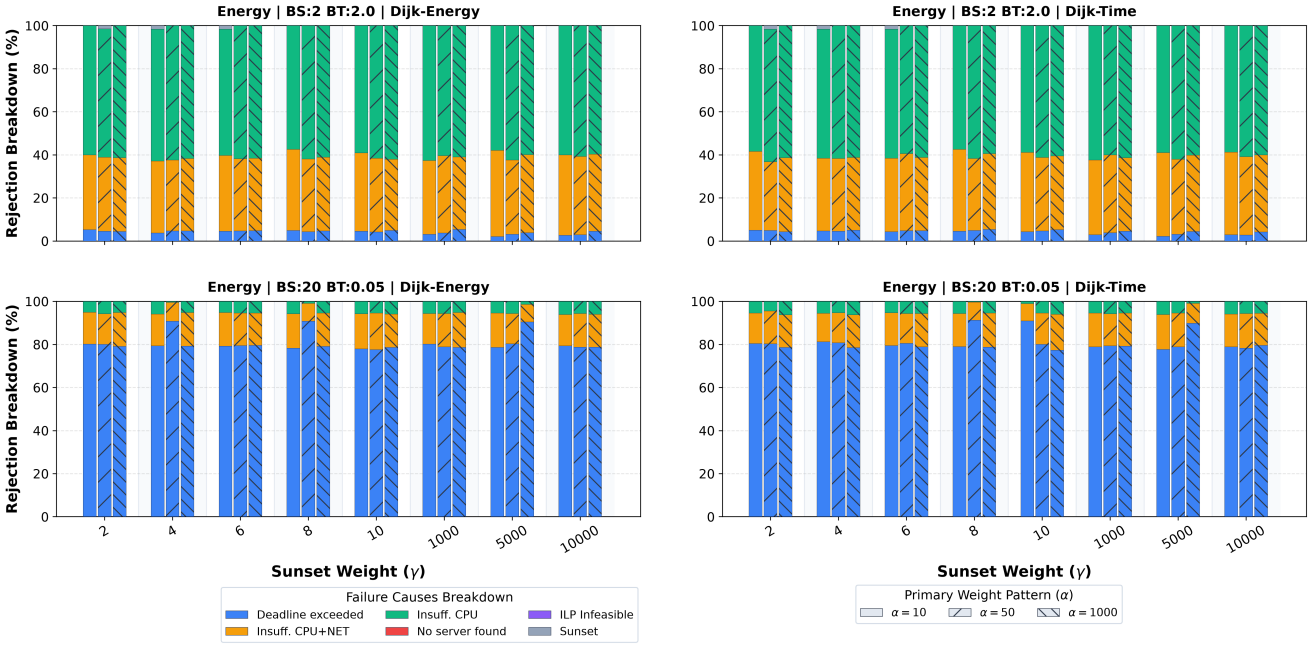

Figure 4.15. Distribution of rejection causes across the $(\gamma \times \alpha)$ parameter space under Energy Mapping at $\lambda = 8$ req/s.

The failure diagnostics reveal the exact operational boundary of the orchestrator under stress:

- **Suppression of Algorithmic and Eclipse Failures in the Operational Window:** Across all configurations within the viable penalty range $\gamma \in [2, 10]$, task rejections categorized as *Sunset*, *ILP Infeasible*, and *No Server Found* remain strictly at 0.0%. Setting $\gamma = 10$ provides a definitive soft barrier against orbital sunset transitions while preserving mathematical feasibility

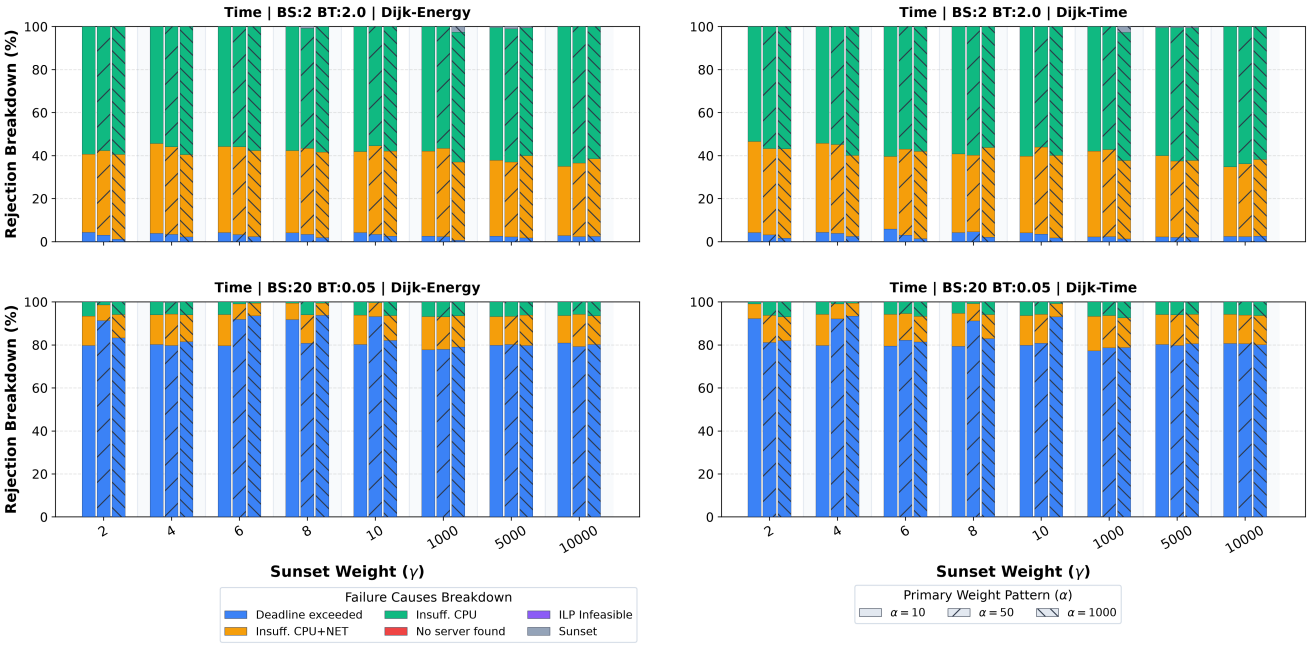

Figure 4.16. Distribution of rejection causes across the ($\gamma \times \alpha$) parameter space under Time Mapping at $\lambda = 8$ req/s.

across all candidate nodes. Residual *Sunset* drops ($\approx 5\%$) emerge exclusively under extreme over-penalization ($\gamma \in \{1000, 5000\}$) combined with low primary weights ($\alpha = 10$) under Energy Mapping at $\lambda = 10$ req/s, where an unbalanced objective distorts feasibility tracking.

- **Capacity Bottlenecks vs. Deadline Limits:** Across the parameter space, rejections are partitioned according to batching policy:
  - For controlled small-batching ($BS = 2, BT = 2.0$ s), drops are overwhelmingly capacity-bounded: *Insufficient CPU* accounts for $\approx 55\%$–60% (green), while *Insufficient CPU+NET* accounts for $\approx 35\%$–40% (orange), with *Deadline Exceeded* (blue) remaining strictly confined below 5%.
  - Conversely, reactive fast-dispatch ($BS = 20, BT = 0.05$ s) remains heavily dominated by *Deadline Exceeded* ($\approx 80\%$–84%, blue) due to onboard queuing contention under continuous single-task arrivals.

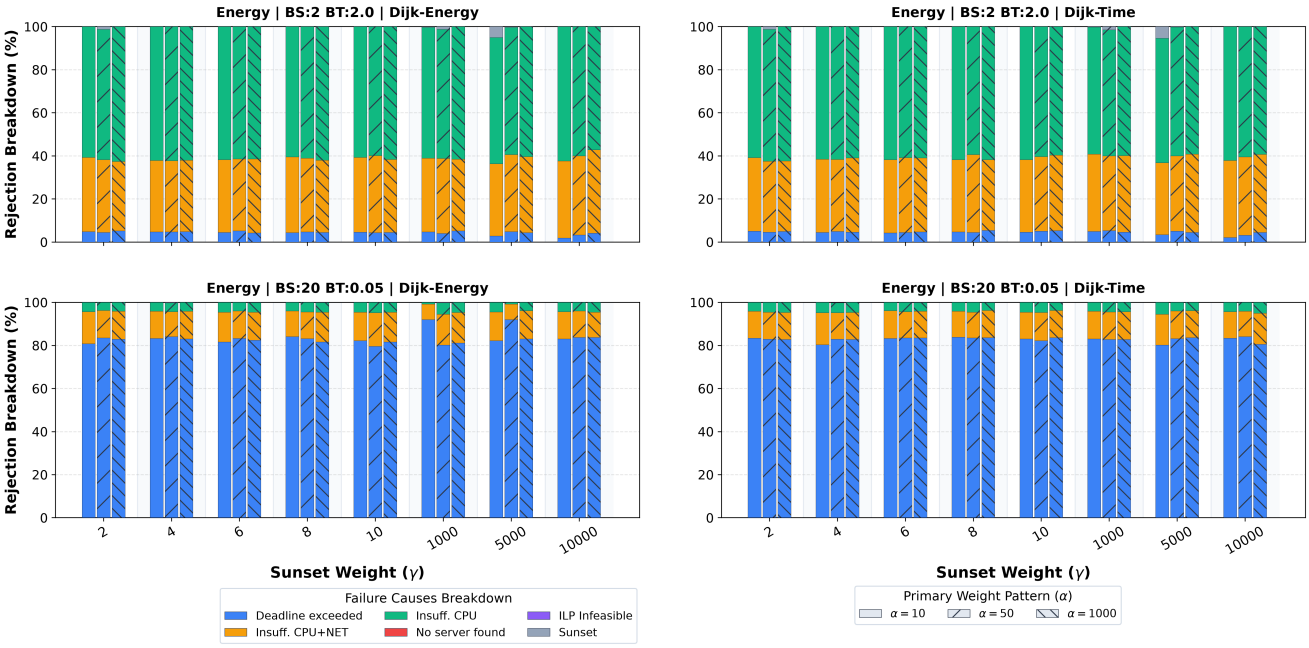

Figure 4.17. Distribution of rejection causes across the ($\gamma \times \alpha$) parameter space under Energy Mapping at $\lambda = 10$ req/s.

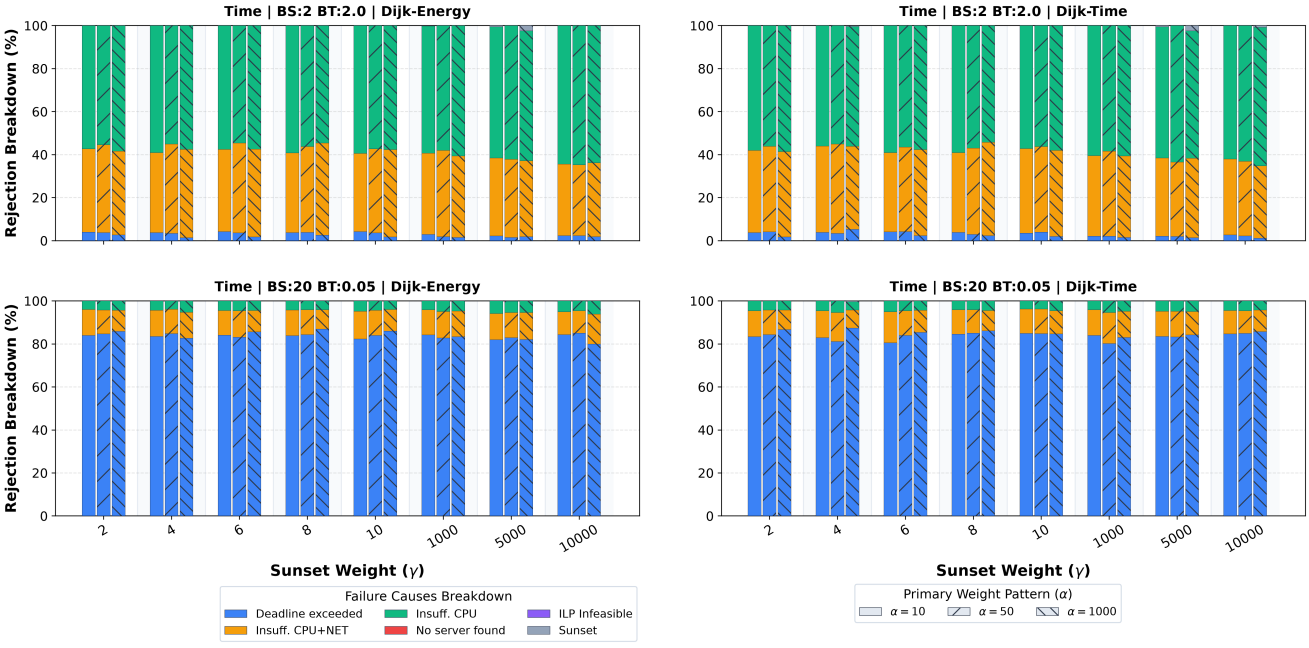

Figure 4.18. Distribution of rejection causes across the ($\gamma \times \alpha$) parameter space under Time Mapping at $\lambda = 10$ req/s.

4.3.3 Latency Inflation and Path-Bloat Dynamics

Figures 4.19, 4.20, 4.21, and 4.22 present the response time decomposition across the ($\gamma \times \alpha$) configurations into onboard task execution (T_{exec}) and network transmission/queuing delay ($T_{net+wait}$).

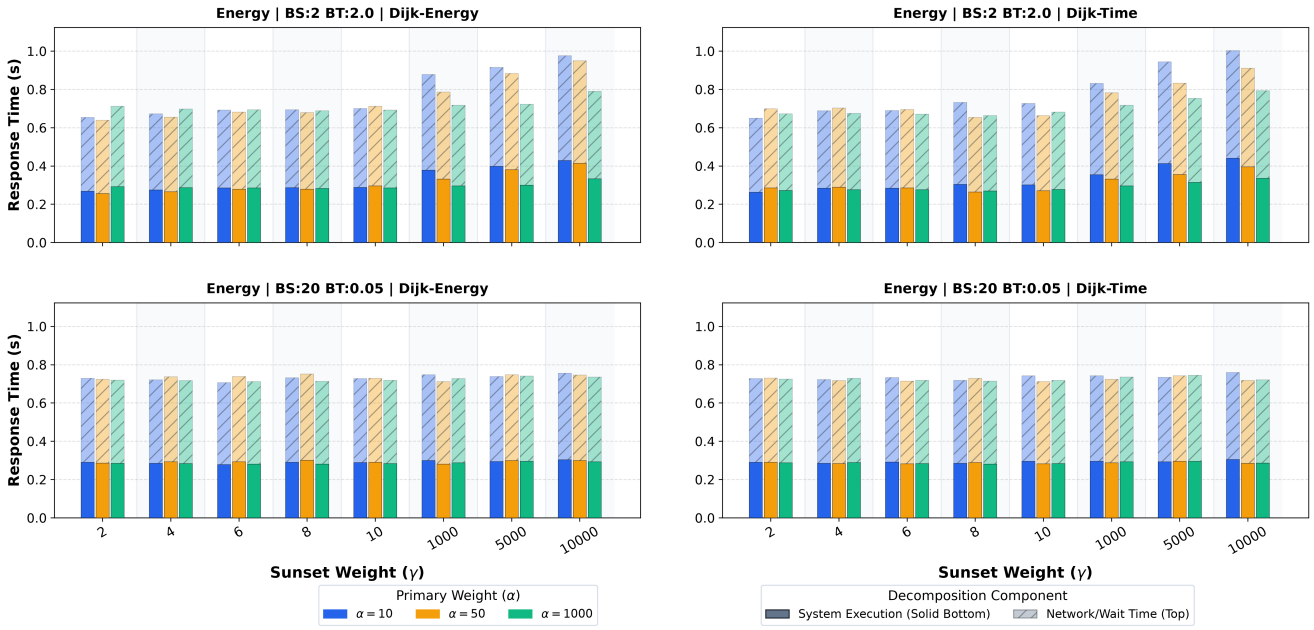

Figure 4.19. Response Time decomposition across the ($\gamma \times \alpha$) parameter space under Energy Mapping at $\lambda = 8$ req/s.

These results illustrate the path-bloat phenomenon under oversized weights:

- **Stable Operating Latency within $\gamma \in [2, 10]$:** In the recommended penalty region ($\gamma \leq 10$), total response time remains tightly bounded around $\approx$ 0.58–0.70 s for $BS = 2$ and $\approx$ 0.72–0.82 s for $BS = 20$. Onboard task execution time on the assigned edge node (T_{exec}) remains stable and compact, consistently requiring 0.22–0.32 s across all operational runs.
- **Path-Bloat Surge at $\gamma \geq 1000$ in Small-Batching:** When γ escalates to 1000–10000 in $BS = 2$ configurations, total response time surges to 0.95–1.15 s, representing an approximate 55% latency penalty. The stacked breakdown confirms that this surge is driven almost exclusively by the *Network/Wait Time* component (which swells from $\approx$ 0.35–0.42 s to over 0.60–0.70 s). To avoid the disproportionate penalty term γZ_{sunset}, the ILP orchestrator detours packets through circuitous multi-hop ISL paths or delegates execution to distant nodes, inflating propagation delay and queue buildup.

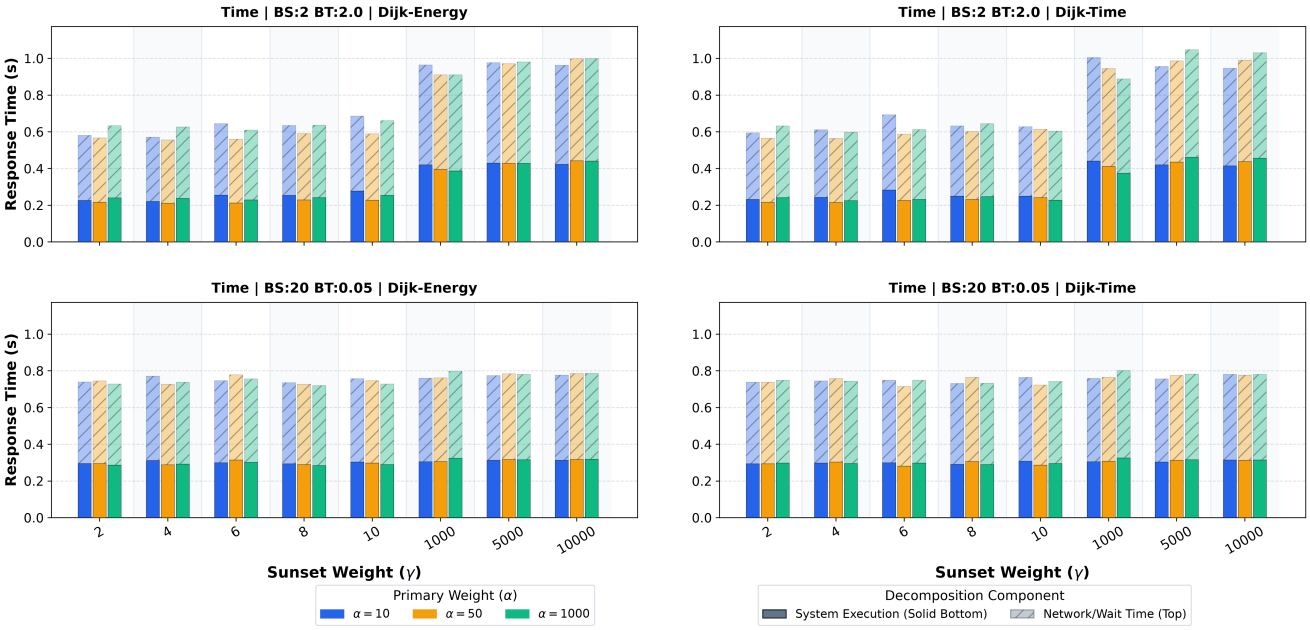

Figure 4.20. Response Time decomposition across the ($\gamma \times \alpha$) parameter space under Time Mapping at $\lambda = 8$ req/s.

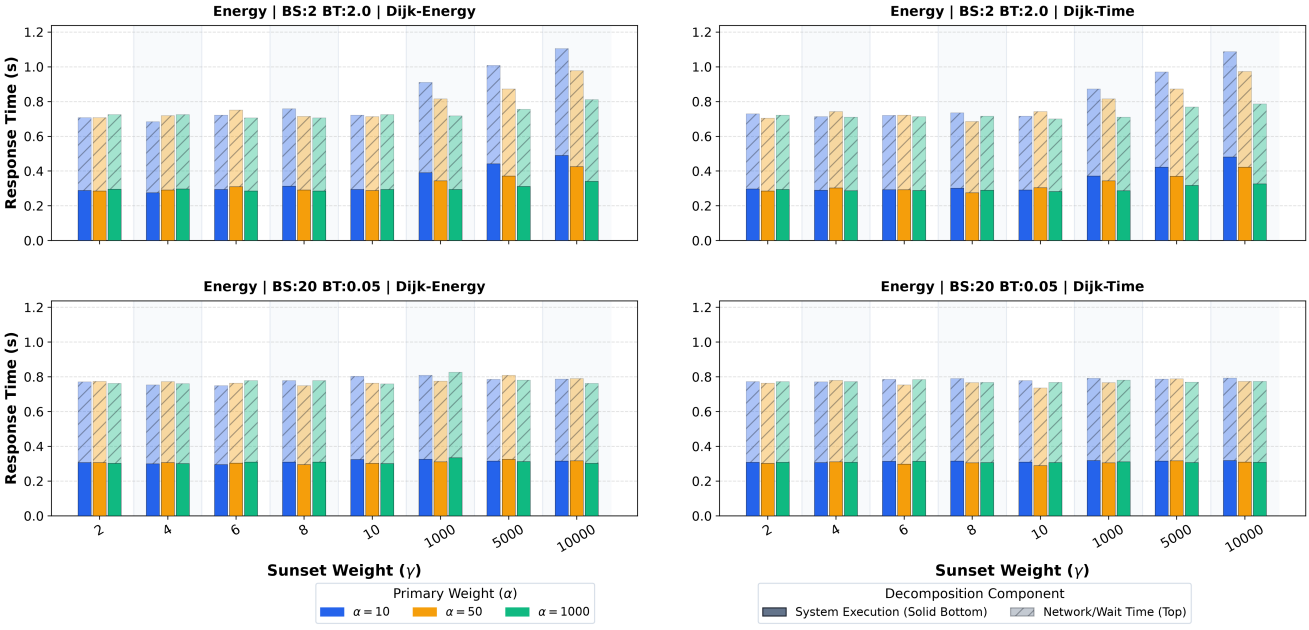

Figure 4.21. Response Time decomposition across the ($\gamma \times \alpha$) parameter space under Energy Mapping at $\lambda = 10$ req/s.

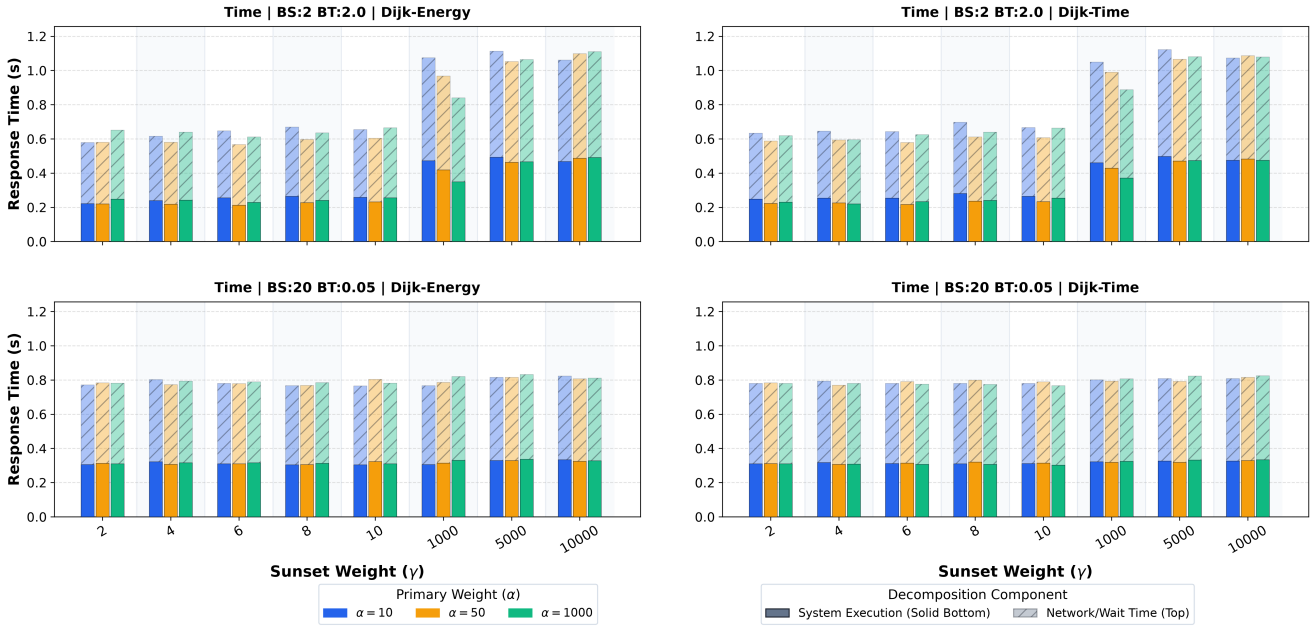

Figure 4.22. Response Time decomposition across the $(\gamma \times \alpha)$ parameter space under Time Mapping at $\lambda = 10$ req/s.

4.3.4 Selection of Calibrated Objective Weights

Correlating the success rate, failure distributions, and latency metrics identifies the optimal operating parameter set:

$$\alpha^* = \mathbf{1000}, \quad \gamma^* = \mathbf{10} \quad (\text{with } \gamma = \mathbf{8} \text{ as a viable equivalent}) \tag{4.4}$$

This combination provides an ideal operational setting: $\alpha = 1000$ ensures that physical deadlines and compute capacity dominate solver decisions, while $\gamma = 10$ provides maximal protection against eclipse transitions without inverting solver priorities or triggering the path-bloat latency penalty observed when $\gamma \geq 1000$.

4.4 Consolidated Validation: Pre- vs. Post-Optimization Benchmark

To validate the multi-objective calibration and quantify the performance gains achieved over the initial setup, we execute a comparative validation benchmark. We re-evaluate the two selected operational configurations across the entire arrival rate spectrum ($\lambda \in [2, 10]$ req/s) under Dijkstra-Time routing with $\Delta D = 10\%$, comparing:

- **Pre-Optimization Baseline:** The initial configuration employing default objective weights ($\alpha = 10, \gamma = 5$) as evaluated during the macro-tuning sweep.
- **Post-Optimization Model:** The calibrated configuration integrating the tuned scalarization weights ($\alpha = 1000, \gamma = 10$) identified in the cross-sensitivity campaign.

4.4.1 Success Rate Recovery and Mapping Divergence

Figure 4.23 presents the direct comparison of Task Success Rate between the baseline and calibrated configurations under both Energy Mapping and Time Mapping.

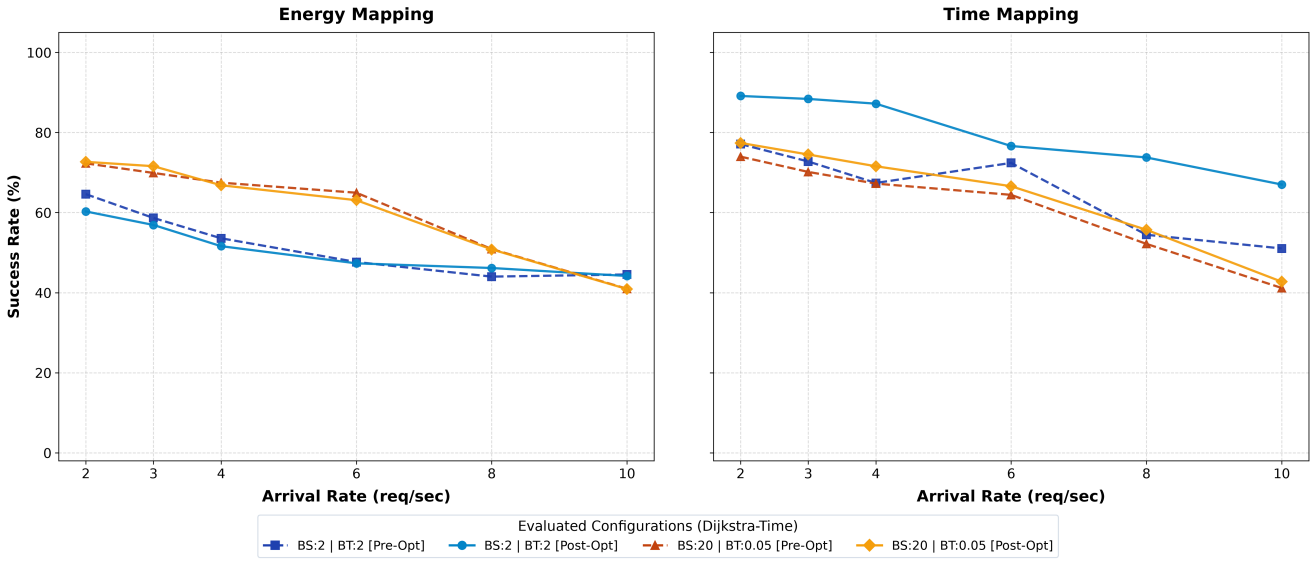

Figure 4.23. Success Rate comparison: Pre-optimization baseline vs. Post-optimization calibrated model for $BS = 2, BT = 2.0$ s and $BS = 20, BT = 0.05$ s under Dijkstra-Time routing ($\Delta D = 10\%$).

The empirical evaluation reveals a clear operational divide depending on the primary mapping objective:

- **Throughput Recovery in Time Mapping:** For the batch-controlled regime ($BS = 2, BT = 2.0$ s), the calibrated weights unlock substantial throughput improvements across the traffic spectrum. The success rate rises from $\approx 77.0\%$ to $\approx 89.0\%$ at $\lambda = 2$ req/s (+12.0%), from $\approx 67.5\%$ to $\approx 87.0\%$ at $\lambda = 4$ req/s (+19.5%), from $\approx 54.0\%$ to $\approx 73.5\%$ at $\lambda = 8$ req/s (+19.5%), and from $\approx 51.0\%$ to $\approx 67.0\%$ under peak load ($\lambda = 10$ req/s, +16.0%). Enforcing $\alpha = 1000$ compels the branch-and-cut solver to prioritize transmission and execution timeliness above all else, maximizing deadline compliance. The localized baseline plateau at $\lambda = 6$ req/s ($\approx 72.5\%$) reflects the topological alignment of Dijkstra-Time observed in Section 4.2.1.

- **Energy Mapping Saturation:** Conversely, under Energy Mapping, scaling $\alpha = 1000$ yields no throughput recovery: the success rate of $BS = 2$ tracks the baseline within a narrow band across all traffic loads (60.0% vs. 64.5% at $\lambda =$ 2 req/s, converging to $\approx 44.0\%$ at $\lambda = 10$ req/s). Under an aggressive energy weight, the solver seeks strictly low-power computation and minimal radio transmission paths, frequently routing requests to energy-efficient satellites that exhibit longer computational queues. This confirms that scaling $\alpha = 1000$ is structurally effective only when aligned with a latency-driven objective.

- **Robustness in Fast-Dispatch Configurations:** For the reactive configuration ($BS = 20, BT = 0.05$ s), performance exhibits minor adjustments under Time Mapping (improving from $\approx 52.0\%$ to $\approx 55.5\%$ at $\lambda = 8$ req/s and from $\approx 41.0\%$ to $\approx 42.5\%$ at $\lambda = 10$ req/s), while remaining virtually identical in Energy Mapping across all arrival rates ($\approx 72.5\%$ at $\lambda = 2$ req/s down to $\approx 41.0\%$ at $\lambda = 10$ req/s). Because the 50 ms timer dispatches requests almost immediately in micro-batches (1–2 tasks), the orchestrator has limited combinatorial freedom, making this policy intrinsically robust to internal weight tuning.

4.4.2 Rejection Causes Evolution

Figure 4.24 illustrates the breakdown of failure causes across all evaluated configurations, comparing the pre-optimization baseline against the calibrated post-optimization model.

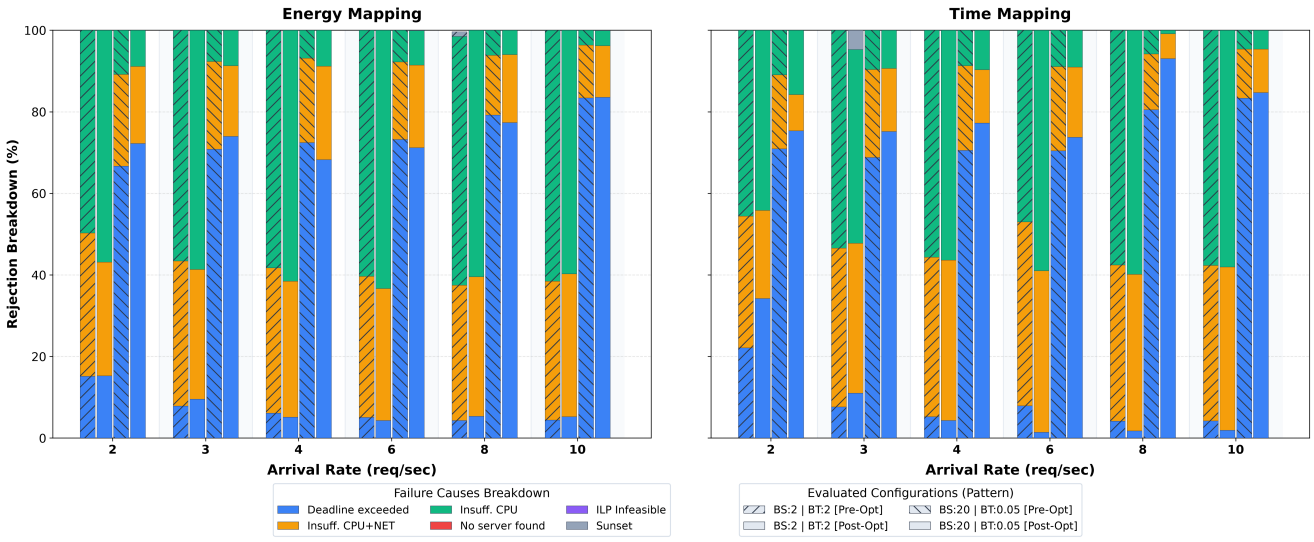

Figure 4.24. Rejection causes distribution comparison: Pre-optimization baseline vs. Post-optimization calibrated model across arrival rates ($\Delta D = 10\%$).

The diagnostic breakdown highlights four distinct operational behaviors across mapping schemes and batching archetypes:

- **Load-Dependent Evolution in Time Mapping ($BS = 2, BT = 2.0$ s):** For controlled small-batching under Time Mapping, weight calibration shifts the nature of task drops according to traffic intensity:
  - *Low-Load Saturation ($\lambda \in [2, 3]$ req/s):* At $\lambda = 2$ req/s, the proportion of rejections attributed to *Deadline Exceeded* increases from 22% in the baseline to 34% in the calibrated model (and from 7% to 11% at $\lambda = 3$ req/s). This relative increase stems directly from the dramatic rise in overall throughput: because the calibrated model ($\alpha = 1000$) boosts the success rate from 77% to 89% at $\lambda = 2$ req/s, total rejections contract to a marginal 11% of incoming traffic. Within this tiny residual

pool of rejected requests, failures are concentrated on borderline tasks whose deadlines cannot be physically satisfied even with maximal latency prioritization.

– *High-Load Capacity Bounding ($\lambda \geq 4$ req/s):* As traffic scales, timing violations are almost entirely suppressed: *Deadline Exceeded* drops to 3% at $\lambda = 4$ req/s, reaches roughly 0% at $\lambda \in \{6, 8\}$ req/s, and remains at $\approx 1\%$ at $\lambda = 10$ req/s. Consequently, task drops become strictly capacity-bounded, partitioned between *Insufficient CPU* (56%–60%, green) and *Insufficient CPU+NET* (40%–41%, orange).

- **Persistence of Queuing Bottlenecks in Fast-Dispatch ($BS = 20, BT = 0.05$ s):** In stark contrast to small-batching, the reactive fast-dispatch configuration remains overwhelmingly dominated by *Deadline Exceeded* across both mappings ($\approx$ 66%–75% at $\lambda = 2$ req/s, scaling up to $\approx$ 83%–92% at $\lambda \in [8, 10]$ req/s). The pre- and post-optimization bars exhibit nearly identical distributions. Because isolated single requests are dispatched without multi-task coordination, candidate satellites experience localized CPU execution queue buildup, pushing task completion beyond deadlines regardless of objective scalarization.

4.4.3 Response Time Containment and Execution Flattening

Figure 4.25 compares the stacked response time decomposition between baseline and calibrated models.

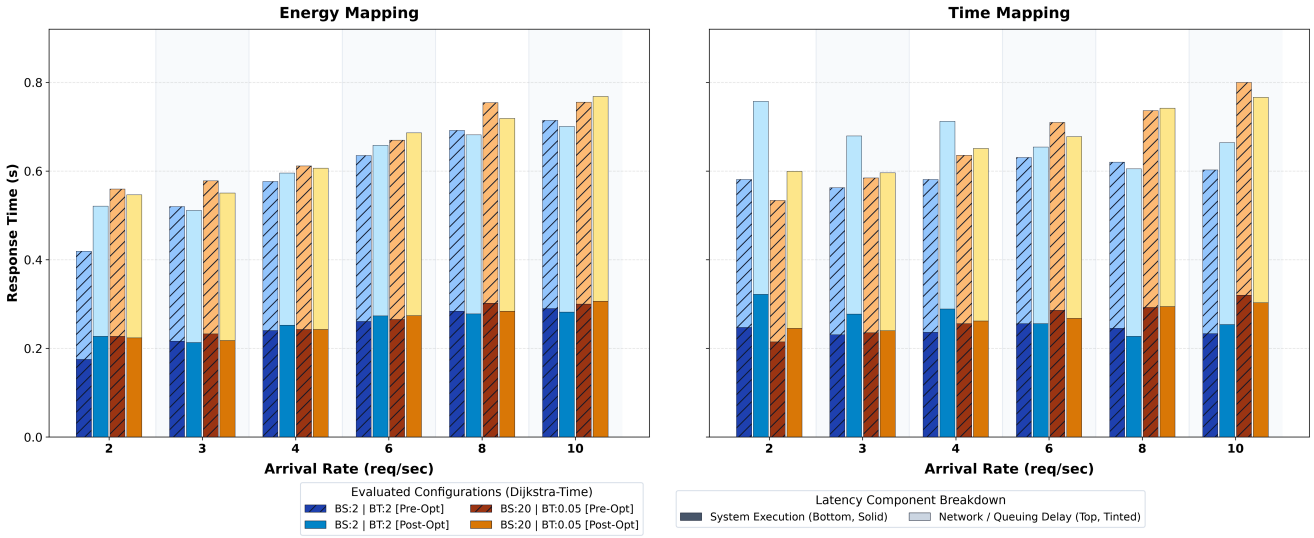

Figure 4.25. Response Time decomposition comparison: Pre-optimization baseline vs. Post-optimization calibrated model showing execution stabilization ($\Delta D = 10\%$).

The latency decomposition validates the efficiency of the tuned model:

- **Admission of Long-Path Tasks at Low Load:** Under Time Mapping at $\lambda =$ 2 req/s, the calibrated $BS = 2$ configuration exhibits a slightly higher average

response time compared to the baseline (0.76 s vs. 0.58 s). This is a direct consequence of the +12.0% increase in task admissions: borderline requests that were previously discarded due to tight deadlines are now successfully scheduled over multi-hop ISL paths, moderately inflating the average completion latency while boosting overall constellation yield.

- **Controlled Network Residency without Path-Bloat:** Maintaining $\gamma = 10$ keeps the transmission and queuing component ($T_{net+wait}$, top translucent segments) confined within 0.36–0.44 s across all arrival rates, avoiding the severe path-bloat latency inflation (> 1.05 s) observed when $\gamma \geq 1000$.

4.5 Summary of Findings and Architectural Readiness

The empirical evaluation and progressive tuning campaign yield four foundational conclusions regarding centralized ILP orchestration in LEO satellite edge networks:

1. **Ingress Queuing Dwell Time is the Dominant Latency Bottleneck:** Pure onboard processing duration on the Satellite Edge Nodes remains tightly bounded and stable ($T_{exec} \approx 0.22$–0.32 s under calibrated configurations). System throughput collapse under elevated arrival rates is driven primarily by buffer residence times during batch aggregation in the ingress queues (T_{wait}), which systematically consume application deadline slack before optimization and forwarding can take place.

2. **Necessity of Fast-Dispatch Mechanisms:** To maintain temporal feasibility in dynamic space environments, centralized orchestrators must enforce low-latency dispatch policies. Operating with aggressive timeouts ($BT \leq 0.05$ s) or minimal batch aggregation boundaries ($BS \leq 2$) is mandatory to suppress buffer dwell times and prevent cascade *Deadline Exceeded* rejections under heavy arrival regimes.

3. **Objective-Dependent Impact of Weight Calibration:** Enforcing an aggressive primary weight ($\alpha = 1000$) provides asymmetric benefits: under *Time Mapping*, it compels the branch-and-cut solver to prioritize transmission timeliness and deadline preservation, delivering up to +19.5% higher task completion rates under heavy traffic. Conversely, calibrating the orbital sunset penalty to $\gamma = 10$ eliminates eclipse-induced task drops while strictly preventing the 55% path-bloat latency inflation (> 1.05 s) observed under excessive penalties ($\gamma \geq 1000$).

4. **Architectural Readiness for Multi-Algorithm Benchmarking:** The two fully calibrated operational archetypes—reactive fast-dispatch ($BS = 20, BT = 0.05$ s) and controlled small-batching ($BS = 2, BT = 2.0$ s), configured with $\alpha = 1000$ and $\gamma = 10$—define the empirically optimized operating envelope of the centralized ILP approach.

These fully tuned centralized baselines establish the reference operating points for Chapter 5, where their performance is systematically benchmarked against

state-of-the-art decentralized heuristics (*OrbitAware*, *DTS-base*, *DTS-Optimal*) and distributed hierarchical ILP formulations across heterogeneous workload profiles.

Chapter 5

Comparative Benchmark

Having formalized the mathematical optimization framework in Chapter 3 and established the calibrated operational parameters in Chapter 4, this chapter presents an extensive comparative performance evaluation of the proposed Centralized Integer Linear Programming (ILP) Orchestrator against state-of-the-art distributed heuristics and hierarchical optimization baselines [6].

Deploying orbital edge intelligence across Low Earth Orbit (LEO) constellations involves balancing competing architectural paradigms [4, 5, 1]. On the one hand, decentralized heuristics provide rapid, low-complexity decisions with negligible queuing overhead [6]. On the other hand, optimization models search the joint combinatorial space of compute allocation and multi-hop Inter-Satellite Link (ISL) routing to achieve theoretically optimal resource utilization [11, 9].

To quantify the operational trade-offs between these paradigms, this benchmark evaluates 9 algorithm configurations across a comprehensive testing matrix spanning diverse workload profiles, real-time payload dimensions, initial battery energy reserves, and request arrival rates.

5.1 Experimental Setup and Baseline Algorithms

The comparative evaluation is conducted within the event-driven discrete-event simulation framework developed in Python with `SimPy` [13], backed by scientific computing routines in `NumPy` and `SciPy` [24, 25]. The simulation faithfully reproduces satellite orbital kinematics using SGP4 ephemeris propagation [22], dynamic line-of-sight visibility windows, optical ISL transceivers [7, 8], processor queues, and onboard battery charge-discharge cycles.

5.1.1 Benchmark Dimensioning and Scenario Parameters

The orbital network is configured as a regional Walker-Delta LEO constellation [16] operating at an orbital altitude of 600 km. Adjacent satellites are interconnected via full-duplex optical ISLs [7, 8] characterized by nominal transmission bandwidths $B_{u,v}$ and dynamically computed propagation latencies. Computational requests generated by Ground Users (GUs) are ingested through regional Access Points (AP_{global}) and forwarded to candidate Satellite Edge Nodes (SENs) [2, 6].

The benchmark explores a multi-dimensional parameter space:

- **Workload Distribution Vector $\boldsymbol{\beta} = (\beta_G, \beta_C, \beta_{CD})$:** Represents the proportion of Generic tasks, CPU-intensive tasks, and CPU & Data-intensive tasks. We systematically evaluate three distinct operational regimes:
  1. *Mixed Multi-Service Profile:* $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$, representing a balanced heterogeneous mission.
  2. *Computation-Heavy Profile:* $\boldsymbol{\beta} = (0, 1, 0)$, isolating processor clock cycles stress with minimal network footprint.
  3. *Communication-Heavy Profile (Data-Intensive):* $\boldsymbol{\beta} = (0, 0, 1)$, exerting concurrent stress on onboard CPU cores, ISL bandwidth, and transceiver transmission energy.
- **Real-Time Payload Vector $\boldsymbol{\alpha} = (\alpha_M, \alpha_L, \alpha_V)$:** Governs the distribution of input/output data payload sizes across Medium (1–5 MB), Large (5–20 MB), and Very Large (20–50 MB) files. Four sub-profiles are benchmarked for every scenario to evaluate the impact of link saturation:
  1. *Standard (Std):* $\boldsymbol{\alpha} = (0.30, 0.50, 0.20)$.
  2. *Large-heavy (L-heavy):* $\boldsymbol{\alpha} = (0, 0.70, 0.30)$.
  3. *Very-Large-heavy (VL-heavy):* $\boldsymbol{\alpha} = (0, 0.30, 0.70)$.
  4. *Pure Large (Pure L):* $\boldsymbol{\alpha} = (0, 1.0, 0)$.
- **Fleet Operational Energy Budget (E_i):** Candidate SENs are initialized with usable battery energy reserves of either 40 kJ (stringent energy constraint) or 80 kJ (relaxed energy constraint) per orbital pass.
- **Traffic Intensity Spectrum (λ):** Request arrival rates vary systematically across $\lambda \in \{2, 3, 4, 6, 8, 10\}$ req/s, transitioning the network from a low-load regime to a heavy-load saturation regime.

5.1.2 Evaluated Algorithms and State-of-the-Art Baselines

The comparative suite comprises 9 algorithmic configurations categorized into three architectural paradigms:

1. **Distributed Heuristics:**
   - `DTS-base:` A greedy distributed baseline that offloads tasks to the geographically closest available SEN possessing sufficient idle CPU capacity [6].
   - `DTS-Optimal:` An enhanced multi-criteria greedy heuristic jointly evaluating candidate processing queues and initial access-point link bandwidth [6].
   - `OrbitAware:` A trajectory-predictive distributed heuristic leveraging orbital ephemeris [22] to compute shortest paths via Dijkstra's algorithm [23] and balance computational load across candidate orbital planes [6].

2. **Distributed / Hierarchical Optimization Models:**
   - `ILP-Hierarchical (Time):` A two-stage mathematical optimization formulation solved locally within satellite clusters, prioritizing task response time [6].
   - `ILP-Hierarchical (Energy):` A distributed formulation prioritizing constellation energy preservation [6].
3. **Proposed Centralized ILP Orchestrator:** Formulated under Integer Linear Programming principles [11] using `PuLP` [14] and solved via exact dual simplex solvers [20]. Following the micro-tuning calibration established in Chapter 4 (which identified the calibrated objective function weighting parameters as $\alpha^* = 1000$ and $\gamma^* = 10$, balancing primary task admission against secondary operational overhead), the orchestrator is evaluated under optimal weights across two viable batching archetypes:
   - `ILP-Centr (BS20, BT0.05, Time/Energy):` Reactive fast-dispatch configuration with an aggressive 50 ms timeout.
   - `ILP-Centr (BS2, BT2, Time/Energy):` Controlled small-batching policy ($BS = 2$, $BT = 2$ s) enabling pairwise combinatorial optimization.

5.2 Comparative Performance Evaluation across Workload Profiles

We evaluate the algorithms across three primary performance metrics:

1. **Task Success Rate (%):** The fraction of total offloaded requests successfully routed, executed, and delivered within the deadline D_r without violating battery limits.
2. **End-to-End Response Time (s):** Decomposed into *Onboard Task Execution Time* (T_{exec}, capturing physical processing of the task on the destination SEN CPU cores) and *Network/Wait Time* ($T_{net+wait}$, capturing batch queuing at the orchestrator, multi-hop ISL propagation, and payload transmission).
3. **Rejection Causes Breakdown (%):** The diagnostic distribution of task drops categorized into *Deadline Exceeded*, *Insufficient CPU+NET*, *Insufficient CPU*, *No Server Found*, *ILP Infeasible*, and *Sunset*.

5.2.1 Mixed Multi-Service Workload Profile ($\boldsymbol{\beta} = (0.40, 0.35, 0.25)$)

The mixed workload profile represents a standard multi-tenant operational scenario containing a heterogeneous combination of generic, CPU-heavy, and data-heavy requests.

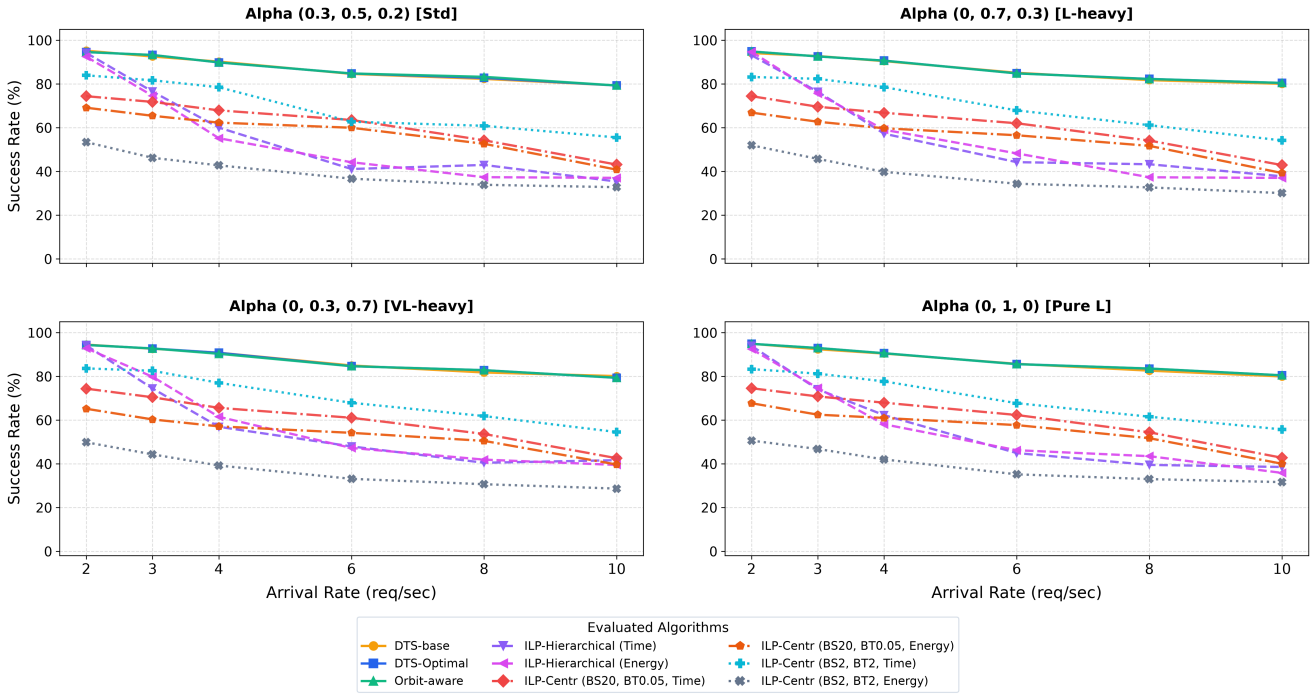

Figure 5.1. Success Rate across arrival rates under Mixed Workload $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$ for 40 kJ battery budget across payload distributions $\boldsymbol{\alpha}$.

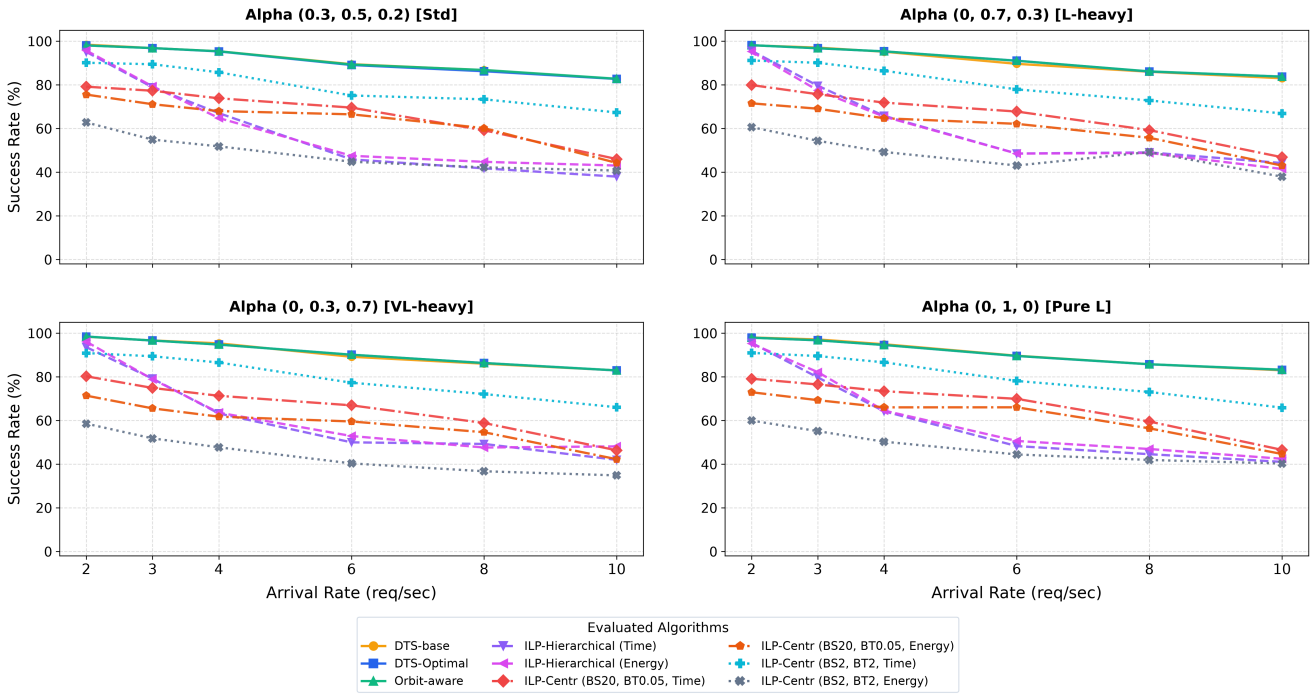

Figure 5.2. Success Rate across arrival rates under Mixed Workload $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$ for 80 kJ battery budget across payload distributions $\boldsymbol{\alpha}$.

5.2.1.1 Success Rate Dynamics

Figure 5.1 and Figure 5.2 illustrate the task completion rate across the arrival rate spectrum $\lambda \in [2, 10]$ req/s under 40 kJ and 80 kJ battery budgets, evaluated across the four payload profiles $\boldsymbol{\alpha}$.

Under light traffic ($\lambda = 2$ req/s), distributed heuristics (`DTS-base`, `DTS-Optimal`, and `OrbitAware`) achieve near-optimal throughput, delivering completion rates of 94–95% at 40 kJ and climbing to 98–99% when the battery budget is expanded to 80 kJ. As traffic escalates to the saturation threshold ($\lambda = 10$ req/s), these heuristics sustain completion rates between 79% and 83%. Their localized greedy decision-making bypasses queuing bottlenecks entirely, allowing task admissions to scale directly with instantaneous physical resource availability.

In contrast, mathematical optimization formulations exhibit substantial performance divergence governed by batching parameters and objective functions:

- **Small-Batching Resilience under Time Mapping:** The centralized small-batch configuration with latency optimization (`ILP-Centr [BS2, BT2, Time]`) consistently achieves the highest throughput among all optimization-based approaches across all arrival rates. It starts at 82–84% (40 kJ) and 90–92% (80 kJ) at $\lambda = 2$ req/s, and maintains 54–56% (40 kJ) and 65–67% (80 kJ) even at $\lambda = 10$ req/s. In contrast, its energy-optimizing counterpart (`ILP-Centr [BS2, BT2, Energy]`) suffers severe degradation, achieving 50–53% (40 kJ) / 59–63% (80 kJ) at $\lambda = 2$ req/s and declining to 29–33% (40 kJ) / 35–38% (80 kJ) at $\lambda = 10$ req/s. Prioritizing energy minimization penalizes forwarding over multi-hop topologies, rejecting tasks that would otherwise be schedulable under time minimization.

- **Fast-Dispatch Performance:** The centralized fast-dispatch configurations (`BS20, BT0.05`) provide competitive throughput at low load, with Time mapping achieving 74% (40 kJ) / 79–80% (80 kJ) and Energy mapping reaching 67–69% (40 kJ) / 72–75% (80 kJ). However, under peak load ($\lambda = 10$ req/s), they degrade to 42–43% (40 kJ) / 46–47% (80 kJ) for Time, and 39–41% (40 kJ) / 42–45% (80 kJ) for Energy. While bounding the timeout to $BT = 0.05$ s prevents ingress queue accumulation, it falls short of `ILP-Centr [BS2, BT2, Time]` by 12–20% at high load, because pairwise combinatorial optimization resolves link contention far more effectively than fast dispatch.

- **Hierarchical Throughput Collapse:** The hierarchical formulations (`ILP-Hierarchical` for both Time and Energy) exhibit an acute inflection point: while achieving 93–95% (40 kJ) and 97–99% (80 kJ) at $\lambda = 2$ req/s, their completion rates drop precipitously once traffic exceeds $\lambda = 3$ req/s, bottoming out at 36–43% (40 kJ) and 39–45% (80 kJ) at $\lambda = 10$ req/s. The two-tier coordination overhead and inter-cluster aggregation delays introduce structural latency that severely compromises task feasibility during traffic surges.

Across payload distributions, shifting from *Standard* $\boldsymbol{\alpha} = (0.3, 0.5, 0.2)$ to *VL-heavy* $\boldsymbol{\alpha} = (0, 0.3, 0.7)$ or *Pure L* $\boldsymbol{\alpha} = (0, 1, 0)$ systematically depresses completion rates by 2–5% across all algorithms due to increased serialization delay over optical ISLs. Doubling the usable battery capacity from 40 kJ to 80 kJ provides a uniform

upward shift of +4% to +12%, with the most pronounced benefit observed in multi-hop optimization configurations.

5.2.1.2 Response Time Analysis

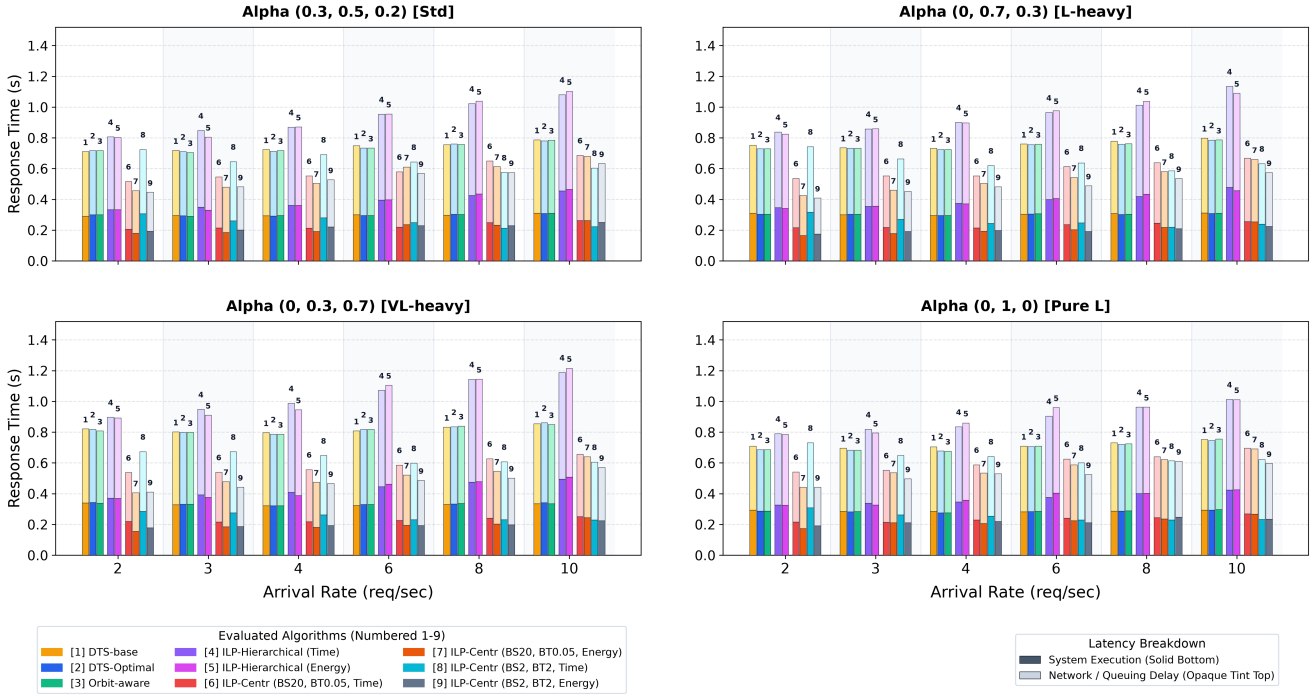

Figure 5.3. Response Time decomposition under Mixed Workload $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$ comparing Onboard Task Execution and Network/Wait latency for 40 kJ.

Figure 5.3 and Figure 5.4 present the end-to-end response time decomposed into physical Onboard Task Execution Time (T_{exec}, solid lower bar) and Network/Queuing Delay ($T_{net+wait}$, opaque upper bar), measured in seconds (s).

The empirical results demonstrate a clear architectural bifurcation in latency dynamics:

- **Heuristic Invariance:** The distributed heuristics maintain remarkably stable response times across all traffic rates, bounded tightly between 0.70 s and 0.80 s under *Std* and reaching at most 0.85–0.86 s under *VL-heavy*. Crucially, T_{exec} remains invariant at 0.30–0.34 s, while $T_{net+wait}$ accounts for 0.40–0.52 s. Because distributed heuristics execute immediate local forwarding without ingress buffering queues, their network delay reflects pure physical ISL propagation and transmission time.
- **Hierarchical Latency Inflation:** `ILP-Hierarchical` models suffer substantial latency expansion under load. Mean response times rise from 0.80–0.85 s (reaching 0.93 s under *VL-heavy*) at $\lambda = 2$ req/s to 1.10–1.28 s at $\lambda = 10$ req/s (peaking at 1.28 s under 80 kJ *VL-heavy*). The decomposition reveals that while onboard task processing time on the destination SEN (T_{exec}) increases

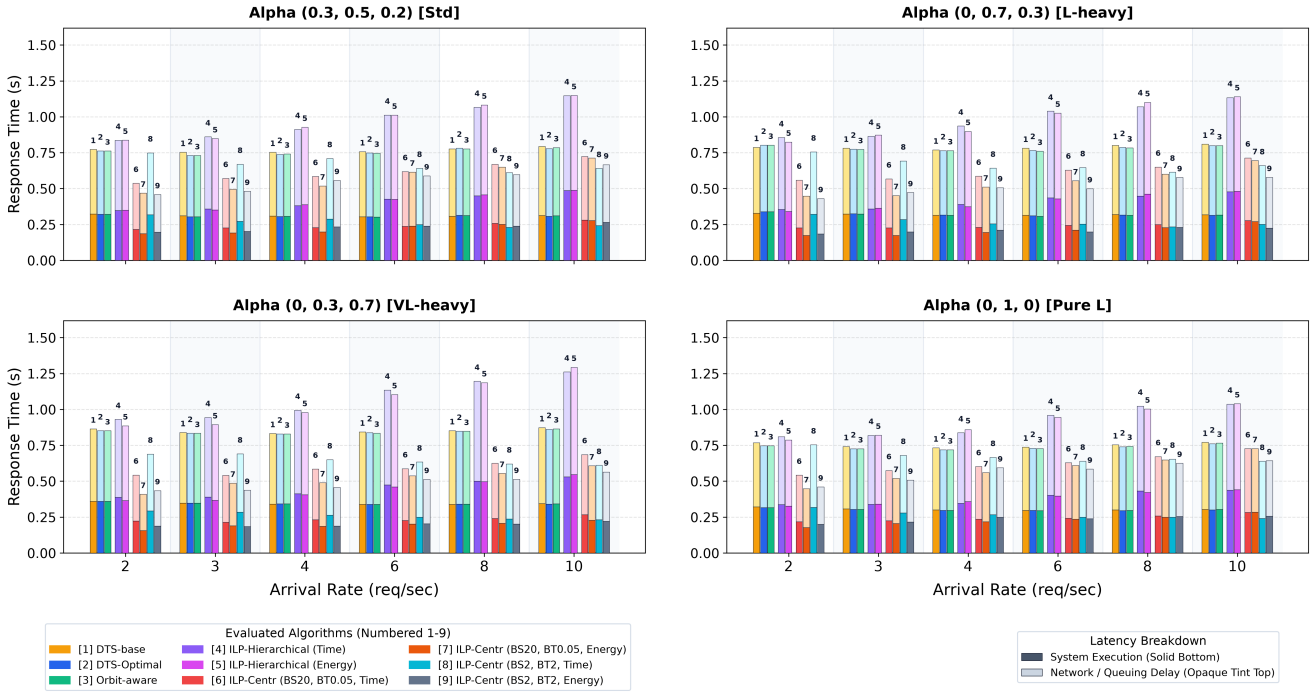

Figure 5.4. Response Time decomposition under Mixed Workload $\beta = (0.40, 0.35, 0.25)$ comparing Onboard Task Execution and Network/Wait latency for 80 kJ.

from 0.33 s to 0.48–0.52 s under concurrent core utilization, the primary growth driver is $T_{net+wait}$ (expanding to 0.70–0.76 s), reflecting acute queuing backlog during hierarchical task aggregation.

- **Low-Latency Bounds of Fast Dispatch:** The centralized fast-dispatch models (`BS20, BT0.05`) achieve the lowest baseline response times in the entire benchmark, recording 0.50–0.54 s for Time (bar [6]) and 0.42–0.48 s for Energy (bar [7]) at $\lambda = 2$ req/s. Under peak load ($\lambda = 10$ req/s), response time expands moderately to 0.65–0.75 s (Time) and 0.60–0.65 s (Energy). Their onboard execution footprint is exceptionally low ($T_{exec} \leq 0.26$ s), while capping the batch timeout at 0.05 s strictly suppresses ingress queuing accumulation.

- **Small-Batch Dynamics:** For `ILP-Centr [BS2, BT2, Time]` (bar [8]), response time starts slightly higher at $\lambda = 2$ req/s (0.72–0.76 s) due to queuing dwell times awaiting batch capacity or timeout trigger ($T_{net+wait} \approx 0.50$–$0.53$ s, $T_{exec} \approx 0.21$–0.23 s). However, as traffic scales to $\lambda = 10$ req/s, response time decreases and stabilizes at 0.56–0.65 s across all payload profiles, because the small batch threshold of 2 is satisfied almost instantaneously by incoming traffic bursts. Meanwhile, `ILP-Centr [BS2, BT2, Energy]` (bar [9]) exhibits low latency (0.45–0.63 s, with $T_{exec} \approx 0.17$–0.23 s) because it aggressively drops tasks requiring multi-hop routing, admitting predominantly single-hop assignments.

5.2.1.3 Failure Diagnostics

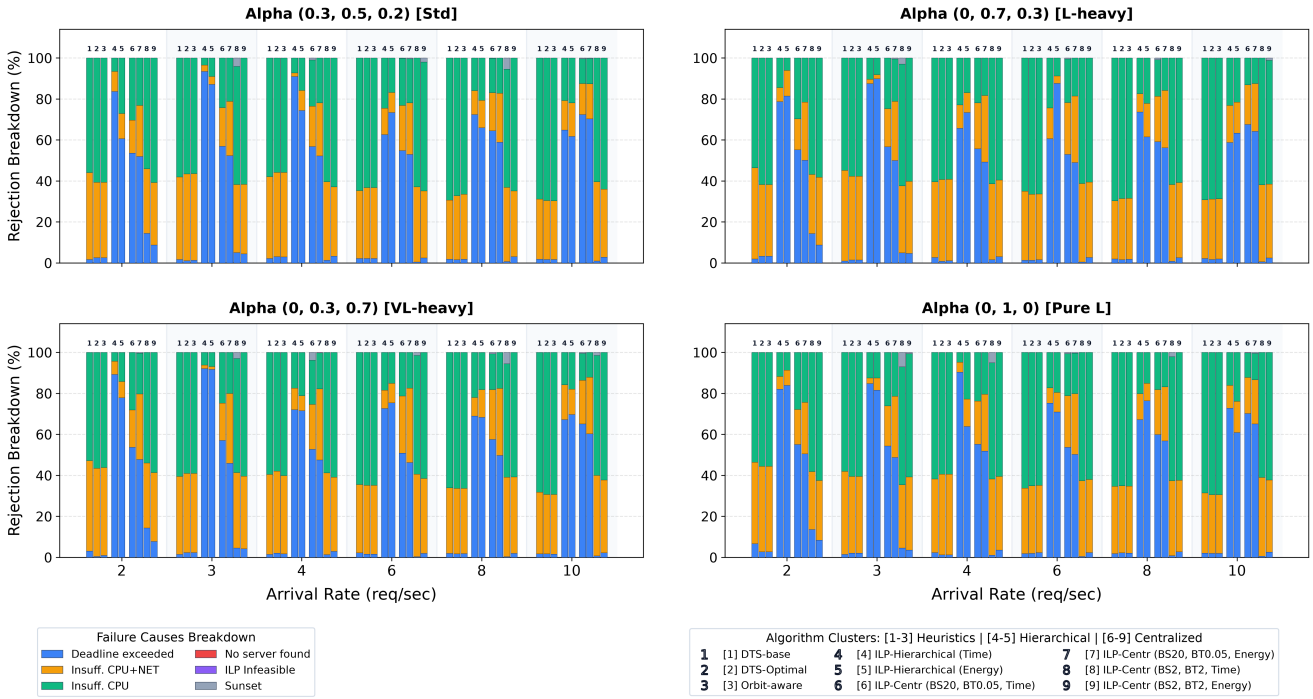

Figure 5.5. Rejection Causes distribution under Mixed Workload $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$ across arrival rates for 40 kJ.

Figure 5.5 and Figure 5.6 illustrate the diagnostic distribution of task rejections across arrival rates, battery budgets, and payload distributions:

- **Heuristic Hardware Bottlenecks:** For distributed heuristics (`DTS-base`, `DTS-Optimal`, `OrbitAware`, bars [1–3]), task drops are overwhelmingly governed by physical hardware saturation. Rejections are split between *Insufficient CPU* (62–70%, green) and *Insufficient CPU+NET* (30–38%, orange) under heavy traffic ($\lambda \geq 6$ req/s). Because these heuristics make instantaneous routing decisions without accumulation buffers, drops occur primarily when candidate SEN CPU cores or egress transceivers are saturated; *Deadline Exceeded* (blue) drops account for $< 2\%$ at high load, although reaching 7–12% at $\lambda = 2$ req/s under 80 kJ where battery availability enables deeper path exploration.

- **Fast-Dispatch Profile (`BS20, BT0.05`):** Centralized fast-dispatch configurations (bars [6] and [7]) exhibit substantial deadline degradation under traffic saturation despite the aggressive 50 ms timeout. At light load ($\lambda = 2$ req/s), *Deadline Exceeded* accounts for 45–52% of drops under Time mapping (bar [6]) and 40–48% under Energy mapping (bar [7]). Under peak traffic ($\lambda = 10$ req/s), queuing backlogs expand deadline failures to 65–75% at 40 kJ and up to 72–80% at 80 kJ, with physical exhaustion accounting for the remainder (*Insufficient CPU+NET* at 12–18% and *Insufficient CPU* at 10–15%).

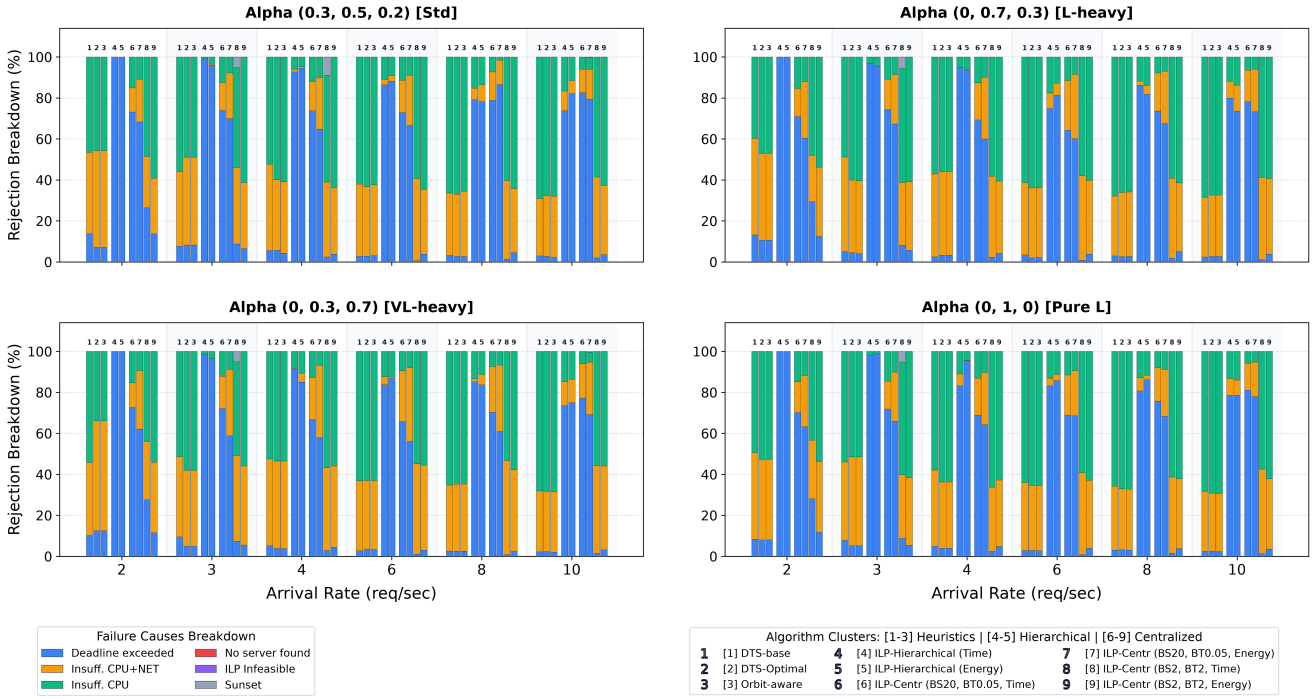

Figure 5.6. Rejection Causes distribution under Mixed Workload $\boldsymbol{\beta} = (0.40, 0.35, 0.25)$ across arrival rates for 80 kJ.

- **Small-Batching Dynamics (`BS2`, `BT2`):** Small-batch configurations demonstrate that restricting batch capacity to $BS = 2$ drastically suppresses queuing timeouts under sustained traffic. Under Time mapping (bar [8]), *Deadline Exceeded* accounts for 10–15% of drops at $\lambda = 2$ req/s (due to dwell times awaiting batch completion, alongside a minor 2–4% share of *Sunset* drops), but diminishes to $< 2\%$ at $\lambda = 10$ req/s as arrival bursts saturate the small threshold immediately. Consequently, rejections are overwhelmingly governed by physical resource exhaustion, split between *Insufficient CPU* (58–62%) and *Insufficient CPU+NET* (36–40%). Under Energy mapping (bar [9]), deadline violations remain virtually absent across all arrival rates ($\leq 2\%$), with failures driven strictly by physical limitations (*Insufficient CPU* at 58–62% and *Insufficient CPU+NET* at 36–40%) as the energy objective rejects multi-hop routes exceeding battery budgets.

5.2.2 Computation-Heavy Stress: CPU-Intensive Profile ($\boldsymbol{\beta} = (0, 1, 0)$)

When the workload consists entirely of compute-intensive requests with minimal payload transfer ($\boldsymbol{\beta} = (0, 1, 0)$), onboard processor clock cycles become the primary system bottleneck.

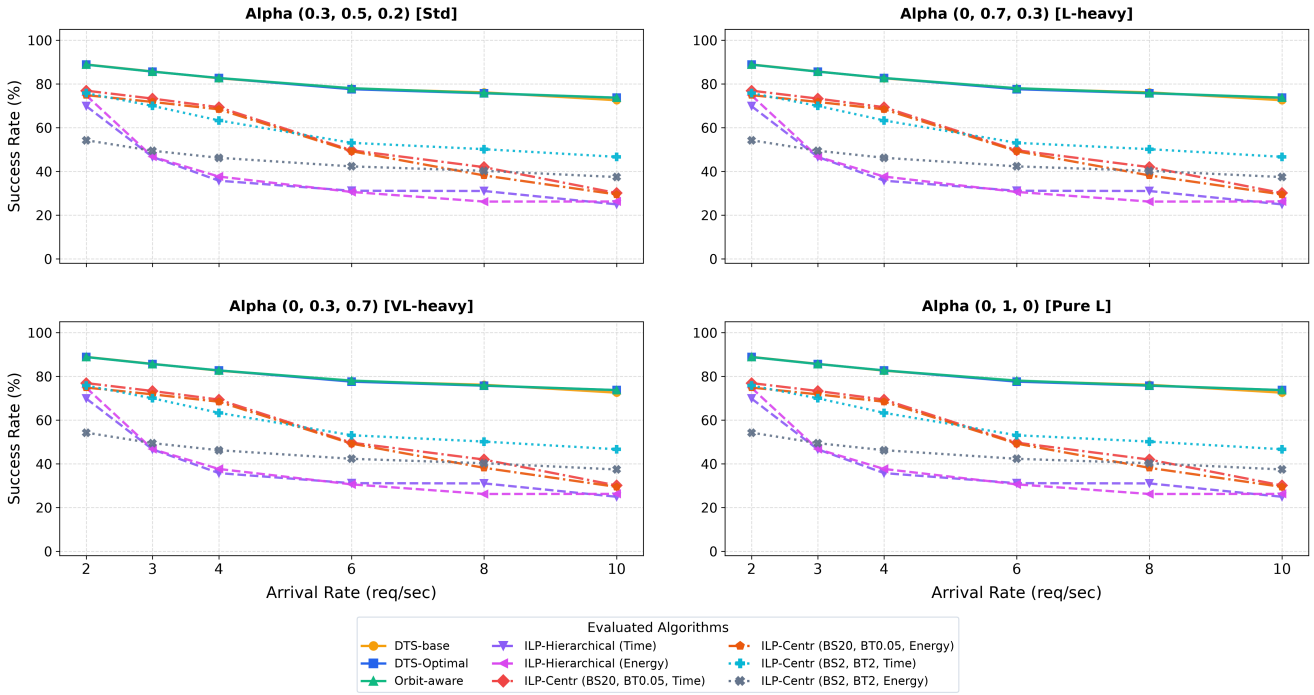

Figure 5.7. Success Rate across arrival rates under CPU-Intensive Workload $\boldsymbol{\beta} = (0, 1, 0)$ for 40 kJ across payload distributions $\boldsymbol{\alpha}$.

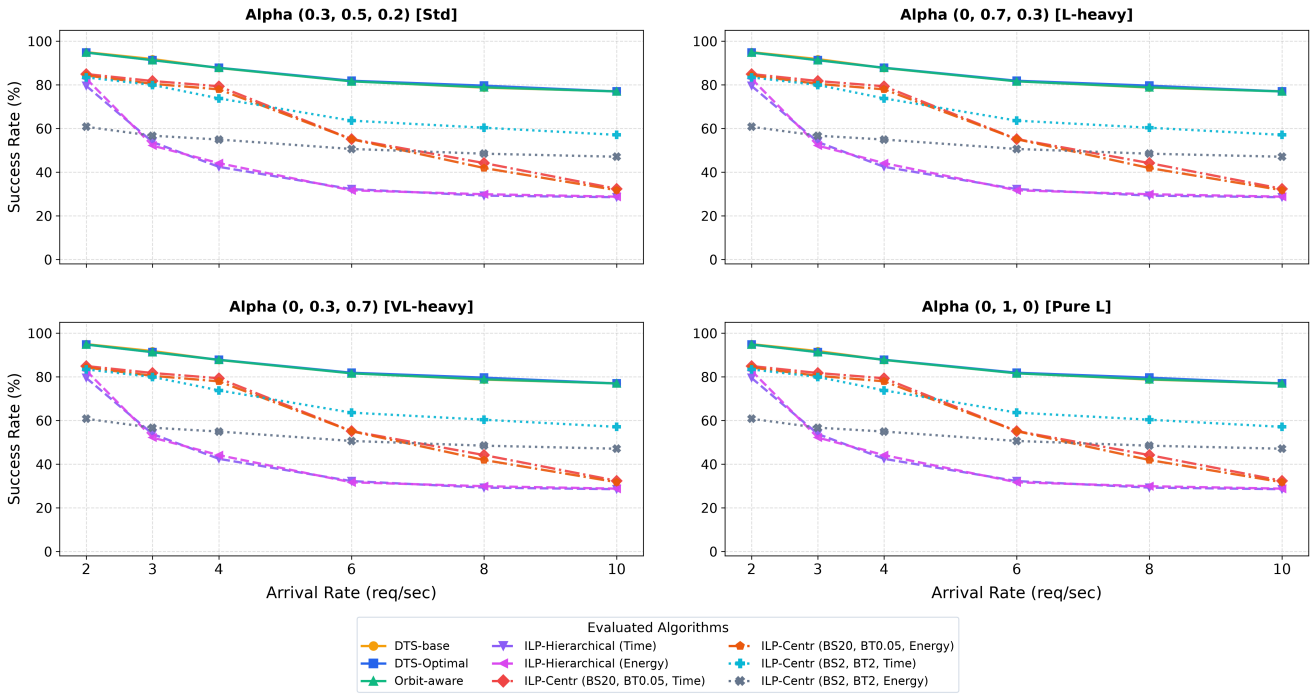

Figure 5.8. Success Rate across arrival rates under CPU-Intensive Workload $\boldsymbol{\beta} = (0, 1, 0)$ for 80 kJ across payload distributions $\boldsymbol{\alpha}$.

5.2.2.1 Success Rate Dynamics and Invariance across Payload Distributions

Figure 5.7 and Figure 5.8 illustrate the task completion rate across the arrival rate spectrum $\lambda \in [2, 10]$ req/s under 40 kJ and 80 kJ battery budgets across the four payload profiles $\boldsymbol{\alpha}$ for the purely computational profile $\boldsymbol{\beta} = (0, 1, 0)$.

The most prominent feature is the complete **invariance across payload vector distributions $\boldsymbol{\alpha}$**. The curves across the four subplots (Std, L-heavy, VL-heavy, and Pure L) are identical because compute tasks carry negligible data volume. In contrast to payload invariance, expanding the fleet battery reserves from 40 kJ to 80 kJ provides a noticeable throughput gain across all schemes (+4% to +11%), driven by the energy consumed by onboard CPU core executions during extended simulation intervals.

Under these operating conditions, a marked performance differentiation is observed among algorithmic paradigms:

- **Resilience of Distributed Heuristics:** The heuristic algorithms (`DTS-base`, `DTS-Optimal`, and `OrbitAware`) demonstrate remarkable robustness, recording completion rates of 88–89% at light load ($\lambda = 2$ req/s) under 40 kJ (climbing to 94–95% at 80 kJ) and degrading gradually to 72–74% at 40 kJ (76–77% at 80 kJ) at peak load ($\lambda = 10$ req/s). Their greedy evaluation without upstream queuing allows them to saturate physical SEN core capacity directly.

- **Collapse of Hierarchical Formulations:** The `ILP-Hierarchical` models suffer an immediate breakdown as soon as traffic exceeds $\lambda = 2$ req/s. The success rate drops precipitously from 70–76% (40 kJ) / 80–84% (80 kJ) at $\lambda = 2$ req/s down to 46–49% at $\lambda = 3$ req/s, before stabilizing at 25–26% under 40 kJ and 28–30% under 80 kJ at $\lambda = 10$ req/s. Multi-tier aggregation delays render the hierarchical paradigm inadequate for dense computational workloads.

- **Superiority of Small-Batching over Fast-Dispatch:** The small-batching configuration with latency optimization (`ILP-Centr [BS2, BT2, Time]`) demonstrates superior resilience under computational stress, sustaining a completion rate of 46–47% at 40 kJ and 56–58% at 80 kJ at $\lambda = 10$ req/s. Centralized fast-dispatch (`ILP-Centr [BS20, BT0.05, Time/Energy]`) achieves 75–77% at 40 kJ and 84–85% at 80 kJ under light load, but degrades steeply to 29–30% at 40 kJ and 31–33% at 80 kJ under full load. Small-batching out-performs fast-dispatch by up to +25% at peak load. Furthermore, `ILP-Centr [BS2, BT2, Energy]` starts at 54% (40 kJ) / 60–61% (80 kJ) at $\lambda = 2$ req/s and sustains 37–38% (40 kJ) / 47–48% (80 kJ) at $\lambda = 10$ req/s, also exceeding fast-dispatch throughput under high traffic.

5.2.2.2 Response Time Analysis

Figure 5.9 and Figure 5.10 illustrate the end-to-end response time decomposition between the physical execution time on SEN cores (T_{exec}, solid lower bar) and the queuing/network delay ($T_{net+wait}$, opaque upper bar), measured in seconds (s).

The analysis highlights a pronounced divergence in system latency behavior:

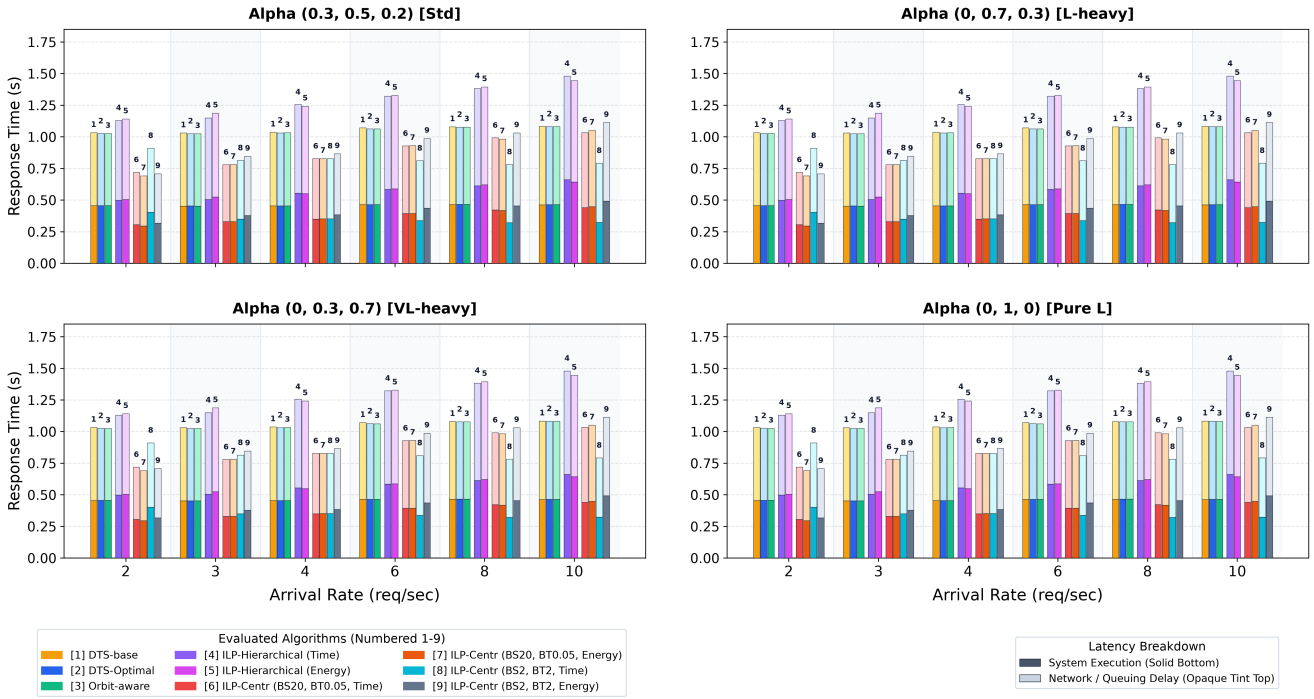

Figure 5.9. Response Time decomposition under CPU-Intensive Workload $\boldsymbol{\beta} = (0, 1, 0)$ for 40 kJ battery budget across payload distributions $\boldsymbol{\alpha}$.

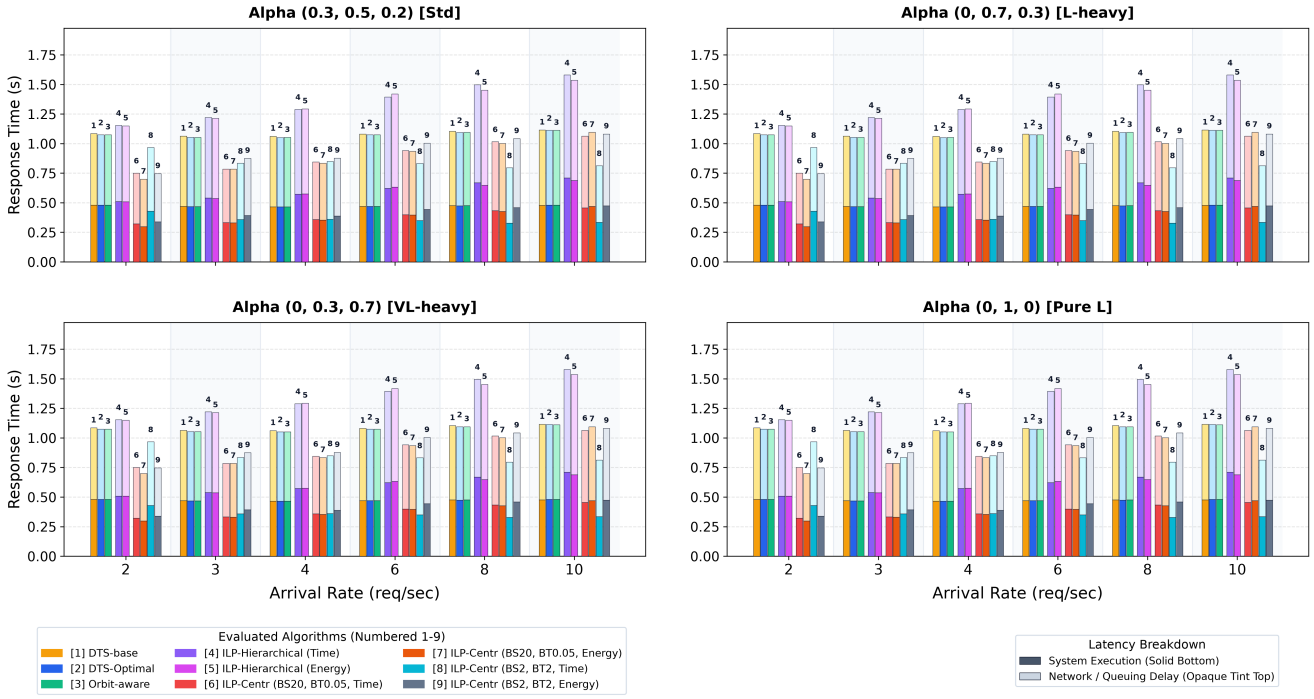

Figure 5.10. Response Time decomposition under CPU-Intensive Workload $\boldsymbol{\beta} = (0, 1, 0)$ for 80 kJ battery budget across payload distributions $\boldsymbol{\alpha}$.

- **Dynamic Invariance of Heuristics:** Distributed heuristics maintain stable response times across the arrival spectrum, bounded between 1.03 s and 1.08 s at 40 kJ (1.07–1.12 s at 80 kJ). The physical execution component T_{exec} remains fixed at ≈ 0.45 s (0.48 s at 80 kJ), while $T_{net+wait}$ accounts for ≈ 0.58–0.64 s. Because forwarding decisions are instantaneous, waiting time coincides solely with physical transit over optical links.

- **Fast-Dispatch Latency Progression:** Fast-dispatch configurations (`BS20, BT0.05`, bars [6] and [7]) exhibit one of the lowest latency under light traffic (0.68–0.75 s), expanding to 0.98–1.05 s (40 kJ) and 1.05–1.10 s (80 kJ) at $\lambda = 10$ req/s, with T_{exec} increasing from ≈ 0.30 s to 0.44–0.48 s.

- **Small-Batch Latency Compression:** In contrast to fast-dispatch, small-batching with Time optimization (`BS2, BT2, Time`, bar [8]) begins at 0.90 s (40 kJ) / 0.96–0.98 s (80 kJ) at $\lambda = 2$ req/s due to the batch completion dwell time ($T_{net+wait} \approx 0.50$–$0.56$ s). However, as traffic scales to $\lambda = 10$ req/s, response time compresses and stabilizes at 0.78–0.80 s (40 kJ) and 0.80–0.83 s (80 kJ), because incoming request bursts fulfill the small batch capacity of 2 almost instantaneously. Meanwhile, `ILP-Centr [BS2, BT2, Energy]` (bar [9]) increases from 0.75–0.78 s at $\lambda = 2$ req/s up to 1.08–1.10 s at $\lambda = 10$ req/s.

5.2.2.3 Failure Diagnostics and Bottleneck Analysis

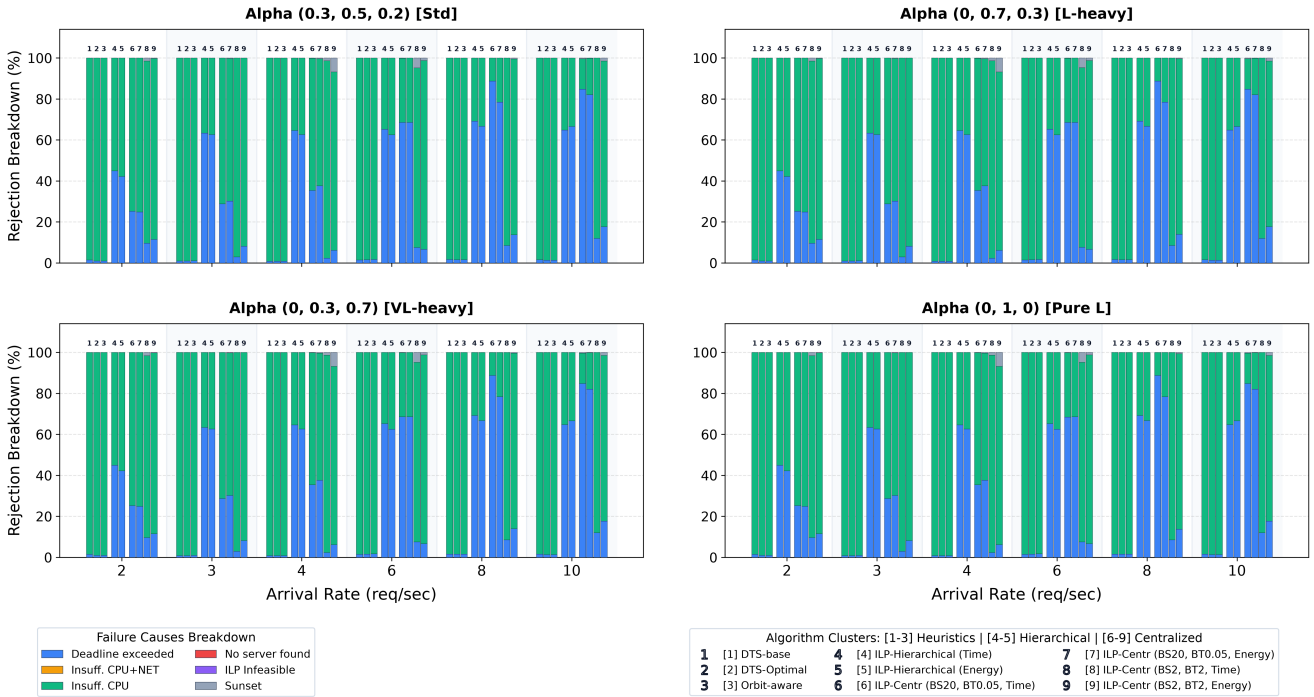

Figure 5.11. Rejection Causes distribution under CPU-Intensive Workload $\boldsymbol{\beta} = (0, 1, 0)$ for 40 kJ across payload distributions $\boldsymbol{\alpha}$.

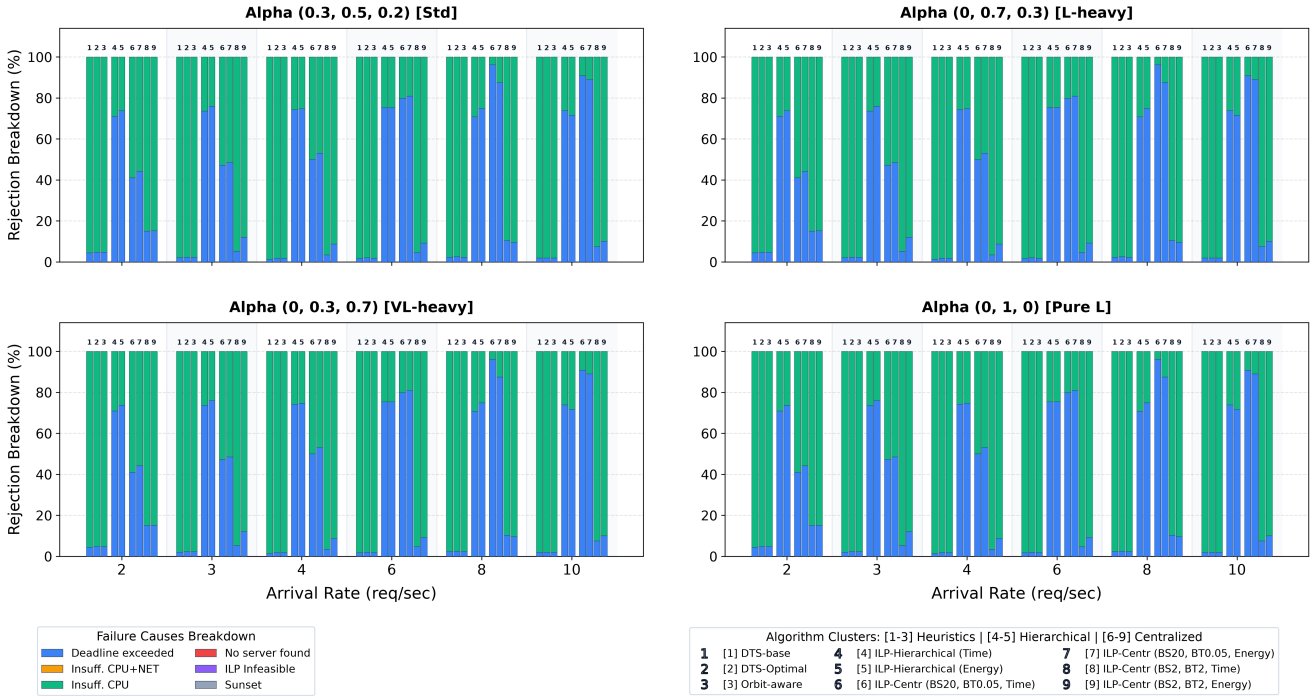

Figure 5.12. Rejection Causes distribution under CPU-Intensive Workload $\boldsymbol{\beta} = (0, 1, 0)$ for 80 kJ across payload distributions $\boldsymbol{\alpha}$.

Figure 5.11 and Figure 5.12 illustrate the diagnostic breakdown of task rejection causes:

- **Distributed Heuristics (`DTS-base`, `DTS-Optimal`, `OrbitAware`):** Exactly 100% **of all rejections** are caused by *Insufficient CPU* (green) across all arrival rates. Rejections for *Deadline Exceeded*, network congestion, or orbital transitions are identically 0%. Because heuristics operate without accumulation buffers, a task is rejected exclusively when candidate node processors have exhausted their available physical cores.
- **Hierarchical Model Bottlenecks:** For `ILP-Hierarchical` models (bars [4] and [5]), *Deadline Exceeded* (blue) represents 40–45% of failures at $\lambda = 2$ req/s and escalates rapidly under heavy load, reaching 82–85% at 40 kJ and up to 90% at 80 kJ at $\lambda = 10$ req/s. The remaining drops are caused by core exhaustion (*Insufficient CPU*, 10–18%).
- **Fast-Dispatch Ingress Queuing Latency (`BS20, BT0.05`, bars [6–7]):** At light load ($\lambda = 2$ req/s), *Deadline Exceeded* (blue) already accounts for 25–30% of failures at 40 kJ and 42–45% at 80 kJ (with *Insufficient CPU* absorbing the rest). As traffic surges to peak load ($\lambda = 10$ req/s), accumulated queuing delay causes deadline drops to expand to 62–66% at 40 kJ and climb to 75–78% at 80 kJ. Physical CPU saturation accounts for the remaining 22–38%.
- **Small-Batching Resilience (`BS2, BT2`):** In sharp contrast, small-batch configurations (bars [8] and [9]) flush pairs of tasks almost instantaneously

under sustained traffic. Consequently, under Time mapping (bar [8]), *Deadline Exceeded* failures remain restricted to just 8–12% at peak load ($\lambda = 10$ req/s), directing 88–92% of rejections to physical core exhaustion (*Insufficient CPU*). Under Energy mapping (bar [9]), deadline drops account for only 10–14%, with 86–90% driven by core exhaustion.

Network saturation (*Insufficient CPU+NET*) remains strictly at 0% across all algorithms, confirming that ISL bandwidth is never contested under compute-only workloads. A negligible fraction (2–5%) of rejections attributed to *Sunset* (grey) appears exclusively under low arrival rates ($\lambda \leq 4$ req/s) in optimization baselines, caused by tasks dwelling in queues across orbital plane visibility transitions.

5.2.3 Communication-Heavy Stress: Data-Intensive Profile ($\boldsymbol{\beta} = (0, 0, 1)$)

In the data-intensive regime ($\boldsymbol{\beta} = (0, 0, 1)$), requests combine the transfer of massive data payloads (20–50 MB governed by $\boldsymbol{\alpha}$) across multi-hop ISL routes, severely testing optical transceivers and energy depletion.

5.2.3.1 Success Rate Dynamics and Optimality Supremacy

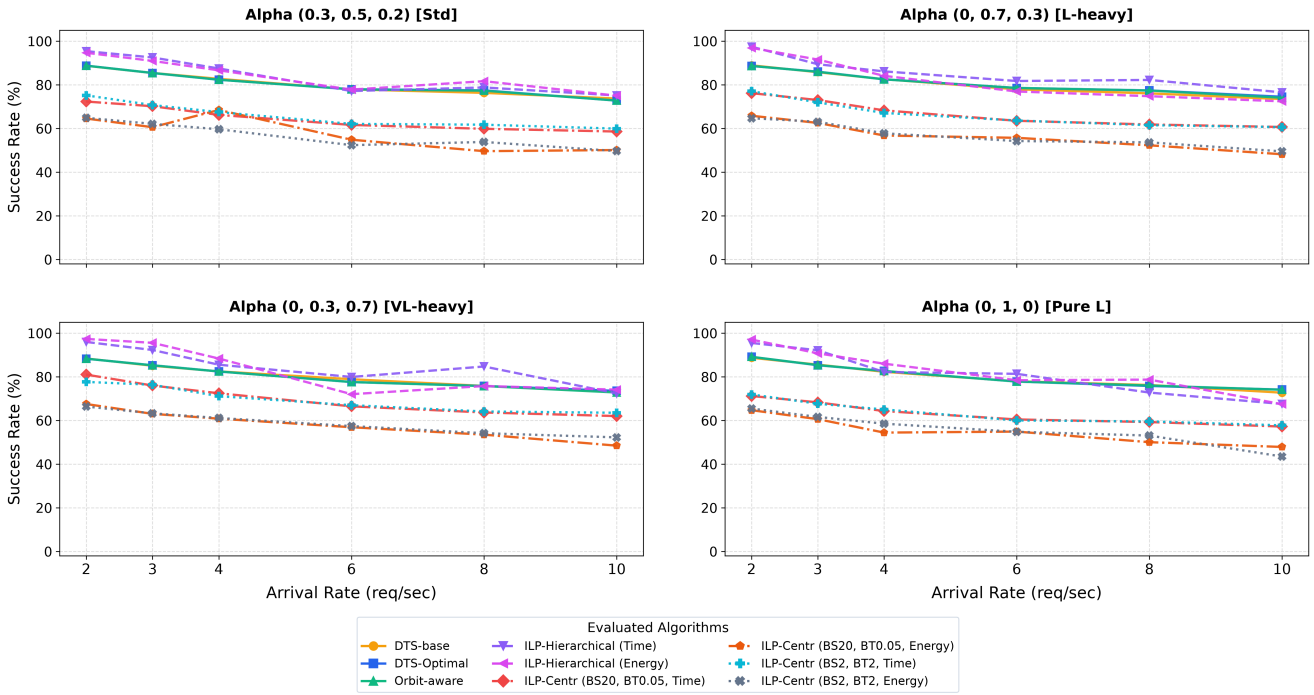

Figure 5.13. Success Rate across arrival rates under Data-Intensive Workload $\boldsymbol{\beta} = (0, 0, 1)$ for 40 kJ battery budget across payload distributions $\boldsymbol{\alpha}$.

Figure 5.13 and Figure 5.14 illustrate the task completion rates across arrival rates $\lambda \in [2, 10]$ req/s under 40 kJ and 80 kJ battery capacities for the communication-heavy profile $\boldsymbol{\beta} = (0, 0, 1)$ across the four payload distributions $\boldsymbol{\alpha}$.

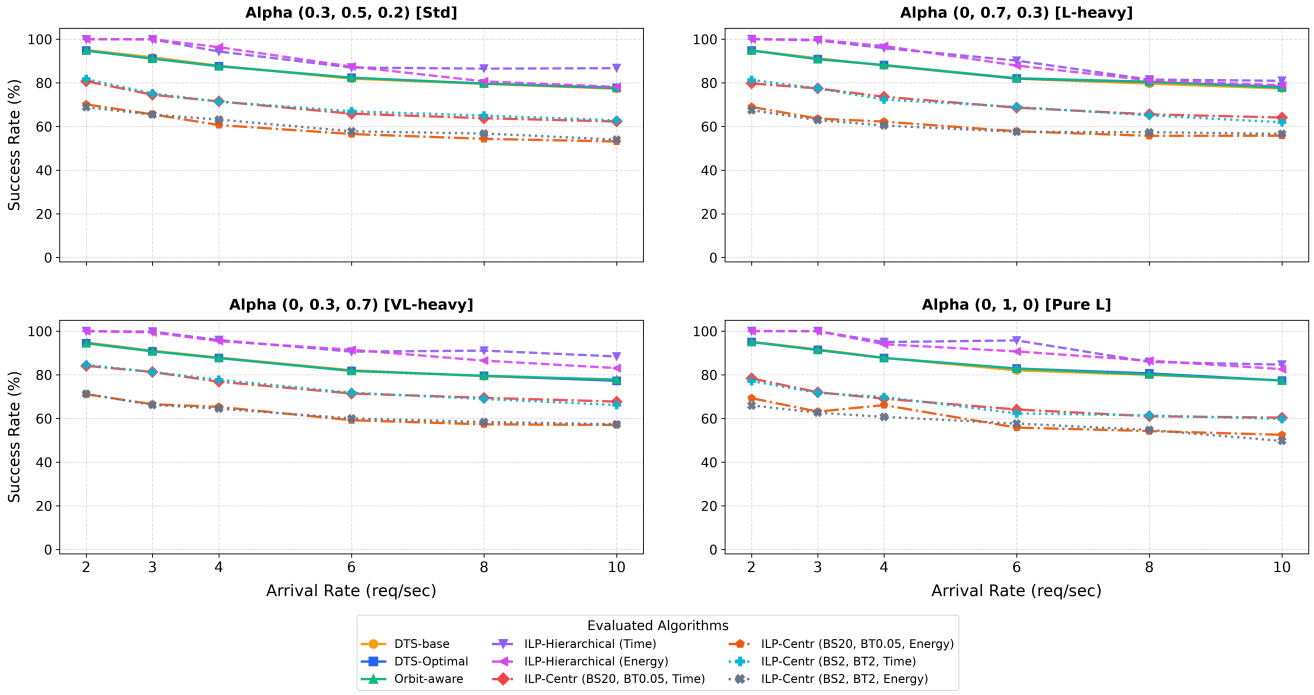

Figure 5.14. Success Rate across arrival rates under Data-Intensive Workload $\boldsymbol{\beta} = (0, 0, 1)$ for 80 kJ battery budget across payload distributions $\boldsymbol{\alpha}$.

The data-intensive regime produces a profound inversion in algorithmic ranking and highlights significant sensitivity to constellation energy reserves:

- **Optimality Supremacy of Hierarchical ILP:** In sharp contrast to the CPU-intensive regime where optimization models collapsed under queuing backlogs, hierarchical optimization decisively outperforms distributed heuristics in communication-heavy scenarios. At low arrival rates ($\lambda = 2$ req/s) under 80 kJ, `ILP-Hierarchical (Time)` and `(Energy)` achieve a near-perfect Success Rate of 99–100% across all $\boldsymbol{\alpha}$ profiles, outperforming distributed heuristics (94–95%). Under maximum arrival stress ($\lambda = 10$ req/s) at 80 kJ, `ILP-Hierarchical (Time)` maintains an outstanding completion rate of 81–89%, sustaining a consistent advantage over distributed heuristics (77–78%).

- **Asymmetric Energy Sizing Sensitivity (40 kJ vs. 80 kJ):** Moving massive payloads (20–50 MB) across multi-hop paths induces substantial transceiver energy drain ($P_{net} \cdot S_r / B_{u,v}$). At 40 kJ, centralized models suffer from early battery exhaustion, reaching 48–63% for Time mapping and 44–52% for Energy mapping at peak load. In contrast, distributed heuristics and `ILP-Hierarchical (Time)` sustain 72–76% at $\lambda = 10$ req/s (with Hierarchical Time reaching a peak of 85% at $\lambda = 8$ req/s in *VL-heavy*) even at 40 kJ. Expanding battery reserves to 80 kJ yields a +4% gain for heuristics, +4–8% for centralized models, and up to +9–15% for hierarchical models under heavy load.

- **Centralized Dispatch Performance:** Centralized latency-optimizing config-

urations deliver strong completion rates under 80 kJ, with fast-dispatch (`BS20, BT0.05, Time`) achieving 79–84% at low load and stabilizing at 59–67% at $\lambda = 10$ req/s. Small-batching with latency optimization (`BS2, BT2, Time`) behaves similarly, sustaining 59–68% at peak load. In contrast, their energy-optimizing counterparts (`Energy` mapping) achieve 65–71% at light load and drop to 49–57% at $\lambda = 10$ req/s due to strict transmission power rejection thresholds.

5.2.3.2 Response Time Analysis

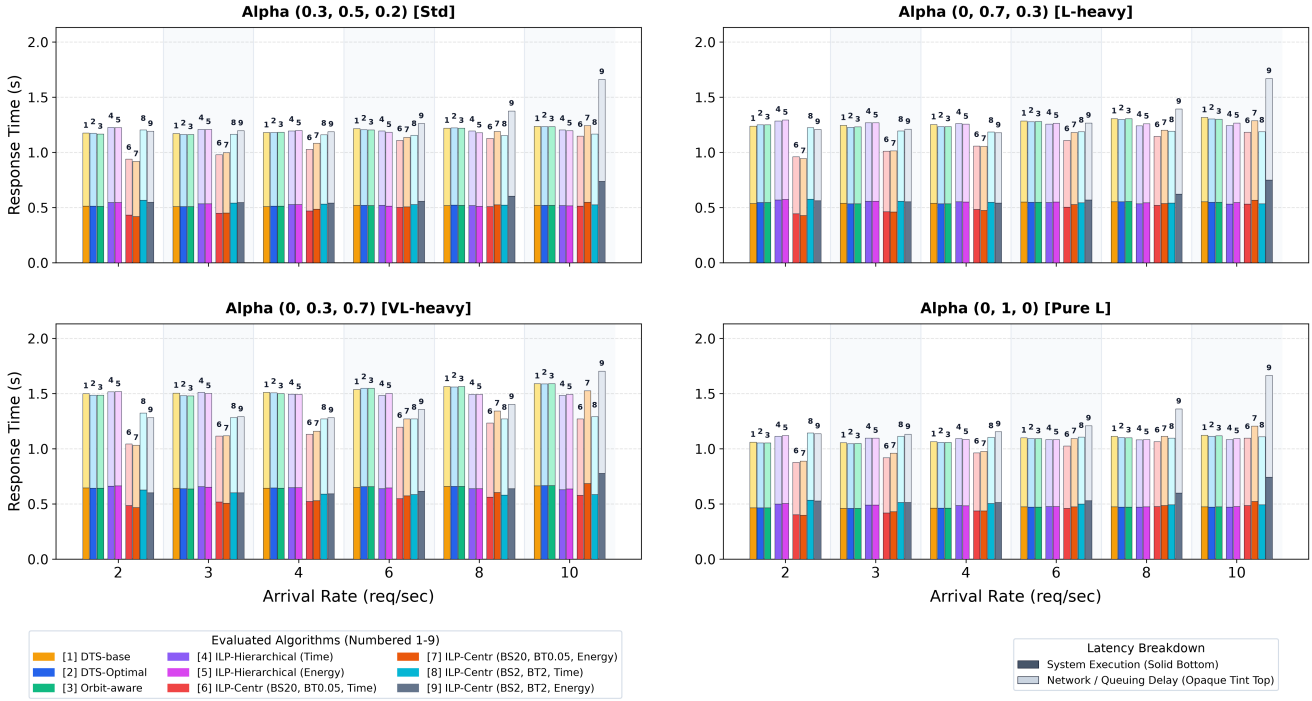

Figure 5.15. Response Time decomposition under Data-Intensive Workload $\boldsymbol{\beta} = (0, 0, 1)$ across arrival rates for 40 kJ.

Figure 5.15 and Figure 5.16 illustrate the end-to-end response time decomposed into physical Onboard Task Execution Time (T_{exec}, solid lower bar) and Network/Wait Time ($T_{net+wait}$, opaque upper bar), measured in seconds (s).

The empirical trends highlight four foundational behaviors:

- **Payload-Driven Latency Scaling:** Response times across all algorithms scale directly with payload size vector $\boldsymbol{\alpha}$. Under *Pure L* (5–20 MB), baseline heuristic response time is bounded around 1.05–1.12 s ($T_{exec} \approx 0.45$–0.48 s, $T_{net+wait} \approx 0.60$–$0.65$ s). When shifting to *VL-heavy* (20–50 MB), serialization delay expands total heuristic latency to 1.50–1.62 s ($T_{exec} \approx 0.65$ s, $T_{net+wait} \approx 0.85$–$0.95$ s). In the balanced *Std* distribution, heuristic latency settles at 1.20–1.28 s.

- **Heuristic Invariance Across Arrival Rates:** Distributed heuristics maintain virtually flat latency curves across the entire spectrum $\lambda \in [2, 10]$ req/s,

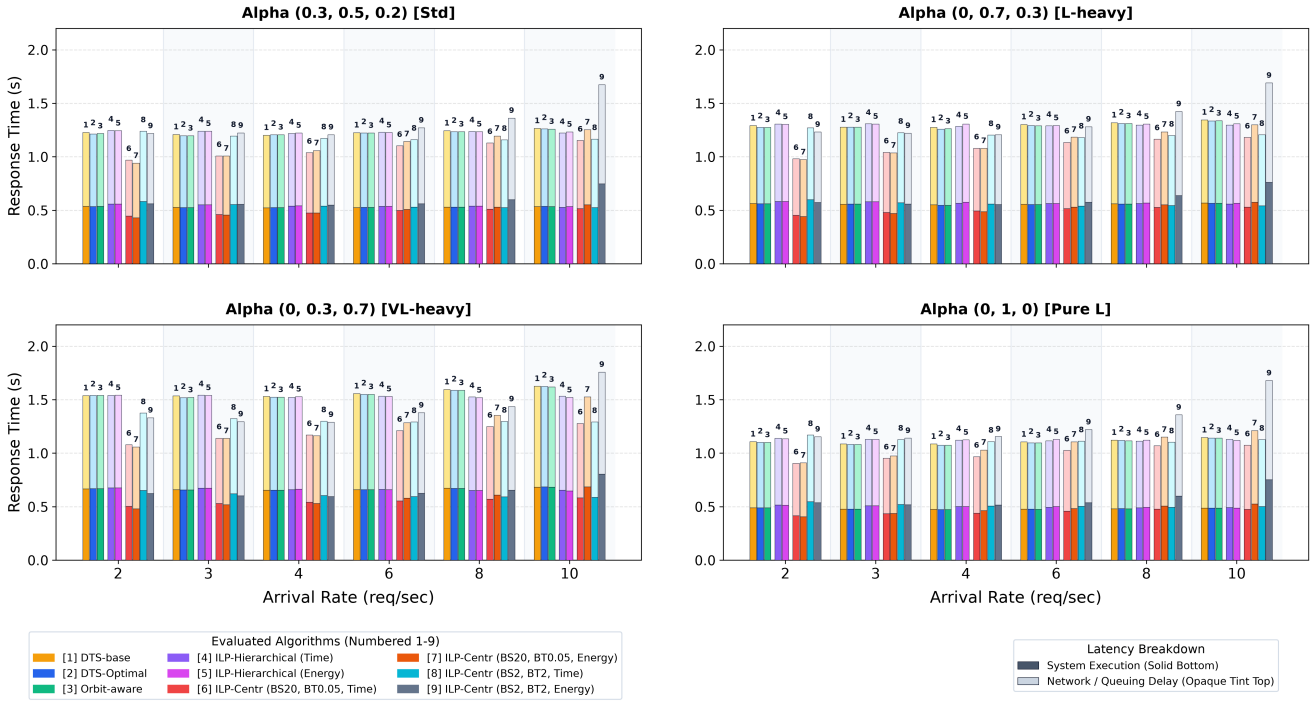

Figure 5.16. Response Time decomposition under Data-Intensive Workload $\beta = (0, 0, 1)$ across arrival rates for 80 kJ.

because instantaneous localized forwarding incurs zero upstream queue accumulation.

- **Latency Boundedness of Hierarchical ILP:** In the data-intensive regime, `ILP-Hierarchical` models exhibit remarkable latency stability ($\approx$ 1.20–1.26 s in *Std*, $\approx$ 1.48–1.55 s in *VL-heavy*, and $\approx$ 1.05–1.12 s in *Pure L*) across all arrival rates, closely tracking distributed heuristics because the absence of complex CPU scheduling eliminates cluster-level queuing bottlenecks.

- **Fast-Dispatch Bounds and Energy Queuing Penalties:** Centralized fast-dispatch models (`BS20, BT0.05`, bars [6–7]) achieve the lowest response times at light load (0.90–1.05 s in *Std*, 0.85–0.95 s in *Pure L*), expanding moderately to 1.15–1.25 s under peak traffic. In contrast, the small-batch energy-minimizing configuration (`ILP-Centr [BS2, BT2, Energy]`, bar [9]) incurs a sharp latency inflation under peak traffic ($\lambda = 10$ req/s), reaching 1.65–1.75 s across all payload distributions due to extended queue dwell times and the selection of detour paths designed to avoid energy-depleted nodes.

5.2.3.3 Failure Diagnostics and Link Saturation

Figure 5.17 and Figure 5.18 display the diagnostic breakdown of task failure causes:

- **Dominance of Joint Link and Energy Exhaustion:** Exactly 100% **of heuristic rejections** and **over** 95–100% **of optimization rejections** are categorized as ***Insufficient CPU+NET*** (orange bars) across all arrival rates

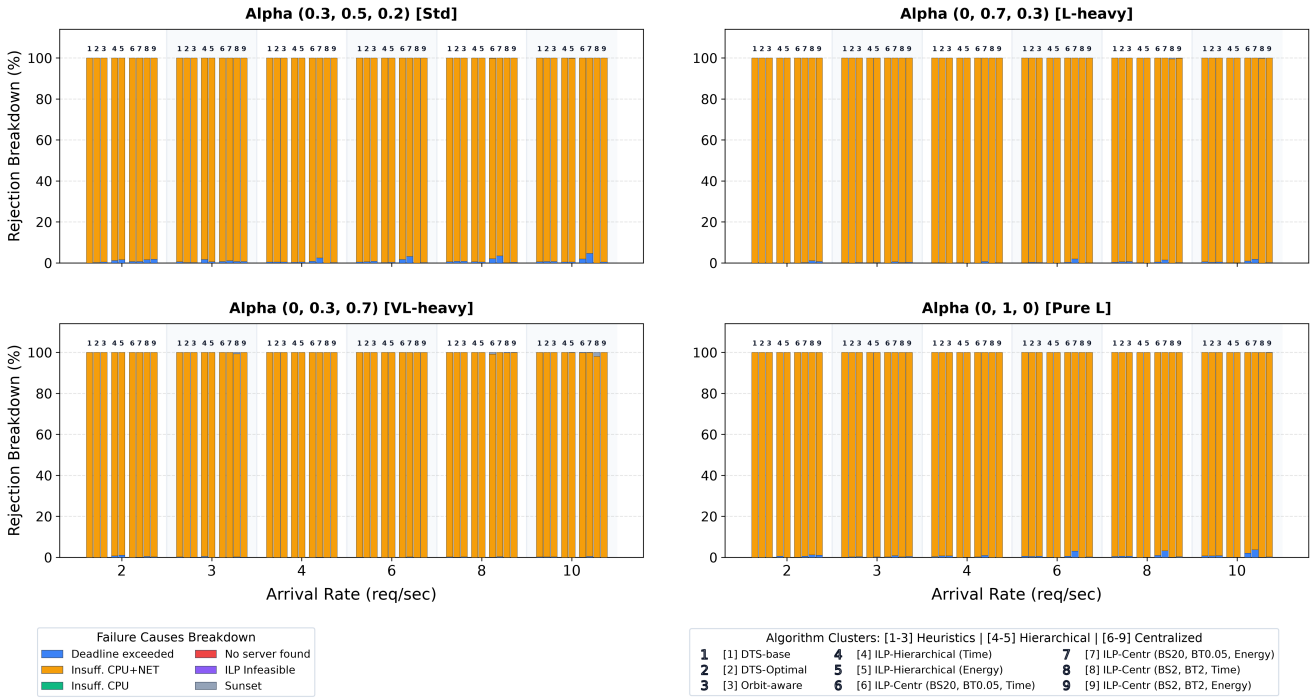

Figure 5.17. Rejection Causes distribution under Data-Intensive Workload $\boldsymbol{\beta} = (0, 0, 1)$ for 40 kJ across payload distributions $\boldsymbol{\alpha}$.

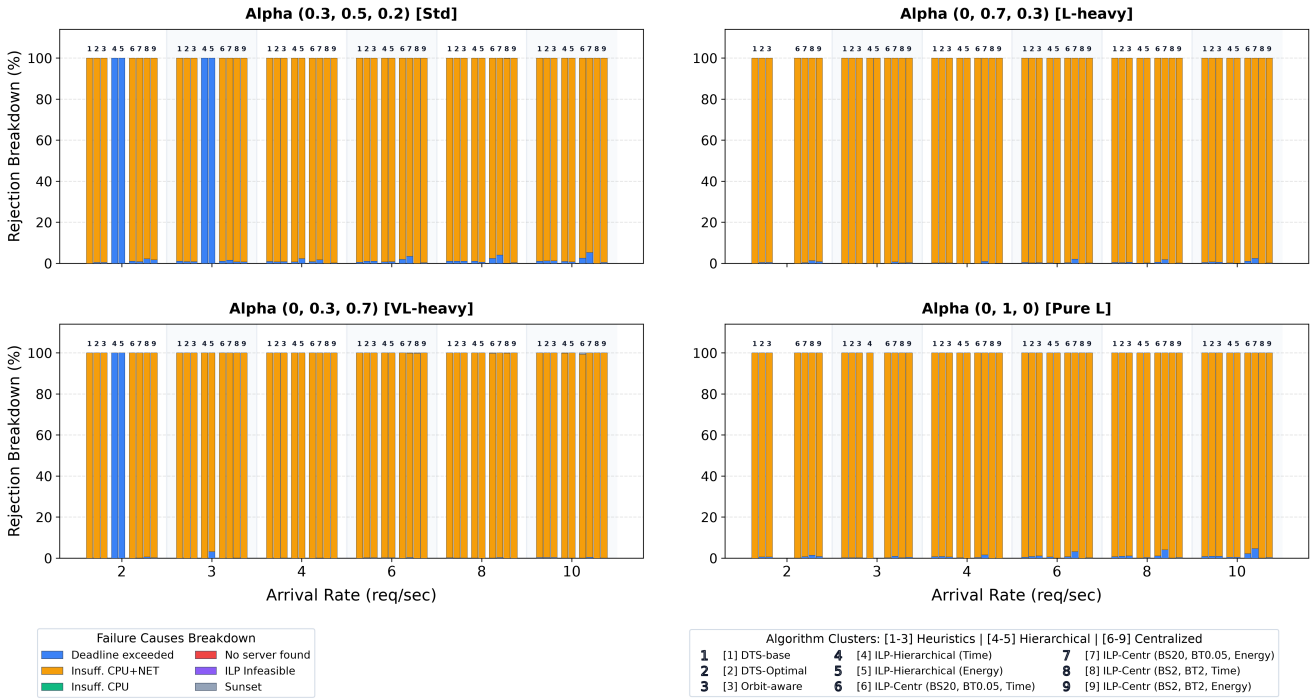

Figure 5.18. Rejection Causes distribution under Data-Intensive Workload $\boldsymbol{\beta} = (0, 0, 1)$ for 80 kJ across payload distributions $\boldsymbol{\alpha}$.

and payload distributions. In this regime, task drops occur strictly when cumulative optical ISL bandwidth or transceiver transmission energy exceeds available path capacity.

- **Total Disappearance of Pure CPU Bottlenecks:** Rejections attributed to *Insufficient CPU* (green) drop to **identically** 0% across all scenarios, confirming that system degradation is governed entirely by networking bandwidth and transceiver energy constraints.
- **Suppression of Queuing Timeouts and Normalization Artifacts:** Unlike the CPU-intensive regime, deadline failures (*Deadline Exceeded*, blue) are virtually eradicated, remaining below 2–5% across moderate-to-high arrival rates. An apparent anomaly occurs in `ILP-Hierarchical` at light load ($\lambda \leq 3$ req/s) under 80 kJ in *Std* and *VL-heavy*, where rejections appear as 100% *Deadline Exceeded*. Conversely, in *L-heavy* and *Pure L* at $\lambda = 2$ req/s, bars [4] and [5] are completely absent because the success rate is exactly 100% (zero drops). Where isolated failures occur under near-perfect throughput, a single sporadic deadline expiration accounts for the entirety (100%) of the normalized failure breakdown.
- **Absence of Orbital Eclipse Interruptions:** Rejections triggered by orbital line-of-sight loss (*Sunset*, grey) remain strictly at 0% throughout all evaluated scenarios.

5.3 Trade-Off Analysis and Operational Discussion

Correlating the empirical findings across the evaluated scenarios reveals three foundational operational trade-offs governing satellite task offloading.

5.3.1 Theoretical Global Optimality vs. Ingress Queuing Latency

The fundamental architectural dilemma of satellite orchestration lies in the trade-off between global combinatorial scheduling efficiency and ingress queuing latency:

- **Distributed Heuristics:** Operate on an immediate dispatching model with zero buffer aggregation. Requests arriving at an Access Point are evaluated instantaneously against local node states. This reactive posture yields exceptional resilience against heavy request arrival rates ($\lambda \geq 6$ req/s), preventing deadline expiration almost entirely ($< 2\%$ at high load). However, lacking global topological visibility, greedy localized forwarding induces inter-plane link saturation and transceiver depletion in data-intensive regimes.
- **Centralized Mathematical Optimization:** Solves an exact Mixed-Integer Linear Program over the constellation network [11, 12]. In compute-intensive and mixed regimes under Time mapping, configurations utilizing extended batch buffers ($BS = 20$) incur queuing delays that erode task deadline slack:

$$\Delta t_{slack} = D_r - (T_{wait} + T_{up} + W_{cpu,i} + T_{exec} + T_{down}) \tag{5.1}$$

When $\Delta t_{slack} < 0$, tasks expire inside the orchestrator queue before assignment can occur, causing high *Deadline Exceeded* drops (62%–80% in fast-dispatch under peak load, and up to 82%–90% in hierarchical models). In contrast, small-batching orchestrators ($BS = 2$) almost entirely avoid buffer dwell latency under heavy traffic, driving deadline drop rates down to $< 2\%$ in Mixed and 8%–14% in CPU-intensive profiles, shifting the operational failure boundary decisively back to physical compute exhaustion.

5.3.2 Energy Budget Sensitivity Across Dimensional Boundaries

The empirical evaluation demonstrates that fleet battery sizing (40 kJ vs. 80 kJ) exerts an asymmetric impact depending on workload dimensionality:

- **Compute-Bound Regimes ($\boldsymbol{\beta} = (0, 1, 0)$):** Shows complete invariance across payload size distributions $\boldsymbol{\alpha}$. Expanding battery capacity from 40 kJ to 80 kJ yields a moderate throughput increase (+4%–11%) across algorithms due to onboard CPU core energy consumption, but cannot resolve deadline expirations caused by queuing delays in large-batch setups.
- **Communication-Bound Regimes ($\boldsymbol{\beta} = (0, 0, 1)$):** Transmitting large payload files (20–50 MB) across multi-hop ISL paths demands substantial transceiver power [7]. Expanding battery reserves to 80 kJ unlocks throughput gains of +4% for heuristics, +4%–8% for centralized models, and up to +9%–15% for hierarchical optimization at peak load.
- **Mixed Multi-Service Regimes ($\boldsymbol{\beta} = (0.40, 0.35, 0.25)$):** Heterogeneous workloads exhibit intermediate sensitivity (+4%–12% throughput improvement under 80 kJ), with benefits concentrating primarily in optimization models that leverage multi-hop ISL routing.

5.3.3 Fast-Dispatch vs. Controlled Small-Batching Orchestration

Benchmarking the two tuned centralized optimization archetypes provides clear design boundaries regarding queue management:

- **Centralized Small-Batching (`BS2, BT2, Time`):** Consistently achieves the highest task completion rate among centralized schemes across all arrival rates and workload profiles (54–67% in Mixed, 46–58% in CPU-intensive, and 59–68% in Data-intensive at $\lambda = 10$ req/s). Under heavy load, the batch capacity of 2 fills almost instantaneously, eliminating buffer dwell time while enabling pairwise combinatorial scheduling that resolves resource contention far more effectively than single-task dispatch.
- **Centralized Fast-Dispatch (`BS20, BT0.05, Time`):** Enforcing an aggressive timeout ($BT = 0.05$ s) maintains the lowest baseline latency under light load (0.50–0.54 s in Mixed, 0.68–0.75 s in CPU, 0.85–1.05 s in Data). However, under heavy load, ingress queuing backlogs accumulate, causing it to admit fewer requests than small-batching (42–47% in Mixed and 29–33% in CPU-intensive, trailing small-batching by 12%–25%). This makes fast-dispatch

suitable primarily when strict low response latency under moderate traffic is prioritized over completion volume under saturation.

5.3.4 Architectural Synthesis and Deployment Guidelines

Synthesizing these trade-offs leads to prescriptive deployment guidelines for Orbital Edge Computing missions [5, 1, 2], summarized in Table 5.1.

Table 5.1. Prescriptive architectural deployment matrix for Orbital Edge Computing missions.

Operational Context	Recommended Paradigm	Primary Rationale
Compute-Intensive ($\boldsymbol{\beta} = (0, 1, 0)$, all λ)	Distributed Heuristics (`OrbitAware / DTS-Optimal`)	Zero ingress queuing; completely prevents deadline collapse by immediate greedy core allocation (72–95% completion rate, 100% CPU-bounded drops).
Data-Intensive ($\boldsymbol{\beta} = (0, 0, 1)$, all λ)	Hierarchical Optimization (`ILP-Hierarchical [Time]`)	Outperforms heuristics under sufficient battery (81–89% at peak load, up to 99–100% at light load).
Mixed (Throughput-Oriented) ($\boldsymbol{\beta}$ mixed, heavy load $\lambda \geq 6$)	Distributed Heuristics (`OrbitAware / DTS-base`)	Maximizes raw task admission rate (79–83%) by eliminating orchestrator ingress queue dwell times.
Mixed (Latency-Throughput Balanced) ($\boldsymbol{\beta}$ mixed, $\lambda \leq 4$ req/s)	Centralized Small-Batching (`ILP-Centr [BS2, BT2, Time]`)	Achieves high success rate (82–92%) and low latency (0.72–0.76 s), while naturally compressing to 0.56–0.65 s as traffic scales.

5.4 Summary of Chapter Findings

The comprehensive comparative benchmark across distributed heuristics, hierarchical ILP, and tuned centralized orchestrators establishes five definitive findings:

1. **Isolation of the Temporal vs. Physical Bottleneck:** In compute-heavy workloads ($\boldsymbol{\beta} = (0, 1, 0)$), failure profiles reveal an architectural batching split rather than an objective-driven one. Distributed heuristics fail strictly due to physical processor saturation (100% *Insufficient CPU*, 0% *Deadline Exceeded*). Centralized configurations with larger batch thresholds ($BS = 20$, bars [6] and [7]) as well as hierarchical formulations suffer extensive deadline expirations (62%–78% in fast-dispatch, and 82%–90% in hierarchical models at peak load) under sustained load, as ingress buffer dwell times deplete task slack. In sharp contrast, small-batching orchestrators ($BS = 2$, bars [8] and [9]) flush requests nearly instantaneously, shifting 86%–92% of rejections back to physical core saturation (*Insufficient CPU*) and restricting deadline violations to just 8%–14%. When transitioning to communication-heavy workloads

($\boldsymbol{\beta} = (0, 0, 1)$), this physical bottleneck shifts completely from CPU exhaustion (0% *Insufficient CPU*) to transceiver and link capacity (100% *Insufficient CPU+NET* for heuristics, and > 95–100% for optimization baselines).

2. **Dimensional Asymmetry of Energy Sensitivity:** Workload dimensionality governs sensitivity to satellite battery capacity. Compute-bound workloads ($\boldsymbol{\beta} = (0, 1, 0)$) are completely invariant to payload size distributions $\boldsymbol{\alpha}$, while benefiting moderately (+4%–11%) from larger battery capacity due to CPU power draw. Conversely, communication-heavy workloads ($\boldsymbol{\beta} = (0, 0, 1)$) exhibit high elasticity, where doubling onboard battery capacity recovers up to +9%–15% throughput in hierarchical models and +4%–8% in centralized configurations by sustaining transceiver power across multi-hop optical paths.

3. **Algorithmic Inversion in Data-Intensive Regimes:** Optimization models are not universally inferior to heuristics under heavy load. In data-intensive scenarios, `ILP-Hierarchical (Time)` decisively outperforms distributed heuristics (99–100% vs. 94–95% at low load, and 81–89% vs. 77–78% at peak load under 80 kJ).

4. **Bounded Nature of Physical Execution Time (T_{exec}):** Across all evaluated algorithms, arrival rates, and workload profiles, the physical processing time on destination SEN cores remains tightly bounded ($T_{exec} \approx 0.17$–0.48 s across heuristics, fast-dispatch, and small-batching, reaching at most 0.52 s in hierarchical models under mixed load and 0.65 s under data-intensive *VL-heavy*). End-to-end latency expansion is driven primarily by ingress queuing, cluster aggregation delays, and optical transmission serialization ($T_{net+wait}$).

5. **Marginalization of Eclipse Interruptions:** Task rejections due to orbital dusk (*Sunset*) are completely eradicated across all distributed heuristics and throughout the entire data-intensive regime (0.0%), and remain strictly marginalized (< 2–5%) in centralized optimization models under light load ($\lambda \leq 4$ req/s) where tasks dwell in queues across orbital plane visibility transitions. This confirms that predictive orbital ephemeris awareness effectively shields offloaded tasks from line-of-sight eclipse drops.

These empirical findings rigorously define the operational boundaries of centralized orbital orchestration, establishing the foundation for the concluding assessments and future research directions presented in Chapter 6.

Chapter 6

Conclusions and Future Work

This thesis investigated the operational feasibility, performance boundaries, and systemic trade-offs of centralized mathematical optimization for computational task offloading and multi-hop routing in Low Earth Orbit (LEO) satellite networks [1, 6]. By designing, formalizing, and implementing a Centralized Integer Linear Programming (ILP) Orchestrator coupled with an event-driven dynamic batching mechanism within a `SimPy` discrete-event simulation environment [13], this work systematically explored the viability of orbital edge intelligence under realistic spaceborne constraints [4, 6].

This concluding chapter synthesizes the primary findings and answers the core research question formulated at the outset of the thesis. Furthermore, it details the critical physical and architectural limitations inherent to centralized space computing and outlines promising future research directions toward resilient, hybrid orbital orchestration paradigms [5, 1, 3].

6.1 Synthesis of Contributions and Research Question Resolution

6.1.1 Answering the Core Research Question

In Section 1.3, this thesis posed the foundational research question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

Based on the multi-tiered experimental evaluations conducted across Chapters 4 and 5, the answer is clear: **the performance ceiling of centralized orbital orchestration is fundamentally architectural rather than computational**.

- **Computational Tractability of the Solver vs. Combinatorial Visibility:** When incoming requests are aggregated into bounded batches ($BS \leq 20$), exact branch-and-cut optimization using the open-source HiGHS solver exhibits remarkable computational efficiency, maintaining negligible mathematical resolution times ($T_{exec}^{ILP} \leq 0.30$–0.60 ms). However, attempting unbuffered single-task dispatch ($BT = 0.0$ s) fails to sustain high throughput under heavy workloads ($\lambda = 10$ req/s): solving the model for isolated requests ($|R| = 1$)

collapses the combinatorial search space into a myopic, single-task greedy allocation. Lacking joint spatial visibility over concurrent arrivals, unbuffered dispatch repeatedly steers closely spaced requests to the same locally optimal edge nodes, rapidly inflating localized onboard queues ($W_{cpu,i}$) and precipitating severe deadline-induced task drops (80%–85%).

- **Bounded Nature of Onboard Physical Task Execution (T_{exec}):** Across all evaluated algorithms, arrival regimes, and workload distributions, the physical execution time required by the destination Satellite Edge Node (SEN) CPU cores to process the assigned workload (T_{exec}) remains stable and strictly bounded ($T_{exec} \approx 0.17$–0.48 s, reaching at most 0.65 s under massive payload transfers). This invariance proves that system saturation is never caused by insufficient raw onboard computational speed on individual satellites.

- **Ingress Queuing Dwell Time as the Dominant Bottleneck:** System throughput collapse under elevated arrival rates ($\lambda \geq 6$ req/s) is governed almost exclusively by the ingress queuing latency (T_{wait}) accumulated in coordinator buffers prior to solver instantiation. In configurations relying on wide aggregation windows ($BS \geq 6$ or $BT \geq 0.4$ s), buffering dwell time systematically exhausts application deadline slack ($\Delta t_{slack} = D_r - [T_{wait} + T_{up} + W_{cpu,i} + T_{exec} + T_{down}]$), triggering cascade *Deadline Exceeded* dropouts (70%–85%) before tasks can be scheduled and dispatched.

- **Operational Batching Archetypes (Small-Batching vs. Fast-Dispatch):** To preserve application deadline margins while retaining combinatorial coordination, two distinct batching policies define the viable operating envelope:

  1. *Controlled Small-Batching ($BS = 2, BT = 2.0$ s):* Establishes the minimal non-trivial combinatorial optimization horizon ($|R| = 2$). Under sustained traffic ($\lambda \geq 4$ req/s), arrival bursts fill the two-request quota within 0.10–0.25 s, flushing batches almost instantaneously. This bounds deadline-induced rejections to $\leq 10\%$ in mixed workloads (8%–14% in compute-heavy stress) and recovers peak throughput under heavy load (54%–67% in mixed, 46%–58% in compute-bound).
  2. *Reactive Fast-Dispatch ($BS = 20, BT = 0.05$ s):* Enforces an aggressive 50 ms timeout, delivering the lowest baseline response latency under light load (0.50–0.54 s in mixed traffic). However, because it dispatches micro-batches with minimal combinatorial depth, candidate nodes suffer localized execution queue expansion under heavy load, trailing small-batching throughput by 12%–25%.

- **Calibrated Objective Weight Tuning and Path-Bloat Prevention:** The micro-tuning cross-sensitivity analysis identified the optimal multi-objective parameter set as $\alpha^* = 1000$ and $\gamma^* = 10$. Enforcing $\alpha = 1000$ under Time Mapping compels the branch-and-cut solver to prioritize transmission timeliness and deadline preservation, unlocking up to +19.5% higher completion rates over baseline weights. Concurrently, calibrating the orbital sunset penalty to $\gamma = 10$ completely suppresses eclipse-related drops (0.0%) while strictly

avoiding the severe path-bloat phenomenon observed when $\gamma \geq 1000$, which inflates end-to-end response time by over 55% (> 1.05 s) due to circuitous multi-hop detours.

6.1.2 Algorithmic Archetypes and Workload Sensitivity

The comparative evaluation against state-of-the-art distributed heuristics (`OrbitAware`, `DTS-base`, `DTS-Optimal`) and hierarchical optimization formulations demonstrated distinct operational boundaries governed by workload dimensionality:

- **Computation-Heavy Workloads ($\boldsymbol{\beta} = (0, 1, 0)$):** Pure computational stress exhibits complete invariance across data payload distributions ($\boldsymbol{\alpha}$). Distributed online heuristics demonstrate superior resilience, sustaining 72%–77% success at peak load ($\lambda = 10$ req/s) with 100% of rejections caused exclusively by physical core saturation (*Insufficient CPU*). In contrast, hierarchical models collapse (25%–30%) due to multi-tier aggregation delay. Among centralized schemes, controlled small-batching ($BS = 2$) successfully shifts 86%–92% of rejections back to physical CPU exhaustion, restricting deadline violations to just 8%–14% and out-performing fast-dispatch by up to +25% at peak load.
- **Communication-Heavy Workloads ($\boldsymbol{\beta} = (0, 0, 1)$):** In data-intensive regimes, an algorithmic inversion occurs: optimization decisively surpasses distributed heuristics. `ILP-Hierarchical (Time)` achieves near-perfect completion rates (99%–100%) under light load and sustains 81%–89% at peak load under 80 kJ, outperforming distributed heuristics (77–78%). Rejections across all schemes shift entirely to transceiver and link capacity (100% *Insufficient CPU+NET*), with pure CPU bottlenecks dropping to 0%. Furthermore, communication-heavy workloads exhibit high elasticity to constellation battery reserves: expanding battery capacity from 40 kJ to 80 kJ yields up to +9%–15% throughput recovery in hierarchical models.
- **Mixed Multi-Service Workloads ($\boldsymbol{\beta} = (0.40, 0.35, 0.25)$):** Heterogeneous multi-tenant scenarios reflect a fundamental operational trade-off between admission volume and latency control. When mission requirements demand raw task throughput, distributed heuristics maximize task admissions (79%–83% at peak load) by eliminating ingress queue accumulation. When centralized SLA coordination and deterministic multi-hop scheduling are required, calibrated small-batching (`ILP-Centr [BS2, BT2, Time]`) provides the only viable mathematical configuration, achieving 82%–92% success under moderate traffic and compressing its end-to-end response time to 0.56–0.65 s as arrival bursts saturate the batch boundary.

6.2 Critical Limitations of the Centralized Approach

Despite the mathematical rigor and global optimality demonstrated by the proposed framework, deploying a pure centralized orchestrator in operational mega-constellations involves structural limitations:

6.2.1 Ingress Buffering and the Batching Dilemma

Centralized mathematical scheduling requires aggregating incoming discrete requests into batches to instantiate the linear program [11, 12]. This creates an inescapable trade-off between optimization horizon and buffering latency. While larger batch sizes allow the branch-and-cut algorithm to uncover superior combinatorial placement patterns across the fleet, the static wait time T_{wait} directly consumes application deadline slack. For mission-critical real-time services characterized by tight sub-second latency bounds ($D_r \leq 500$ ms–1.0 s), any artificial queuing introduced before solver execution severely degrades service availability [2].

6.2.2 Single Point of Failure and Space Hazards

Relying on a designated orbital Master node introduces an acute Single Point of Failure (SPOF) [5]. Spaceborne edge computing operates in an exceptionally harsh environment subject to extreme thermal cycling, total ionizing dose (TID), and single-event upsets (SEUs) induced by cosmic rays and solar particle events [4, 5]. If the central Master satellite suffers a radiation-induced latch-up, an onboard computer reboot, an attitude control anomaly, or requires an autonomous orbital collision avoidance maneuver, the entire task scheduling and forwarding pipeline across the constellation is immediately paralyzed.

6.2.3 State Telemetry Overhead and Information Staleness

To formulate an accurate mathematical representation of constellation constraints, the orchestrator must maintain an updated global state matrix containing processor queue lengths ($W_{cpu,i}$), residual battery reserves ($E_{rem,i}$), and active ISL bandwidths ($B_{u,v}$). Transmitting this telemetry across dynamic inter-satellite links incurs control signaling overhead that scales with the number of satellites ($\mathcal{O}(|S|)$) [1].

Due to the extreme orbital velocities of LEO spacecraft (~ 7.5 km/s), continuous relative motion alters line-of-sight angles and link propagation delays [7]. Consequently, by the time state telemetry is gathered, buffered, and solved, the physical state of the constellation may have drifted. This state staleness risks rendering calculated schedules sub-optimal or infeasible mid-execution, necessitating reliance on emergency fallback routines.

6.3 Future Research Directions

The conclusions and limitations identified in this thesis pave the way for several natural extensions and advanced research trajectories.

6.3.1 Two-Tier Hybrid Orchestration Architectures

To reconcile the trade-off between centralized optimality and distributed responsiveness, future architectures should implement the prescriptive deployment matrix synthesized in Chapter 5 via a two-tier hybrid framework:

- **Tier 1 (Fast-Path Heuristic Layer):** Time-critical and compute-intensive requests ($\boldsymbol{\beta} = (0, 1, 0)$) are evaluated immediately at ingress access points using decentralized, zero-queuing heuristics (such as `OrbitAware` or `DTS-Optimal`), ensuring zero-buffering dispatch and eliminating deadline expiration.
- **Tier 2 (Global Optimization Layer):** Data-intensive tasks ($\boldsymbol{\beta} = (0, 0, 1)$) carrying large payload transfers are directed to ILP solvers [9, 15]. This tier exploits mathematical optimization where it demonstrably outperforms heuristics (81%–100% success), maximizing optical link utilization and preserving fleet battery energy.

6.3.2 Traffic-Aware Adaptive Batching via Reinforcement Learning

The static batching thresholds (BS, BT) evaluated in this work could be replaced by an intelligent, dynamic batching engine. A lightweight Reinforcement Learning (RL) agent—such as a contextual bandit or Q-learning controller—could be deployed at the ingress gateway to dynamically modulate the timeout threshold $BT(t)$ in real time [10]:

$$BT(t) = f\left(\hat{\lambda}(t),\, \min_{r \in Q}\{D_r - t_{arr,r}\},\, \bar{W}_{cpu}(t)\right) \tag{6.1}$$

By tracking traffic arrival surges ($\hat{\lambda}$) and the minimum residual slack time of buffered requests, the orchestrator could trigger immediate flushes during high-risk intervals and expand batch sizes during traffic lulls, optimizing the trade-off between scheduling efficiency and deadline preservation.

6.3.3 Multi-Agent Distributed Reinforcement Learning (MADRL)

To eliminate the centralized Single Point of Failure and bypass the communication overhead of global telemetry aggregation, decentralized Multi-Agent Deep Reinforcement Learning (MADRL) presents a compelling alternative [10]. Equipping each Satellite Edge Node with an autonomous actor-critic policy trained under Centralized Training with Decentralized Execution (CTDE) would allow satellites to make collaborative routing and scheduling decisions based solely on localized, one-hop neighborhood observations [10]. This model could approximate the global efficiency of ILP solutions while achieving the instantaneous decision speeds characteristic of distributed heuristics.

6.3.4 Hardware-in-the-Loop (HIL) Prototyping on Space-Qualified Hardware

Finally, extending this simulation-based validation toward physical implementation requires testing the optimization pipeline on physical embedded hardware architectures. Deploying the `HiGHS` solver [20] and the batching engine on radiation-hardened system-on-chip (SoC) architectures—such as the ARM Cortex-R52, Cortex-A53, or fault-tolerant space-grade RISC-V platforms (e.g., NOEL-V)—will provide empirical measurements of CPU instruction overhead, memory footprint, cache contention, and power consumption during real-time solver execution [5].

6.4 Concluding Remarks

The transition from passive bent-pipe relaying to active Orbital Edge Computing marks a significant evolution in space-based information systems [1, 4, 3]. This thesis demonstrated that centralized Integer Linear Programming is computationally viable on spaceborne processing platforms, keeping solver resolution times negligible under bounded batches while destination node physical processing times remain stable and compact ($T_{exec} \approx 0.17$–0.48 s). However, the temporal latency imposed by request buffering in ingress queues represents a fundamental physical boundary that limits the scalability of centralized scheduling under high-frequency traffic.

By quantifying the operational trade-offs between centralized mathematical optimization, distributed hierarchical solvers, and autonomous heuristics across diverse mission profiles, this work provides architectural guidelines for designing the next generation of resilient, energy-aware orbital computing systems [6].

Bibliography

[1] Y. Zhang, X. Li, H. Zhou, and D. Chen, "A comprehensive survey on orbital edge computing: Systems, applications, and algorithms," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2100–2135, 2023.

[2] I. Leyva-Mayorga, M. Martinez-Gost, M. Moretti, A. Pérez-Neira, M. Vázquez, P. Popovski, and B. Soret, "Satellite edge computing for real-time and very-high resolution earth observation," *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 6180–6194, 2023.

[3] S. Wang, Q. Zhang, C. Zhao *et al.*, "Satellite computing: Vision and challenges – Tiansuan constellation," *IEEE Internet of Things Journal*, vol. 10, no. 20, pp. 17 550–17 563, 2023.

[4] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite clusters as accelerators for earth observation," *IEEE Micro*, vol. 40, no. 1, pp. 13–21, 2019.

[5] ——, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 939–954.

[6] E. Casalicchio *et al.*, "Handling of energy- and time-constrained heterogeneous satellite computing services," in *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2025, pp. 1–12.

[7] Q. Chen, G. Giambene, L. Yang, C. Fan, and X. Chen, "Analysis of inter-satellite link paths for LEO mega-constellation networks," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 3, pp. 2743–2755, 2021.

[8] A. U. Chaudhry and H. Yanikomeroglu, "Laser intersatellite links in a Starlink constellation: A classification and analysis," *IEEE Vehicular Technology Magazine*, vol. 16, no. 2, pp. 48–56, 2021.

[9] L. Zhong, Y. Li, M.-F. Ge, M. Feng, and S. Mao, "Joint task offloading and resource allocation for LEO satellite-based mobile edge computing systems with heterogeneous task demands," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 7, pp. 11 337–11 352, 2025.

[10] W. Fan, Q. Meng, G. Wang, H. Bian, and Y. Liu, "Satellite edge intelligence: DRL-based resource management for task inference in LEO-based satellite-

ground collaborative networks," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10 710–10 728, 2025.

[11] L. A. Wolsey, *Integer Programming*. New York, NY, USA: John Wiley & Sons, 1998.

[12] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004.

[13] S. Scherfke and O. Lünsdorf, *SimPy: Discrete Event Simulation for Python*, 2023, version 4.1.1. [Online]. Available: https://simpy.readthedocs.io/

[14] S. Mitchell, M. O'Sullivan, and A. Dambrosio, "Pulp: A linear programming toolkit for Python," The University of Auckland, Tech. Rep., 2011, version 2.8.

[15] Q. Jiang, L. Zheng, Y. Zhou, H. Liu, Q. Kong, Y. Zhang, and B. Chen, "Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 63, pp. 1–19, 2025.

[16] J. G. Walker, "Satellite constellations," *Journal of the British Interplanetary Society*, vol. 37, pp. 559–571, 1984.

[17] A. Neumann, C. Aichele, M. Lindner, and S. Fux, "Better approach to mobile ad-hoc networking (B.A.T.M.A.N.)," IETF Draft, Tech. Rep., 2008, draft-wunderlich-openmesh-manet-routing-00.

[18] X. Yang, H. Dai, Y. Tang, and X. Yi, "A general metric of load balancing in δ-range," in *Proceedings of Advanced Parallel Processing Technologies (APPT)*. Springer, 2003, pp. 311–321.

[19] P. Peng, T. Xu, X. Chen, C. C. Zarakovitis, and C. Wu, "Blocked-job-offloading-based computing resources sharing in LEO satellite networks," *IEEE Internet of Things Journal*, vol. 12, no. 2, pp. 2287–2290, 2025.

[20] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[21] C. Bagini, "Centralized ILP Orchestrator for Satellite Computing," https://github.com/baggio2003/Centralized-Scheduling-in-Satellite-Computing, 2026, gitHub repository.

[22] D. Vallado, P. Crawford, R. Hujsak, and T. S. Kelso, "Revisiting spacetrack report no. 3: Rev 2," in *AIAA/AAS Astrodynamics Specialist Conference and Exhibit*, 2006, p. 6753.

[23] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[25] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

[26] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

Acknowledgments

We have finally reached the end of this seemingly endless journey. It is still hard to believe, but four whole years have truly gone by since it all began. It feels like only yesterday when I graduated from high school and decided to embark on this new adventure. This path brought about so many changes in me; it allowed me to grow, mature, and deepen my knowledge. It wasn't always a walk in the park, but somehow we have finally made it to the finish line. Or almost...

Now, however, begins the hardest part of the entire thesis: writing the acknowledgments. I am not particularly comfortable with this sort of thing, but I truly care about thanking everyone who accompanied and supported me along the way; without them, I wouldn't have made it to this point. Therefore, I ask you, the readers, to read these lines with patience and understanding. Even though they come straight from the heart, words can hardly capture everything I wish to say. I would ask you to read only the section addressed to you, but knowing some of you, you will probably read the whole thing anyway. So please, just don't make me feel any more embarrassed than I already am. Thank you.

First and foremost, I want to thank my family. To my parents: my mother, who has always supported and encouraged me, and my father, who, despite some disagreements, has always stood by me and pushed me to do better. You have given me so much, while I have given back so little, and that is something I will forever carry in my heart. Thank you for everything. Do you remember, Mom, helping me review history? Or art? Neither of us ever enjoyed them, yet you were always there helping me study. The days when you could help me with my coursework are long gone, but I will never forget everything you did for me. And you, Dad? Do you remember high school? Every year on the first day of school, you wanted me to wait so you could take a photo of me heading out. It has been four years since you last took one of those photos. Take one now, at this finish line, to remind yourself how much I have grown, and how much you helped me grow along the way.

I want to thank my grandparents for watching over me all these years, even though I am probably responsible for quite a few of those gray hairs on your head, right Grandma? Now that this chapter of my life is coming to an end, what do you think about celebrating? Maybe with a big plate of polpette al sugo? This time, I promise I'll help you make them.

Brother... according to what everyone tells me, I am supposed to include you here as well. I don't really know what I should be thanking you for, but I know that somehow you helped me grow, even if I'm not entirely sure how, but whatever. What I am sure of is that, as an older brother, you paved the way for me in so many things, as you well know, and for that I truly thank you. It took us quite a few years before we could even tolerate each other, but I actually don't mind the alliance we have going on right now. Don't let it go to your head, though.

Next, I want to thank my friends, who stood by me throughout this entire journey and helped me stay sane.

First of all, I want to thank Luca and Mattia, who have been close to me ever since middle school. Luca, who would have thought that after twelve years we would still be in touch? It feels like an eternity since we first met. Every now and then I still look back at how I was when we met, and I am honestly amazed you stayed

friends with me for so many years. Not only did we keep that bond alive, but you are one of the people I feel most comfortable with, someone who saw me at my absolute worst, just remember the 'Jägermeister' incident :), and who always listened and supported me. I only hope I have been as good of a friend to you as you have been to me.

Mattia, I don't know how well you remember it, but back in middle school we honestly didn't get along all that well, and I never would have believed that after so many years we would become such great friends. Yet here we are. Throughout all these years you were always there for me and supported me, and those walks of ours helped me so much on several occasions. Whenever I had doubts and had no idea what to do, I would always turn to you, and you would listen and give me invaluable advice. For that, I can never thank you enough.

I also want to thank my high school crew: Francesco, Alessandro, Alessio (better known as Domiziano), Giulio, and Martina. You accompanied me through a chapter of my life that I will always look back on with great affection, from the hours spent in the classroom and the countless laughs, to the desperate nights cramming for exams and all the time spent together. We certainly had our share of friction along the way, but the fact that we are still here after nine whole years says everything about what we built.

What makes this group so special to me is how naturally all our different worlds fit together. Whether it is having Francesco to thank for introducing me to anime and rock and metal music, alongside our shared passion for FromSoftware games, or being glad that Alessandro is holding the line with me as the only other computer scientist of the bunch. It is looking forward to finally screaming along at a Sleep Token concert with Alessio, knowing that with Giulio nine years of great evenings and memories speak for themselves without needing any specific hobby attached, and knowing that even if dragging Martina out of the house is an achievement in itself, she will always be just as important to me. Thank you for all these years, and here is to many more.

I also want to thank two people who, despite the distances, have stayed close to me throughout these years: Lara and Alice.

Lara, we clicked almost instantly back in high school; after barely knowing each other for a few months, we were already spending hours on the phone deep into the night. It has been six years since you moved abroad, and with your recent move to Australia, seeing each other has become an even bigger logistical challenge (and just so you know, I feel terribly guilty every time you tell me to come visit!). Yet, despite the distance, we still talk every single week as if no time has passed. You have always stayed close to me through it all, someone I could always confide in, and you remain one of the people I feel closest to in my life. Who knows whether this time you will actually settle down permanently there in Australia, or if you will get bored once again and change countries. In the meantime, I will be right here whenever you want to come visit.

Alice, even though we first met back in our first year of high school, it wasn't until our third year that we actually started talking. It happened completely at random, and back then, I never could have imagined we would end up having such a close

bond.

Between all the memories we made together, our trips, the concerts we shared, and all the ones we still have planned, we have experienced so much. Yet we also went through our darkest times, and that year and a half of complete silence was painful. Finding the courage to start over was terrifying and full of hesitation, but putting our pride aside was the best decision we could have made, and I truly hope you are just as glad that we reconnected as I am. And yes, I still have to apologize for inevitably dropping halfway through every single book you ever recommended to me. Today, you are truly one of the people I feel most comfortable with, and having you in my life and being able to talk to you means more than words can say. Thank you for proving that what we built was always worth fighting for.

Then comes my university chapter, which I definitely could not have survived without Alessio, Andrea, Davide, and Allegra. Alessio and Andrea were by my side from day one, while Davide came along to complete the squad, and together we somehow dragged each other through all these chaotic years.

Andrea, let's be honest, I definitely wouldn't be crossing this graduation finish line today without our mutual survival pact. From that insane gamble of pulling two exams out of nowhere in just two weeks (sure, they were only IUM and OGA, but props to us anyway), to the months of pure agony and four attempts it took to finally beat Databases 2, we survived every bit of it together. I might be stepping across the line first, but your turn is right around the corner. Thank you for dragging me into Magic: The Gathering, even if my wallet will never truly forgive you for it. And a huge thank you to Allegra for the Magic games, for our essential gossip sessions, and for graciously lending us your boyfriend and letting us kidnap him and drive him crazy whenever we needed him. To all of you, thank you for sharing the dread of every exam season, the sweat, and the countless laughs along the way.

I also have to give a huge shoutout to my Discord crew: Miriana, Bengu (allegedly known to some as Davide), and Gabriel. What started completely at random through Destiny and mutual friends turned into countless late night gaming sessions and calls that kept me company over the years, especially during the lockdowns. We might not get to see each other in person very often, but we are living proof that genuine, lasting bonds can truly be built online. Knowing that I could just put my headset on after an exhausting day and find you guys there made all the difference in the world. Thank you for all the laughs, the venting sessions, and for always being around.

Everything I have written so far is honestly only a fraction of what I truly think and feel, but before this section gets endlessly long and hopelessly cringe, it is probably best to draw the line here.

Finally, I have to thank myself. To be completely honest, there were plenty of moments when I just wanted to give up and throw in the towel. Somehow, I stayed stubborn enough to see it through, and looking back right now, I am genuinely proud of what I managed to achieve. I really prefer not to think about the Master's degree looming ahead just yet, but for now, I survived.

P.S. Just to put it permanently on the record for all my friends studying physics who never miss an opportunity to tease me: Computer Science is still definitely better.